

KAMP 제조AI데이터셋 활용 권한

No. GB20251026-PU1760458278-262829



⚠ 제조AI데이터셋 활용 주의사항

연구/공식적으로 사용하실 때에는 꼭 아래와 같이 'KAMP 출처(reference)'를 남겨주시고, 인용 시 활용한 내용과 문서 등은 아래 이메일로 보내주시기를 바랍니다.
(E-mail : kamp@kaist.ac.kr)

국문 출처 표기 양식

중소벤처기업부, Korea AI Manufacturing Platform(KAMP), 사출성형 공급망 최적화 AI 데이터셋, KAIST(울산과학기술원, (주)에이비에이치), 2022.12.23., www.kamp-ai.k

영문 출처 표기 양식

Ministry of SMEs and Startups., and KAIST (Korea Advanced Institute of Science and Technology). (2022, December 23). Injection molding supply chain optimization AI dataset. Korea AI Manufacturing Platform(KAMP). <https://www.kamp-ai.kr/>

제조AI데이터셋 개요

제조AI데이터셋명 : 사출성형 공급망 최적화 AI 데이터셋

업종 : 사출성형 | **목적** : 공급망 최적화 | **유형** : csv

알고리즘 : 1차원 합성곱 신경망 (One-dimensional Convolutional Neural Network, 1D CNN)

사용조건 : 콘텐츠 변경 허용

제공기관 : KAIST (수행기관 : 울산과학기술원/(주)에이비에이치)

등록일 : 2022-12-23

이용자 정보

회원 ID : junseok0913

다운로드일 : 2025-10-26 20:55:01

활용목적 : 제조AI 솔루션 개발 및 고도화

인공지능 제조 플랫폼(KAMP, Korea AI Manufacturing Platform)에서 다운로드한 제조AI데이터셋의 저작권을 위하여 문서 고유 번호와 워터마크를 부여하여, 해당 문서의 표준을 준수하였음을 증명합니다.

운영기관 : KAIST 제조AI빅데이터센터

주소 : 대전광역시 유성구 문지로 193 KAIST 문지캠퍼스 행정동

전화번호 : 042-350-1331 | 이메일 : kamp@kaist.ac.kr

「사출성형 공급망최적화 AI 데이터셋」

분석실습 가이드북



「사출성형 공급망최적화 AI 데이터셋」

분석실습 가이드북





Contents

1 분석요약	04
2 분석 실습	
1. 분석 개요	05
1.1 분석 배경	05
1) 공정(설비) 개요	
2) 이슈사항(Pain point)	
1.2 분석 목표	09
1) 분석 목표	
2) 데이터 정의 및 소개	
3) 데이터 분석 기대효과	
4) 시사점(implication) 요약기술	
2. 분석실습	10
2.1 제조데이터 소개	10
1) 데이터 수집 방법	
2) 데이터 유형/구조	
3) 데이터 (품질) 전처리	
2.2 분석 모델 소개	13
1) 데이터 흐름 및 AI 분석모델 적용 흐름도	
2) AI 분석모델	
2.3 분석 체험	20
1) 필요 SW, 패키지 설치 방법 및 절차 가이드	
2) 분석 단계별 프로세스 - Flow Chart	
[단계 ①] 라이브러리/데이터 불러오기	
[단계 ②] 데이터 탐색	
[단계 ③] 데이터 정제(전처리)	
[단계 ④] 주요 변수 선택	
[단계 ⑤] 모델의 입력 데이터 형태 설정	
[단계 ⑥] 학습, 검증, 평가 데이터셋 분리 및 스케일링	
[단계 ⑦] 모델 구축 및 학습	
[단계 ⑧] 모델 성능 확인	
[단계 ⑨] 분석결과에 대한 논의 및 해석	
3. 유사 타현장의 분석 적용	50
부 록	
분석환경 구축을 위한 설치 가이드	52
데이터 품질 전처리 [실습 코드]	65



「사출성형 공급망최적화 AI 데이터셋」

분석실습 가이드북

- 필요 SW : Python, Anaconda – Jupyter Notebook
- 필요 패키지 : Pandas, NumPy, Matplotlib, scikit-learn, TensorFlow
- 분석 환경 : [CPU] Intel i5 3.1 GHz, [RAM] 40GB
- 필요 데이터 : 사출성형.csv

1 분석요약

No	구분		내용
1	분석 목적 (현장 이슈, 목적)		- 제조업의 특성상 여러 기업이 네트워크로 이어져 있기 때문에 다양한 기업과 연계하여 물품을 수주 및 발주하는 특징을 가진다. 따라서, 여러 기업의 상황에 따라 변동적인 수주 및 발주에 대한 대처와 생산 공정을 위한 원자재 수급 등에 대한 대처가 유연할 필요가 있다. - 사출성형 공정의 특성상 다양한 종류의 품목을 생산할 뿐만 아니라, 온도 및 습도 등의 외재적 변수의 영향을 받는 생산 능력으로 인해 발주, 수주, 생산량 등이 영향을 받게 된다. - 따라서 생산성 향상 및 효율적 재고관리를 위해 발주 및 생산 계획 데이터를 분석하고 AI 기반의 예측 모델을 구축함으로써 효율적인 생산량 설정 및 공정 계획 수립을 수행할 수 있다.
2	데이터셋 형태 및 수집방법		- 분석에 사용된 변수명 : Product_Number, 'T일 예정 수주량', 'T+X일 예정 수주량', '작년 T일 예정 수주량', '작년 T+X일 예정 수주량', 'T일 예상 수주량', 'T+X일 예상 수주량', DateTime, DoW, Temperature, Humidity - 수집 방법 : 생산업체의 Internet Data Center 내의 생산 계획 데이터 - 데이터셋 파일 확장자 : .csv
3	데이터 개수 데이터셋 총량		- 데이터 개수 : Row 수 (34,617개), Column 수 (20개) - 데이터셋 총량 : 4 MB
4	분석적용 알고리즘	알고리즘	1차원 합성곱 신경망 (One-dimensional Convolutional Neural Network, 1D CNN)
		알고리즘 간략소개	일반적으로 이미지 등의 데이터 내 패턴을 찾는 데에 유용하다고 알려진 합성곱 신경망 (Convolutional neural network, CNN)은 시계열 특성을 지닌 데이터를 활용한 예측 및 분류 목적으로도 활용된다. 모델 내 합성곱 필터의 이동 방향의 수 혹은 형태에 따라 1차원, 2차원, 3차원 합성곱 신경망으로 분류되며, 주로 입력 데이터의 형태에 따라 다르게 사용된다. 본 분석에서는 시계열적 특성을 지닌 데이터 처리를 위해 1차원 합성곱 신경망을 활용하며, 이는 적은 연산량과 빠른 처리 속도를 가진다는 장점이 있다.
5	분석결과 및 시사점		- 1차원 합성곱 신경망을 활용하여 데이터의 시계열적 특성 및 변수간 관계를 학습하여 미래 수주 수량을 예측한다. - 여러 생산품의 다양한 시점별 수주 수량 및 생산 계획 수량을 통합적으로 고려하여 미래 수량을 예측했을때 보다 정확하게 예측이 가능하다.



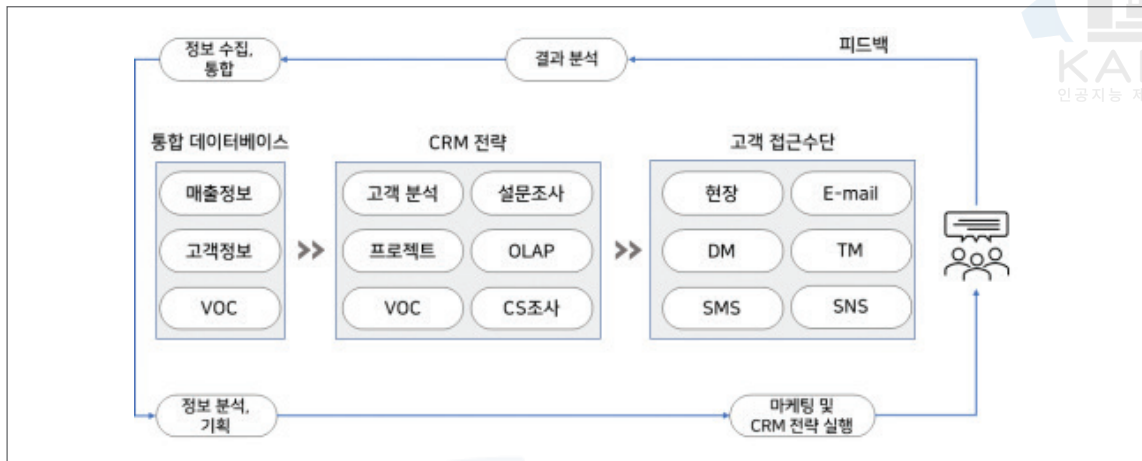
2 분석 실습

1. 분석 개요

1.1 분석 배경

1) 공정(설비) 개요

- 해당 기업은 볼트류 및 너트류를 제조하는 회사로 다양한 기업과 연계하여 물품을 수주 및 발주하고 공정을 진행하고 있으며, CRM을 기반으로 Supply Chain을 관리하고 있다.
- 볼트 및 너트는 소재·기판산업과 제조·서비스업 사이에서 주요 산업분야로의 판매를 수행하는 산업간 가치사슬의 핵심 연결고리로, 다양한 판매처를 가져 복잡한 공급망을 가지게 된다.
- 해당 기업은 볼트, 너트류를 직접적으로 활용하는 제조업 분야의 기업들과 볼트, 너트류 유통 공급망을 형성하고 있으며, 이에 대한 네트워크 형태의 데이터를 보유하고 있다.
- **Customer Relationship Management (CRM)**
 - CRM은 기업에서 고객 관리를 관리하기 위한 방법론 또는 전략으로, 기업의 영업/마케팅과 관련된 프로세스와 고객 데이터베이스를 체계화하여 관리하는 방법론이다.
- **CRM의 활용**
 - 고객과 나누었던 커뮤니케이션 이력 등 고객에 관계된 다양한 정보를 포함하며, 해당 정보를 바탕으로 고객을 만족시켜 새로운 매출을 이끌어내기 위한 도구로 CRM 시스템을 활용한다.
 - 누가 언제 무엇을 얼마나 구입했는지에 대한 고객 정보와 이전 문의사항, 클레임 이력 등을 저장하고, 이를 바탕으로 빠르고 정확하게 응대한다.
 - 고객에게 기존 보유상품과 관련된 상품을 구매하도록 하거나(Cross-Sell), 고객에게 조금 더 개선된 상품을 구매하도록 하는(Up-Sell) 경우, 과거 요구사항이나 커뮤니케이션 이력 등을 분석하여 구매 확률을 분석하기 위해 어떤 방식을 활용해야 할지 알 수 있다.



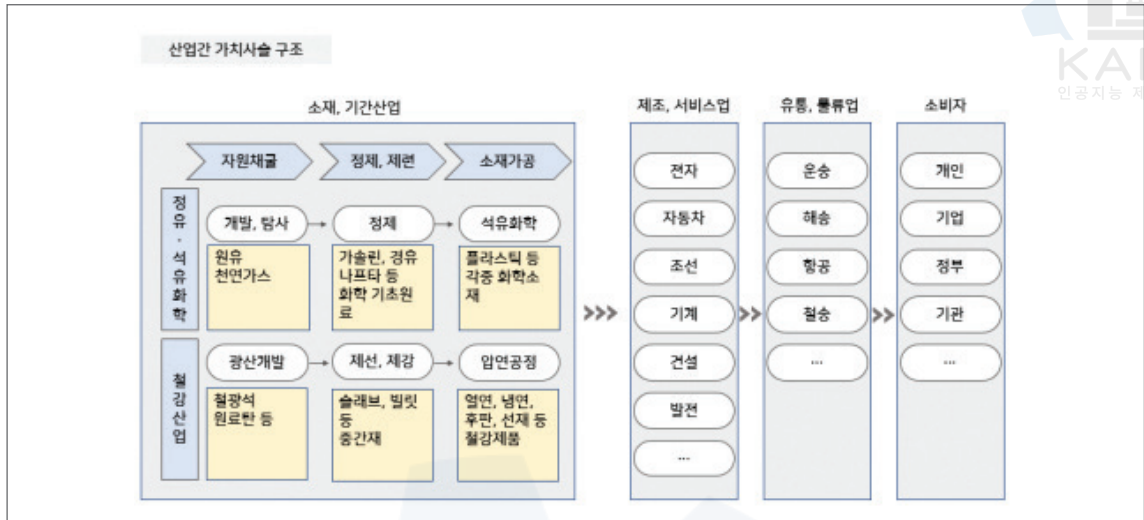
[그림 1] CRM 워크플로우

• CRM의 장점

- 마케팅 및 CS업무에 있어서, 고객의 수요에 대응한 전략 수립이 가능하다.
- B2B(기업간 거래, Business to Business) 뿐만 아니라 B2C(기업과 소비자의 거래, Business to Customer), B2G(기업과 정부의 거래, business-to-government) 등에도 적용될 수 있으며, 신규 시장 또는 고객의 발굴에 유리하다.
- 시장 및 고객의 맥락을 파악하여 기업의 비전 및 비즈니스 방향성 수립을 위한 목적으로 활용될 수 있다.
- 고객 데이터 또한 기업의 고유 자산으로, 이에 대한 활용 방안 확보 및 데이터를 통한 가치 창출을 수행할 수 있다.
- 세일즈/마케팅 등 GTM 활동과 운영에 소요되는 비용을 효과적으로 감소시킬 수 있다.

• CRM의 한계 및 단점

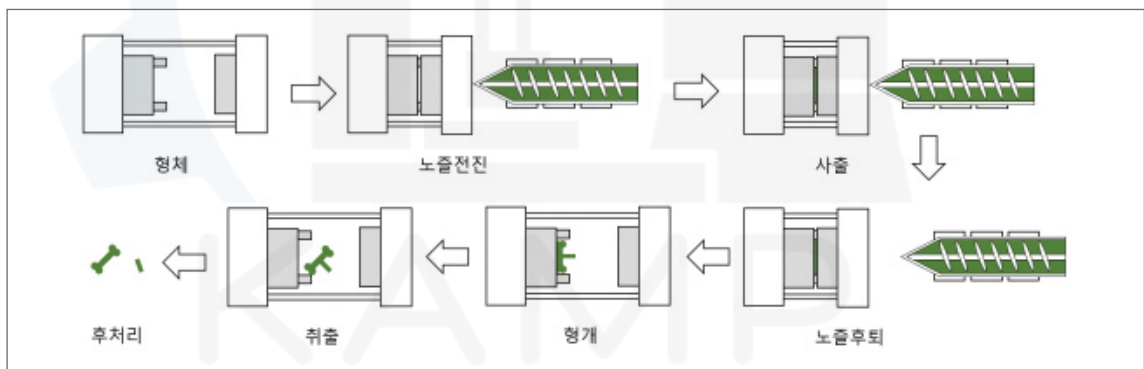
- 기존에 데이터가 존재하지 않을 경우, 얻을 수 있는 이점이 전무하므로 기존 운영 과정에서 해당 데이터를 충분히 수집하여야 한다.
- 고객과 관련된 정보, 영업활동을 중심으로 특화되어 있어 타 시스템과의 연계성을 확보하지 못할 시 별도의 서드파티 시스템으로 작용할 가능성이 높다.
- 시스템 활용 결과에 대한 정량적 영향 평가 및 가치 분석이 어렵다.



[그림 2] 산업간 가치사슬 구조

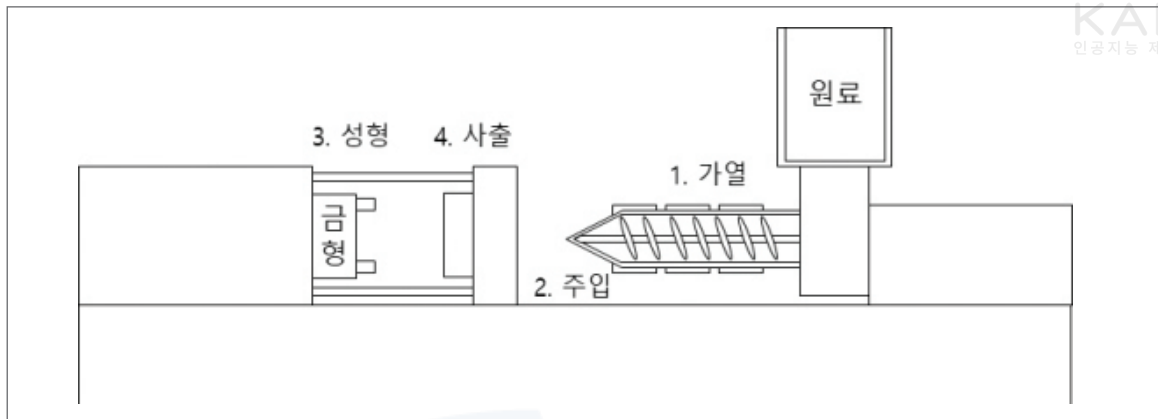
• 사출성형 공정

- 사출성형 공정은 금형을 이용한 성형법 중 하나로, 합성 수지(플라스틱) 등의 재료를 가열하여 녹이고, 금형에 주입한 뒤 냉각시켜 원하는 성형을 실행한다.
- 가공 과정은 재료의 가열, 주입, 냉각, 사출, 후가공의 순서로 이루어진다.
- 복잡형상을 포함하여 다양한 형태의 부품을 연속하여 대량으로 생산하기에 적합하여 다양한 분야에서 폭넓게 활용된다.



[그림 3] 사출성형 공정흐름도

• 사출성형 설비



[그림 4] 사출성형 설비 개요

- 사출성형 장비는 사출물의 형상을 성형하는 금형부와 원료의 가열 및 주입을 수행하는 주입부로 크게 나눌 수 있다.
- 해당 설비에서 설비 자체의 상태를 관리하기 위한 부위별 온도와 진동, 전력 데이터 등이 활용될 수 있다.
- 공정 품질에 대한 데이터는 금형의 형상과 금형 온도, 주입 속도, 모터 RPM, 가열 온도 등이 품질과 관계된다.

2) 이슈사항(Pain point)

• 공급망 상의 문제 현황

- 수주 및 발주 내역에 맞추어 생산이 이루어지는 제조업의 특성상 네트워크로 연결된 기업의 상황에 따라 수 발주량이 민감하게 변동한다. 이러한 문제에 대처하기 위해 수 발주처에 대한 대처를 유연화하고 공정 원자재 수급 탄력성을 증가시킬 수 있는 방안이 필요하다.
- 현재 수주량에 대한 분석이 제대로 진행되고 있지 않아 수주 예측이 안되고 있는 상황이며, 이에 따라 품목별로 재고가 많이 남는 경우가 발생하고 있다. 또한, 재고가 남는 만큼 필요없는 공정이 추가로 진행되는 것이기 때문에, 효율적인 공급망 구축을 위해서는 수주량 예측을 통해 재고량을 줄일 필요가 있다.

• 문제 해결 장애요인

- 현재, 기존 네트워크 내 존재 업체들에 대한 데이터 분석이 이루어지지 않아 잠재적 위험요인의 도출과 특이사항 발생 시 대처 방안의 마련이 이루어지지 않은 상황이다.
- 재고량에 대한 데이터는 있으나, 수주량에 대한 데이터가 없어 정확한 Supplychain



데이터 분석이 어려운 상황이며, 이에 따라 재고량을 낮추는 방향으로 데이터 분석은 가능하다. 즉, 수주량 예측을 정확하게 할 수 없다.

• 극복 방안

- AI 알고리즘을 활용하여 데이터 분석을 진행하고 특이사항이 발생할 경우 유연하게 수주, 발주 및 원자재 공급을 할 수 있도록 정보를 제공하고 공정 계획을 세울 수 있도록 가이드 할 수 있는 시스템을 구축하고자 한다.
- 우선적으로 재고량이 줄어드는 양을 통계를 내어 수주량에 대한 1차적인 예측을 진행하고 이에 맞추어 생산 계획을 세울 수 있도록 플랜을 잡는다면 어느 정도는 정확도 확보가 가능할 것으로 보인다.

1.2 분석 목표

1) 분석 목표

- 원자재 및 중간재 수급부터 완제품 공급 및 유통망 구조까지 포함한 데이터셋에 대하여, AI 알고리즘 기반의 데이터 분석을 진행하고 탄력적 대응이 필요한 특이사항을 검출하고자 한다.
- 재고량 데이터를 통해 평소 수주량을 분석하고 이를 통하여 평균 수주량을 예측하고 품목별 불균형 해결 및 효율적인 장비 가동을 하고자 한다.

2) 데이터 정의 및 소개

- 해당 분석에서 사용되는 공정 데이터들은 실제 22년 1월 26일부터 22년 5월 11일까지의 유통 데이터이며, 제품 종류에 따른 T일 예정 수량을 활용한다.

3) 데이터 분석 기대효과

- 데이터 분석을 통해 유통 구조에 위험요소가 될 수 있는 특이사항을 검출하고 조기 대응함으로써 공급망의 안정성 확보할 수 있음을 기대한다.
- 복잡한 유통 구조에 대해 맞춤형으로 유연하게 수주, 발주 및 원자재 공급을 할 수 있도록 정보를 제공하고, 이를 기반으로 효율적인 공정 계획을 세울 수 있을 것으로 판단된다.



4) 시사점(implication) 요약기술

- 1차원 합성곱 신경망을 활용하여 데이터의 시계열적 특성 및 변수간 관계를 학습하여 미래 수주 수량을 예측할 수 있다.
- 여러 생산품의 다양한 시점별 수주 수량 및 생산 계획 수량을 통합적으로 고려하여 미래 수량을 예측했을 때보다 정확한 예측이 가능하다.

2. 분석실습

2.1 제조데이터 소개

1) 데이터 수집 방법

- 제조 분야 : 사출성형
- 수집 장비 : 원자재 및 중간재, 완제품 유통 데이터
- 수집 기간 : 2022년 01월 26일 ~ 2022년 05월 11일
- 수집 주기 : 1일

2) 데이터 유형/구조

- 데이터셋 구조 : 테이블 형식(tabular)
- 데이터 개수 : column 수 20개, row 수 34,617개, 데이터수 692,340개
- AI 데이터셋 주요 변수(속성) 정의

데이터셋에는 일자, 시간 및 부품 식별자(비식별화됨)와 다양한 수주량 정보가 포함되어 있으며 주요 변수는 아래 표와 같다. 수주 수량 관련 변수를 독립 변수로, T일 예정 수주량을 종속 변수로 활용할 것이다.

속성(column)	설명	데이터형	개수
Product_Number	제품 종류)	string	34,617
T ~ T+4일 예정 수주량	예정 수량(수주량)	int	34,617
작년 T ~ T+4일 예정 수주량	작년 수량(수주량)	int	34,617
T ~ T+4일 예상 수주량	기업에서 측정한 예상 수량(수주량)	int	34,617
DateTime	측정 시 시간을 초 단위까지 기록(H:MM:SS)	object	34,617
DoW	요일	object	34,617
Temperature	온도	float	34,617
Humidity	습도	float	34,617



- 주요 변수 기술 통계

속성(column)	데이터형	개수	평균	표준편차	최소값	중앙값	최대값
T일 예정 수주량	int	34617	101.468	168.004	0.0	2.0	30.0
T+1일 예정 수주량	int	34617	67.014	131.213	0.0	0.0	5.0
T+2일 예정 수주량	int	34617	48.758	115.2	0.0	0.0	0.0
T+3일 예정 수주량	int	34617	51.889	115.112	0.0	0.0	0.0
T+4일 예정 수주량	int	34617	60.856	123.275	0.0	0.0	1.0
작년T일 예정 수주량	int	34617	63.71	127.47	0.0	0.0	1.0
작년T+1일 예정 수주량	int	34617	70.923	133.088	0.0	0.0	8.0
작년T+2일 예정 수주량	int	34617	89.927	146.436	0.0	0.0	28.0
작년T+3일 예정 수주량	int	34617	76.281	138.962	0.0	0.0	11.0
작년T+4일 예정 수주량	int	34617	55.156	120.927	0.0	0.0	0.0
T일 예상 수주량	int	34617	71.218	134.574	0.0	0.0	5.0
T+1일 예상 수주량	int	34617	73.145	135.423	0.0	0.0	8.0
T+2일 예상 수주량	int	34617	71.893	135.46	0.0	0.0	5.0
T+3일 예상 수주량	int	34617	77.255	138.454	0.0	0.0	10.0
T+4일 예상 수주량	int	34617	76.309	138.171	0.0	0.0	6.0
Temperature	float	34617	18.06	5.222	8.191	13.41	19.544
Humidity	float	34617	35.618	80.93	11.53	21.799	28.932

3) 데이터 (품질) 전처리

• 데이터 품질 전처리 목적

- 실제 공정에서 발생하는 데이터는 의미 없는 값을 가지거나 누락 및 오타의 발생으로 데이터 품질이 떨어지는 경우가 빈번하다.
- 품질이 낮은 데이터를 이용하면 좋은 분석 방법론을 이용하더라도 좋은 결과를 도출하기 어렵기 때문에, 데이터 품질 전처리는 데이터 분석에서 필수적인 단계이다.
- 따라서 데이터의 6가지 품질 지수를 파악하고, 데이터 전처리를 통해 품질 지수를 향상하는 것이 데이터 전처리의 목적이라고 할 수 있다.

• 데이터 품질 지수

- 제조 현장에서 수집되는 데이터 및 AI 분석모델을 적용하기 위한 데이터를 가공하기 전 품질 확인이 우선시 되어야 한다. 다량의 데이터를 확보하는 것도 중요하지만, 데이터 사이언스 분야에서 쓰이는 표현인 'garbage in, garbage out'이 의미하는 것처럼



럼 결함이 있는 입력 데이터를 이용하면 좋지 않은 결과를 도출할 수 있다. 그러므로 데이터의 품질이 전제되었을 때 AI 분석모델의 우수한 성능을 기대할 수 있다.

- 수주 데이터의 경우 결측치 제거 등의 데이터 전처리를 통해 데이터의 품질을 제고할 수 있다. 본 연구에서는 데이터 품질 확인을 위해 Gartner에서 제시한 아래와 같은 6종의 데이터 품질 평가지표를 고려하였다.

- ① 완전성(Completeness) : 필수 항목에 누락(결측치)이 없어야 한다.
- ② 유일성(Uniqueness) : 데이터 항목은 유일해야 하며 중복되어서는 안 된다.
- ③ 유효성(Validity) : 데이터 항목은 정해진 데이터 유효 범위 및 도메인을 충족해야 한다.
- ④ 일관성(Consistency) : 데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 동일한 데이터 간에 불일치가 발생하지 않아야 한다.
- ⑤ 정확성(Accuracy) : 실제 존재하는 객체의 표현 값이 정확히 반영되어야 하며, 정보의 수집 주기를 유지해야 한다.
- ⑥ 무결성(Integrity) : 데이터베이스 자료의 오류 없이 변화에 영향을 받지 않고 데이터의 정확성과 일관성, 유효성이 보호되어야 한다.

• 데이터 품질 지수 세부 설명

- 완전성 품질 지수 = $(1 - (\text{결측치} / \text{전체 데이터 수})) \times 100$

- ① 데이터의 결측치를 확인하기 위해 pandas 패키지의 'isna()' 와 'sum()' 메서드를 이용하여 총 결측치 개수를 구한다.
- ② 결측치의 개수를 이용하여 완전성 품질 지수를 계산한다.

- 유일성 품질 지수 = $(1 - (\text{중복 데이터 수} / \text{전체 데이터 수})) \times 100$ 본 실습의 데이터는 'CRET_TIME' 열을 기준으로 유일한 값을 가지는지 판단한다.

- 유효성 품질 지수

- ① 데이터가 유효 범위 내에 있는가?
- ② 데이터가 올바른 형식으로 구성되어 있는가?
- ③ 수집된 날짜 안에 들어가 있는가?

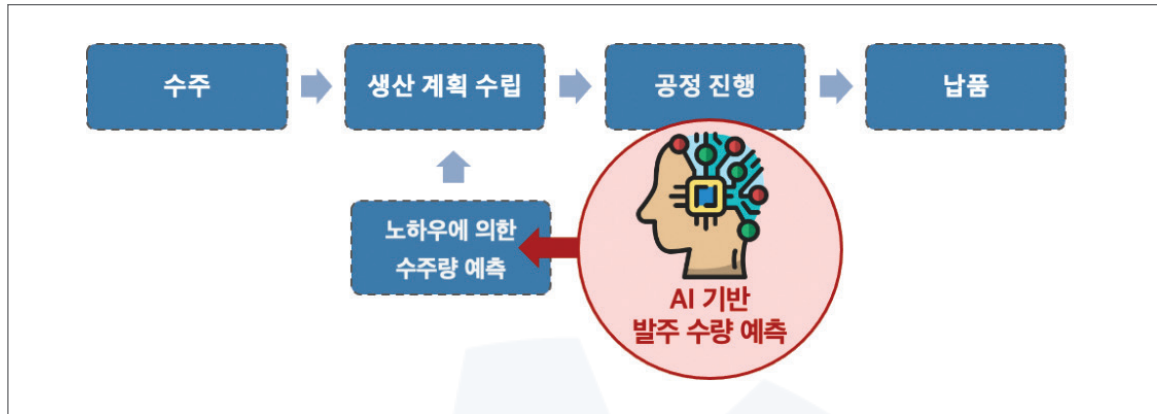
- 일관성 품질 지수 = $(\text{일관성 만족 데이터 수} / \text{전체 데이터 수}) \times 100$

- 정확성 품질 지수 = $(1 - (\text{정확성 위배 데이터 수} / \text{전체 데이터 수})) \times 100$

- 무결성 품질 지수 = $(1 - (\text{유일성, 유효성, 일관성 지수 중 100\%가 아닌 지수 개수} / 3)) \times 100$

2.2 분석모델 소개

1) 데이터 흐름 및 AI 분석모델 적용 흐름도



[그림 5] 제조 공정에서의 데이터 흐름 및 AI 모델 적용 단계 도식화

2) AI 분석모델

• 제조 수주 데이터 분석 방법

- 시계열 예측이란?

- 시계열(time-series) 데이터는 일정한 기간 동안 특정 시간 간격으로 관측(수집)된 시간적 순서를 가진 데이터를 말한다. 수집된 시계열 데이터를 활용하여 미래에 발생할 사건 혹은 값 등을 예측하는 것을 시계열 예측이라 한다. 데이터가 특정한 시간 단위(time step) 마다 하나의 수집된 값을 가지면 단변량(univariate) 시계열, 여러 값을 가지면 다변량(multivariate) 시계열로 정의한다.

- 시계열 특성을 가진 제조 수주 데이터의 분석

- 수주 데이터의 특성상 당일 수주 수량뿐만 아니라 향후 1일부터 4일까지 이내의 계획 수주 수량의 정보를 포함하고 있다. 또한 부품별로 데이터를 분석할 때, 일자별로 다양한 수주 정보가 순차적으로 존재하는 다변량 시계열 데이터 형태로 볼 수 있다. 미래, 즉 예측 시점으로부터 특정 일자 이후의 수주 수량을 예측하는 것이 본 분석의 목적이므로 과거에 계획한 수주 다변량 시계열 데이터를 활용하는 AI 분석모델을 구축한다.

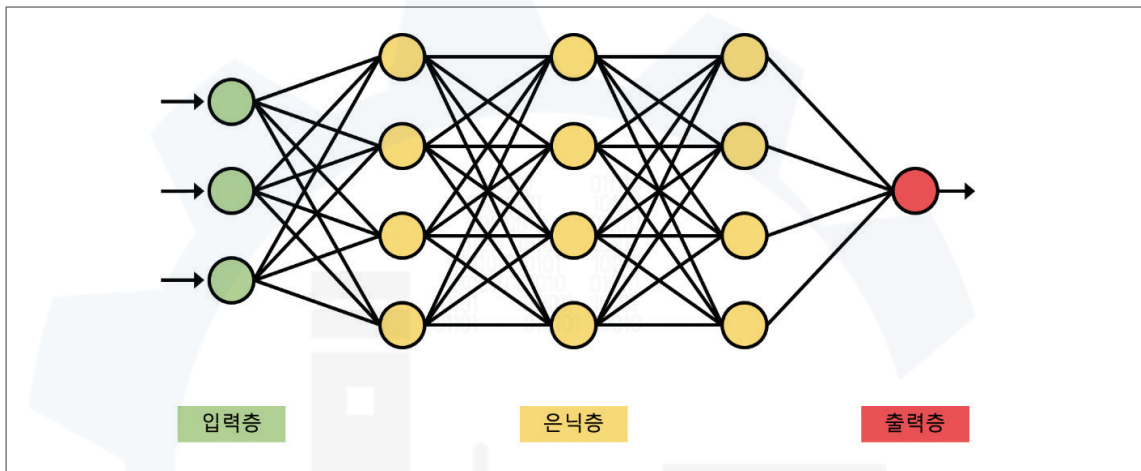
- AI 모델을 활용한 분석 목적

- 현재 노하우에 기반하여 수주량을 예상하고 이를 기반으로 생산 계획을 수립함으로써, 불측의 수주 진행에 대한 결함이 일어나는 문제가 발생할 수 있다. 이러한 문제를 방지하고자 단변량 시계열 및 다변량 시계열 분석 시, 다수의 통계 기반 방법론은 인위적인(handcrafted) 특성 추출 및 변환 단계를 요구한다. 하지만 인공지능망(Artificial Neural Network, ANN) 기반 방법론은 이러한 특성 추출을 자동적으로 수행할 수 있으며, 특히 본 분석에서 사용될 장단기 메모리의 경우 시계열 특성뿐만 아니라 장기적인 의존성 또한 학습할 수 있다. 따라서 AI 분석모델을 활용하여 수주 수량을 예측할 경우, 수주 관련 다양한 특성을 통합적으로 고려하는 분석을 실시할 수 있다.



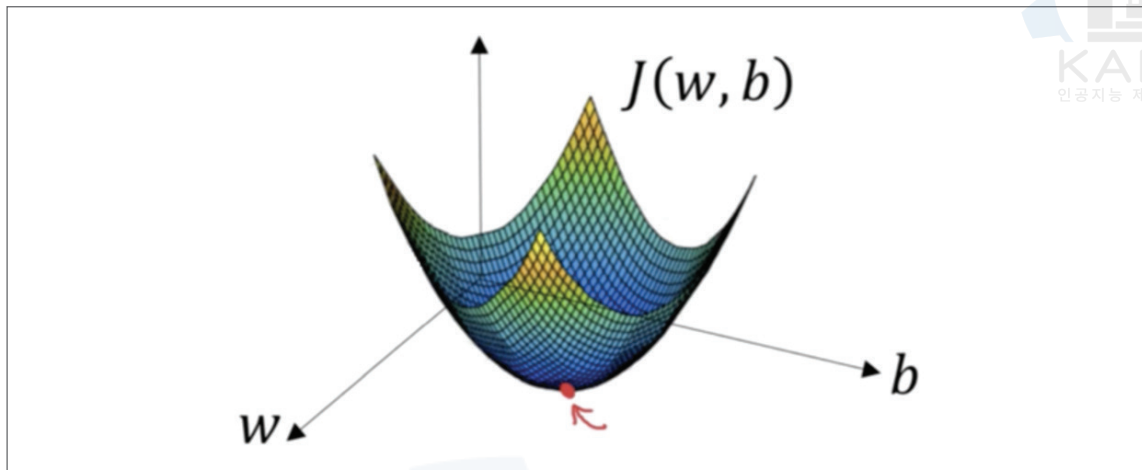
• 인공지능망 기반 알고리즘

- 인공지능망은 신경망이라 불리는 구조를 연쇄적으로 쌓은 후, 데이터를 활용하여 신경망을 구성하는 모수(parameter)를 학습시키는 모델이다. 일반적으로 신경망은 [그림 6]과 같이 입력층(input), 은닉층(hidden), 출력층(output)으로 구성되며 각 층은 여러 노드(node)를 가진다. 은닉층은 입력층으로부터 데이터를 받아 연속적인 연산(행렬곱 등)을 수행하고 출력층을 거쳐 최종 출력 값을 배출한다. 신경망의 층 사이에는 활성화 함수(activation function)를 사용하여 선형 및 비선형 변환을 적용할 수 있으며, 이러한 점 덕분에 인공지능망을 심층적으로 구성 시 데이터에 내재된 복잡한 특성 등을 모델링할 수 있다.



[그림 6] 인공지능망의 예시

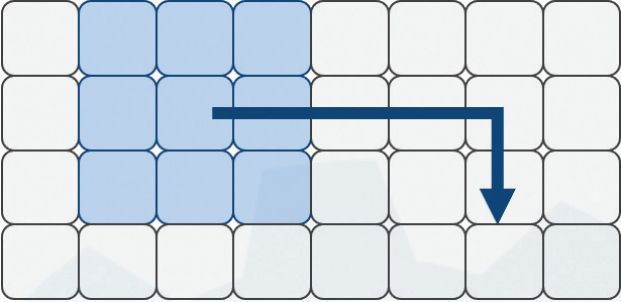
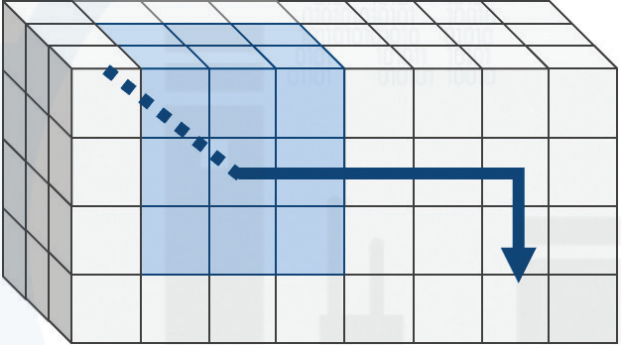
- 데이터를 사용하여 인공지능망을 학습시킨다는 것은 데이터를 모델의 입력과 출력으로 사용하여 인공지능망으로 하여금 입력과 출력 사이의 관계를 모방하는 함수를 학습하도록 하는 것이다. 학습 시에는 인공지능망 내 여러 모수들의 최적값을 찾게 되는데, 이 과정에서 일반적으로 경사 하강법(gradient descent)을 사용한다. 신경망의 예측값과 데이터의 실제값의 차이는 손실 함수(loss function)를 사용하여 계산되며 이를 학습 지표로도 볼 수 있다. 경사 하강법은 손실 함수의 편미분 값을 사용하여 모수의 값을 손실 함수 값이 감소하는 방향으로 반복적으로 변화시키는 방법이다. 편미분 값을 구하는 과정에서 오류 역전파(error backpropagation)를 사용하는데, 연쇄 법칙(chain rule)에 따라 신경망 내의 연결되어 있는 모든 모수에 대한 편미분 값을 계산하게 된다. 이러한 과정을 통해 신경망의 모수는 점차 최적의 값에 가까워지게 되고, 이를 신경망 학습이라 한다.



[그림 7] 경사 하강법(gradient descent)

〈출처: <https://builtin.com/data-science/gradient-descent>〉

- 합성곱 신경망(Convolutional Neural Network, CNN)합성곱 신경망은 시야의 일부 범위 안에 있는 시각 자극에만 반응하는 인간의 시신경 구조에 착안해 개발된 알고리즘으로, 이미지 내의 패턴을 찾는 데 유용한 것으로 알려져 있다. 일반적으로 합성곱 신경망은 이미지 분류(Image classification)나 객체 인식(Object detection)과 같은 컴퓨터 비전 분야에서 주로 사용되어 왔다.
- 하지만 최근에는 데이터에 내재된 국소적 특징을 효과적으로 추출하는 합성곱 신경망의 장점을 활용하여, 이미지뿐만 아니라 자연어처리(Natural language processing, NLP) 혹은 시계열(time-series) 특성 파악을 위한 목적으로도 응용되고 있다. [표 1]처럼 합성곱 필터의 이동 방향의 수에 따라 1차원, 2차원, 3차원 합성곱 신경망으로 분류될 수 있는데, 이는 입력 데이터의 형태에 따라 결정될 수 있다. 각각 시계열 데이터, 이미지 데이터, 볼륨 데이터(비디오 혹은 MRI 등 3차원 이미지 데이터)를 입력으로 받는다.

종류	커널 진행 방향	입력 데이터의 형태 (sample 제외)
1차원 합성곱 신경망	 kernel_size: 3	2차원 텐서 (timesteps, features) 예 : 시계열 데이터
2차원 합성곱 신경망	 kernel_size: (3,3)	3차원 텐서 (height, width, channel) 예 : 이미지 데이터
3차원 합성곱 신경망	 kernel_size: (3,3,2)	4차원 텐서 (volume or length, height, width, channel) 예 : 볼륨 데이터

[표 1] 합성곱 신경망의 종류와 특징

- 1차원 합성곱 신경망(1D Convolutional Neural Network, 1D CNN)본 분석에서는 시계열 특성이 있는 데이터 처리를 위해 1차원 합성곱 신경망을 활용하며, 이는 순환 신경망 기반의 시계열 모델보다 적은 연산량과 빠른 속도로 데이터를 처리하는 장점이 있다.

• 해당 AI 방법론(알고리즘) 선정 이유 기술

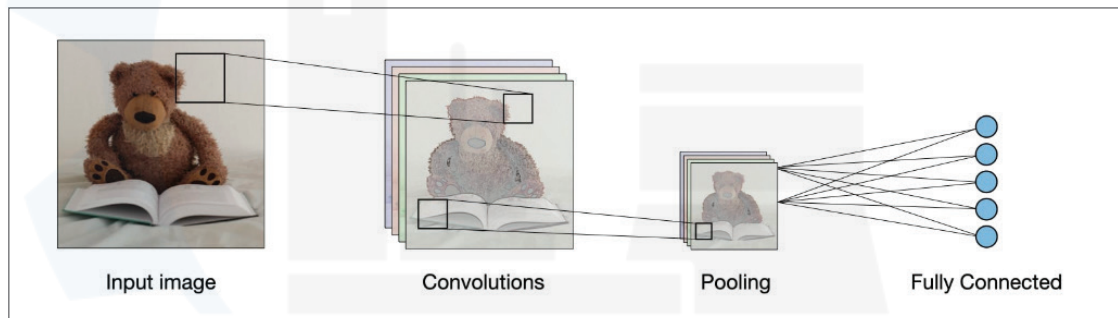
- 제조 데이터 분석 알고리즘 : 1차원 합성곱 신경망(1D CNN)본 분석에서는 1차원 합성곱 신경망을 기반으로 다변량 시계열 데이터를 활용한 수주 수량 예측 모델을 구축한다. 앞서 설명한 대로, 1차원 합성곱 신경망은 시간 정보가 담긴 데이터를 이용한 예측 혹은 분류 문제를 해결하는 데에 활용된다. 또한, 각 뉴런이 이전 층과 연결될

때 작은 국소적 일부 영역에 연결함으로써 근접한 영역의 특성에 집중하여 시간적 특성을 성공적으로 학습에 활용할 수 있다. 따라서, 생산성 향상, 납기일 준수 및 최적의 재고 관리를 위해 시계열 특성을 가진 제조 데이터를 활용한 AI 기반의 수주 수량 예측 시스템에 1차원 합성곱 신경망을 적용하는 것이 적절하다고 사료된다.

• 적용하고자 하는 AI 분석 방법론(알고리즘)의 구체적 소개

- 일반적으로 합성곱 신경망은 3종류의 층으로 모델을 구성할 수 있다. 여러개의 합성곱 층과 풀링이라 부르는 서브샘플링(subsampling) 층으로 구성되며, 마지막에는 하나 이상의 완전 연결 층이 이어진다. 이 완전 연결 층은 모든 유닛 i 가 모든 출력 유닛 j 에 가중치 w_{ij} 로 연결되어 있는 다층 퍼셉트론이다.

- ① 합성곱 층(Convolution layer) : 입력 데이터와 커널의 합성곱을 통해 출력을 얻은 뒤 활성화 함수를 거치는 단계이다. 데이터의 특징을 추출한다.
- ② 풀링(Pooling) : 출력을 샘플링하여 요약통계량으로 대체하여 데이터 입력 변화의 영향을 줄이고 연산을 줄이는 단계이다. 앞서 추출된 데이터의 특징 정보를 요약한다.
- ③ 완전 연결 층(Fully connected layer) : 앞에서 추출된 특징을 통해 원하는 결과를 예측할 수 있도록 학습하는 단계이다.



[그림 8] 합성곱 신경망의 구조

<출처: <https://stanford.edu/~shervine/l/ko/teaching/cs-230/cheatsheet-convolutional-neural-networks>>

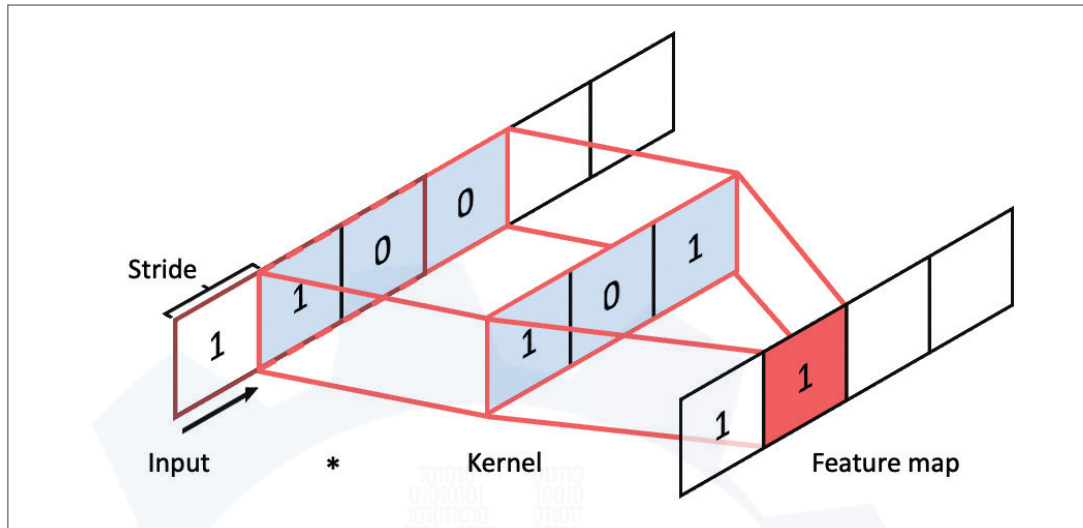
- 다음은 1차원 합성곱 신경망의 동작 단계에 대한 설명이다.

- ① 합성곱 층 : 합성곱 단계에서 입력 데이터의 각 부분에서 다수의 합성곱을 수행하여 특성 맵(feature map)이 얻어지고, 이후 비선형 활성화 함수를 거친다. 합성곱은 두 함수에 관한 연산으로, 한 함수를 반전, 전이시킨 뒤 이동시키며 다른 함수와 곱한 값들을 더하는 계산이다. 신경망에서는 함수를 반전시키지 않는 교차상관(cross-correlation) 연산을 사용하기도 하지만 기계학습 분야의 관례상 두 연산 모두 합성곱이라고 부른다.

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

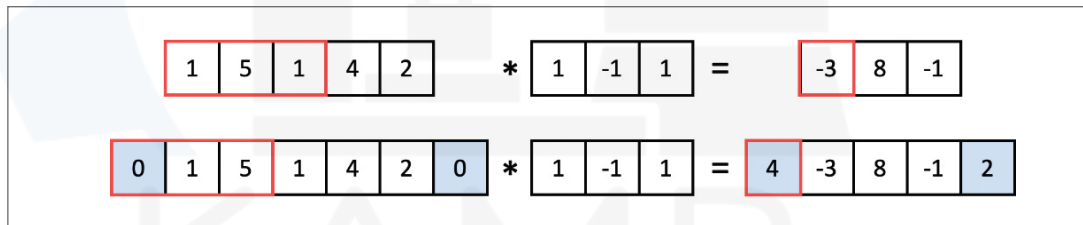
[그림 9] 합성곱 연산

일반적으로 사용되는 활성화함수는 Rectified Linear Unit(ReLU)로, 비선형성을 네트워크에 도입하는 것을 목표로 한다. 이때 입력 데이터를 이동하며 합성곱 되는 공용 파라미터를 커널(kernel) 혹은 필터(filter)라고 하고, 각 합성곱 이후 다음 합성곱을 위해 입력 데이터상에서 커널이 이동하는 크기를 스트라이드(stride, 보폭)라고 한다.



[그림 10] 1차원 합성곱 신경망에서 입력 데이터와 커널의 합성곱을 통해 특징 맵 추출

커널과 스트라이드의 크기에 따라 출력의 크기가 입력에 비해 작아질 수 있는데, 이는 신경망의 표현력을 제한한다. 따라서 이를 막기 위해 입력의 경계면에 특정 값을 채우는데, 이를 패딩(padding)이라고 하며, 보편적으로 쓰이는 0으로 채워 넣는 방식은 제로-패딩(zero-padding)으로 불린다.



[그림 11] 1차원 합성곱 신경망에서의 제로-패딩 예시

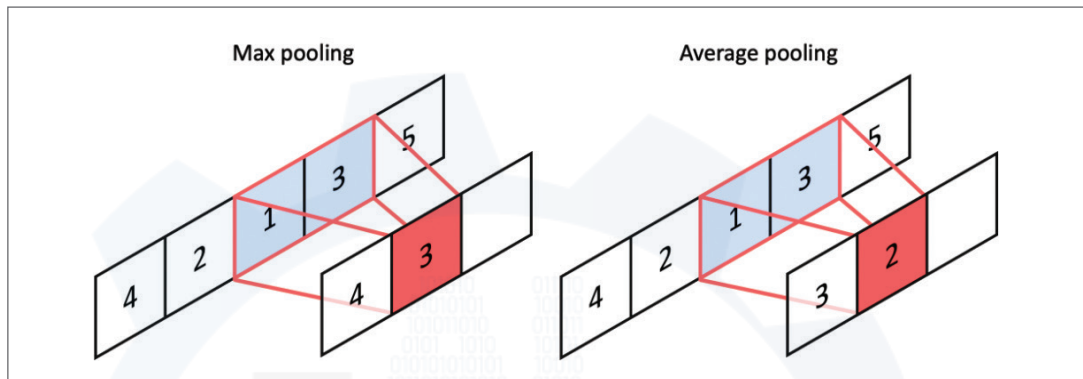
패딩의 유무, 방식에 따라 합성곱은 valid, same, full 세 가지로 나뉘어진다. valid 합성곱은 패딩을 사용하지 않아 입력에 비해 출력의 크기가 감소하고, same 합성곱은 입력과 출력의 크기가 유지될 정도로만 패딩을 사용한다. full 합성곱은 입력의 양 끝에 있는 경계면 데이터들이 중앙 데이터와 같은 횟수의 합성곱이 진행되도록 충분히 큰 양으로 패딩을 사용한다.

합성곱 계층에서 얻어진 출력의 길이(O)는 데이터 입력의 길이(I), 커널 길이(F), 데이터 앞쪽과 뒤쪽 패딩의 양(P), 스트라이드(S)로 계산될 수 있다.

$$O = \frac{I - F + P_{start} + P_{end}}{S} + 1$$

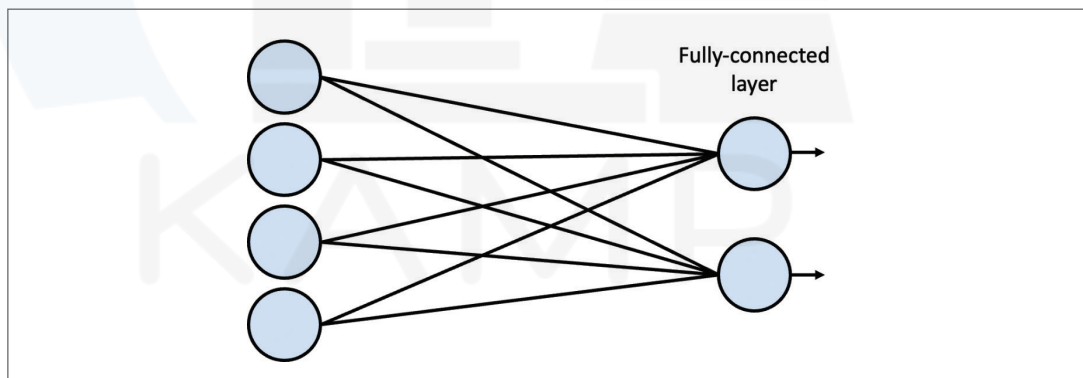
[그림 12] 합성곱으로 얻어진 출력 데이터의 길이 계

- ② 풀링 : 합성곱 연산 이후 활성화 함수를 거친 출력은 풀링 연산을 이용해 좀 더 수정된다. 풀링 연산은 특정 위치의 출력을 주변 출력들의 요약 통계량으로 대체하는 다운샘플링 기법으로, 일반적으로 최댓값 풀링(max pooling)과 평균값 풀링(average pooling)이 사용된다. 최댓값 풀링은 주변 출력들 중 가장 큰 값을 사용하고, 평균값 풀링은 주변 출력들의 평균값을 사용한다. [그림 13]에서는 풀링 사이즈(pool_size)가 2일 때의 최댓값 풀링과 평균값 풀링을 적용한 것을 도식화하였다. 이러한 방식으로 풀링은 입력 데이터 변화의 영향을 줄여주고, 특징의 구체적 위치 보다는 해당 특징의 존재 유무에 더 집중할 수 있도록 한다. 또한, 특징을 유지하는 동시에 정보량을 줄여 연산량을 줄이는 역할을 한다.



[그림 13] 최댓값 풀링과 평균값 풀링

- ③ 완전 연결 층 : 완전 연결 층(Fullyconnected layer)은 한 층의 모든 노드가 이전 층의 모든 노드와 연결된 상태이다. 합성곱 계층과 풀링을 통해 데이터의 특징을 추출했다면, 완전 연결 층에서는 추출된 특징을 통해 원하는 결과값을 예측하도록 한다. 합성곱 층의 출력 결과를 완전 연결 층에 통과시키기 위해 데이터를 평탄화하는 레이어(Flatten layer)를 사용한다.



[그림 14] 완전 연결 층(Fullyconnected layer)

• AI 분석 방법론(알고리즘) 구축 절차 설명

- 수주 수량 데이터를 활용하여 AI 분석모델을 적용하기 위해서는 적합한 형태로 데이터셋을 변환하는 것이 필요하다. 전처리 과정을 통하여 수집된 데이터에 대해 정제 및 필터링을 진행한다. 이후 데이터셋을 학습-검증-평가 데이터로 분리하여 일련의 학습 과정 내에서 목적에 부합하게 사용한다.

- 본격적인 AI 분석 방법론 구축으로는, 선택한 알고리즘인 1차원 합성곱 신경망을 정의하는 과정이 요구된다. 인공신경망 기반 모델의 경우, 조정할 수 있는 초매개변수(hyper-parameter)가 존재한다. 예를 들어, 장단기 메모리의 경우 은닉층의 노드 개수, 활성화 함수, 규제항 및 손실 함수의 종류 등을 사용자의 선택에 따라 다양하게 조정할 수 있다. 따라서 먼저 적합한 초매개변수를 설정함으로써 모델을 정의한다. 이후 준비된 학습 데이터셋을 활용하여 학습을 진행함과 동시에 검증 데이터셋으로 모델 성능을 검증한다. 필요시에는 모델 성능 제고를 위해 모델을 재정의하거나 변경할 수 있다. 최종적으로 학습이 종료된 모델을 저장하고 평가용 데이터셋을 사용하여 성능을 평가한다.

2.3 분석 체험

1) 필요 SW, 패키지 설치 방법 및 절차 가이드

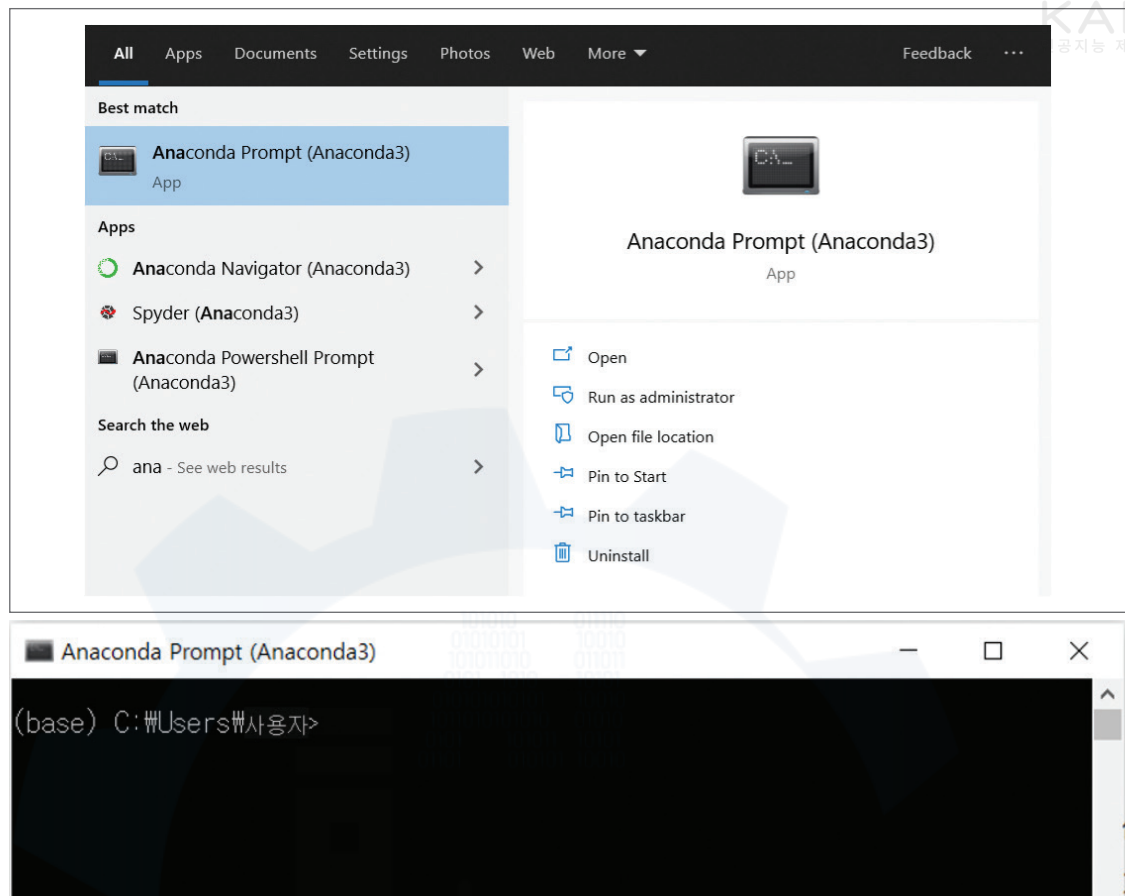
▶ 필요 SW 설치

- 프로그래밍 언어인 파이썬(Python)을 사용하며 개발 환경으로는 주피터 노트북(Jupyter Notebook)을 활용한다. 관련 SW 설치는 3. 부록 [분석 환경 구축을 위한 설치 가이드]를 참고하면 된다.

▶ 가상환경 구축 가이드

- 3. 부록 [분석 환경 구축을 위한 설치 가이드]에서 설치한 아나콘다(Anaconda)에서 파이썬의 독립적인 가상의 작업 환경을 구축한다. 파이썬의 모듈은 다양한 버전이 존재하기 때문에 충돌이 일어나기 쉬우며, 이러한 충돌을 방지하기 위해 실습마다 독립적인 가상 환경 구축을 권장한다. 가상 환경을 사용하기 위해서는 환경을 구축하고 실행(활성화)해야하고, 실습이 끝난 이후에는 환경을 비활성화 시켜야 한다. 주피터 노트북에서 가상 환경을 실행하는 구체적인 순서 및 실행 방법은 아래와 같다.

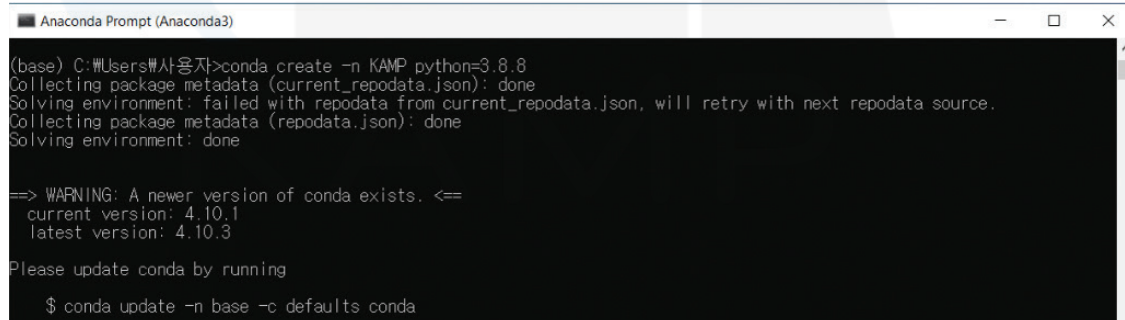
- 아나콘다 프롬프트 실행



- 가상 환경 생성

```
# conda create -n 가상환경이름 python=버전
```

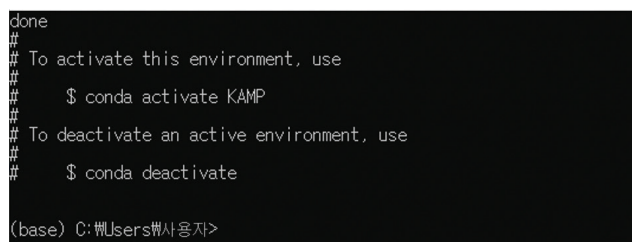
```
conda create -n KAMP python=3.7.0
```



...

```
Proceed ([y]/n)? y # 해당 문구가 뜨면 'y' 를 입력한다.
```

...





- 가상 환경 실행

가상 환경이 활성화되면 아래 그림처럼 명령 프롬프트 앞에 (가상 환경 이름)이 표시된다.

```
# conda activate 가상환경이름
```

```
conda activate KAMP
```

```
(base) C:\Users\회사용자>conda activate KAMP
```

```
(KAMP) C:\Users\회사용자>
```

- 분석이 끝난 후에는 가상 환경을 종료한다.

```
conda deactivate
```

```
(KAMP) C:\Users\회사용자>conda deactivate
```

```
(base) C:\Users\회사용자>
```

- 주피터 노트북에서 가상 환경 실행

주피터 노트북에서 가상 환경을 실행하기 위해 가상 환경 비활성화(종료) 상태에서 아래의 코드를 작성한다. 주피터 노트북 실행창에서 빨간 박스처럼 생성한 가상 환경이 있는 것을 확인할 수 있다. 해당 가상 환경을 선택 후 실습을 진행한다.

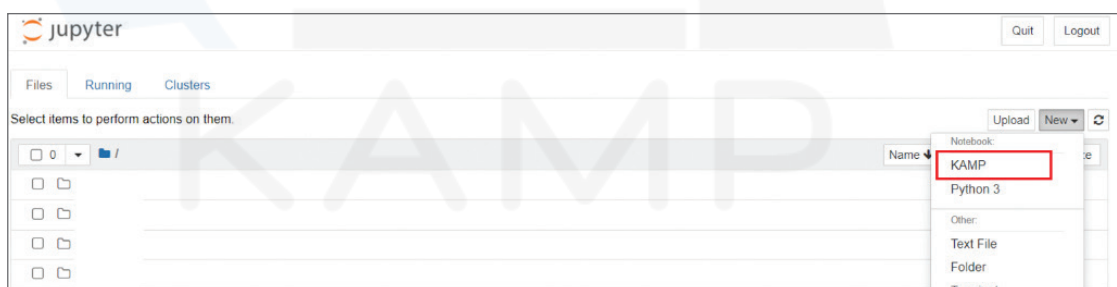
```
pip install ipykernel
```

```
# python -m ipykernel install --user --name 가상환경이름 --display-name "가상환경이름"
```

```
python -m ipykernel install --user --name KAMP --display-name "KAMP"
```

```
# jupyter notebook 실행
```

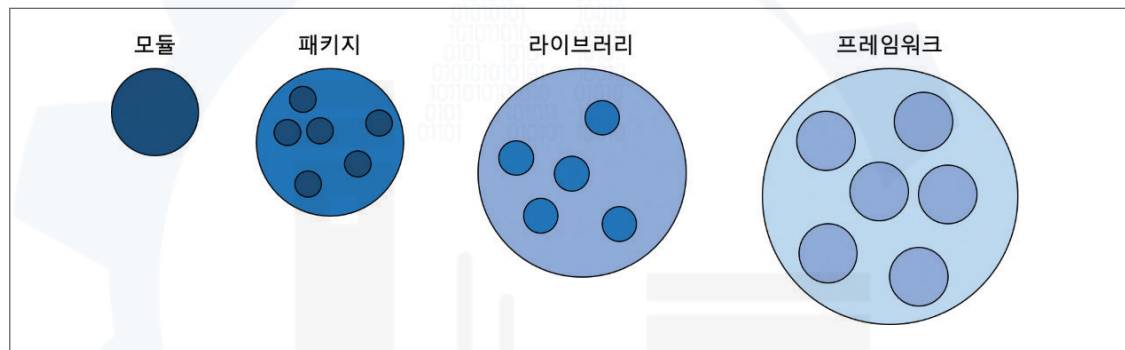
```
jupyter notebook
```



▶ 패키지 설치 가이드

- 패키지 설치 전 파이썬 관련 용어 정리

모듈 (module)	- 함수나 변수 또는 클래스를 모아 놓은 파일 - 파이썬 확장자(.py) 파일에 특정 함수, 변수, 클래스 등이 저장된 코드이다.
패키지 (package)	- 여러 모듈의 집합 - 하나의 디렉토리(경로)에 놓인 모듈들의 집합을 의미하며, 일반적으로 __init__.py라는 패키지 초기화 파일이 존재한다. - 예) NumPy : 수치해석 패키지, pandas : 데이터 분석 패키지
라이브러리 (library)	- 여러 패키지의 집합 - 파이썬을 설치할 때 기본적으로 설치되는 라이브러리를 표준라이브러리라고 한다. 파이썬 공식이 아닌 외부에서 개발한 모듈과 패키지를 묶어 외부 라이브러리라고 한다. - 예) matplotlib : 데이터 시각화를 생성하기 위한 표준 라이브러리
프레임워크 (framework)	- 라이브러리의 집합 - 목적에 맞게 코드를 작성할 수 있는 기본 아키텍처이며, 필수 구성 요소를 제공한다.



[그림 15] 파이썬 관련 개념도

▶ 필요 패키지 설치 방법

- 분석 환경인 주피터 노트북 내에서 데이터 분석 및 AI 분석모델 적용을 위한 패키지를 설치한다. `pip install <패키지 이름>` 을 입력하여 원하는 라이브러리(혹은 패키지)를 설치할 수 있다.

```
!pip install pandas
!pip install numpy matplotlib scikit-learn tensorflow
```

- 이미 설치되어 있는 패키지 외에 신규 설치가 필요한 패키지들이 자동으로 설치된다.

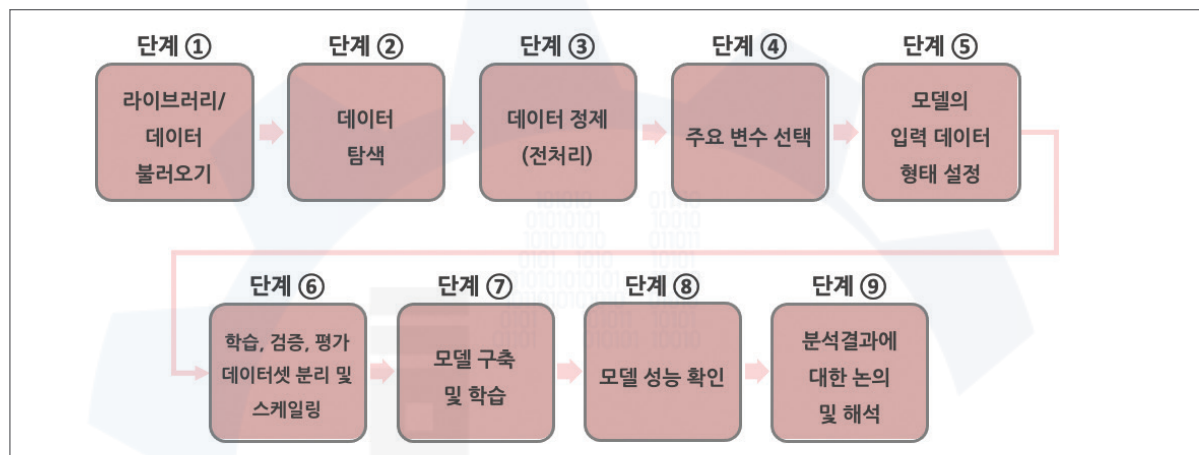
```
Requirement already satisfied: pandas in /home/ubuntu/anaconda3/lib/python3.6/site-packages (0.23.0)
Requirement already satisfied: python-dateutil>=2.5.0 in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from pandas) (2.7.3)
Requirement already satisfied: pytz>=2011k in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from pandas) (2018.4)
Requirement already satisfied: numpy>=1.9.0 in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from pandas) (1.19.5)
Requirement already satisfied: six>=1.5 in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from python-dateutil>=2.5.0->pandas) (1.15.0)
```

- 설치된 패키지들의 전체 목록을 확인할 수 있다.

```
!pip list
```

Package	Version
abs1-py	0.14.1
alabaster	0.7.10
alembic	1.4.1
anaconda-client	1.6.14
anaconda-navigator	1.8.7
anaconda-project	0.8.2

2) 분석 단계별 프로세스 - Flow Chart



[그림 16] AI 분석모델 활용 흐름도

[단계 ①] 라이브러리/데이터 불러오기

①-1. 패키지 불러오기

- import <패키지 이름>, 혹은 from <패키지 이름> import <하위 모듈 이름> 을 사용하여 전체 패키지를 불러오거나 특정 패키지 안의 모듈만 불러오기가 가능하다. 또한 import <패키지 이름> as <약자> 를 사용하여 추후 패키지 사용 시 약자(alias)로 이름을 대신할 수 있다.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import *
from numpy import newaxis

import warnings
warnings.filterwarnings(action='ignore')
pd.set_option('display.max_columns', 15)
```

[코드 1] 필요 패키지 불러오기



①-2. 데이터 불러오기

- csv (comma-separated values)는 데이터의 각 항목을 쉼표(,)로 구분한 텍스트 데이터로, 첫 줄에 있는 헤더가 각 값들이 어떤 의미를 가지는지 나타낸다.
- 본 분석에서 사용할 데이터는 '사출성형.csv'이며, 해당 형식 데이터는 pandas 패키지 내의 'read_csv()' 함수를 사용해 데이터프레임으로 불러올 수 있다. 데이터를 불러올 때, 분석 코드와 데이터가 같은 폴더에 위치하지 않는다면 해당 데이터의 절대경로 혹은 상대경로를 포함하여 불러와야 한다.
- 절대경로란 원하는 파일의 경로를 Root 디렉토리부터 모두 적는 것이고, 상대경로란 현재 위치한 폴더를 기준으로 디렉토리를 적는 것이다. 'C:/Users/name/Desktop/code/'에 분석 코드가, 'C:/Users/name/Desktop/data/'에 데이터가 위치한다고 가정할 때 '사출성형.csv'의 절대경로는 'C:/Users/name/Desktop/data/사출성형.csv'가 되고, 상대경로는 '../data/사출성형.csv'가 된다. (이 때 '../'는 상위 폴더를 의미한다.)



[그림 17] 디렉토리 구조 시각화

- 데이터프레임은 pandas 패키지에서 제공하는 데이터 구조로, 2차원 데이터를 행과 열의 형태로 저장하여 사용한다. 열은 같은 속성을 가지는 변수의 데이터, 행은 개별 관측 대상에 대한 속성 데이터의 모음을 의미한다. 'head()' 메서드를 통해 불러온 데이터의 상단에 위치한 행들을 볼 수 있다.

```
df = pd.read_csv('data/사출성형.csv') # 상대경로 사용 예시
df.head()
```

	Product_Number	T일 예정 수주량	T+1일 예정 수주량	T+2일 예정 수주량	T+3일 예정 수주량	...	T+4일 예상 수주량	DateTime	DoW	Temperature	Humidity
0	Product_ac	0	0	0	2	...	0	2022-03-13 7:04	Sunday	20.767619	50.015714
1	Product_c2	166	130	130	130	...	126	2022-02-25 11:34	Friday	13.666164	12.710959
2	Product_d7	21	11	10	18	...	0	2022-02-10 6:31	Thursday	14.106374	28.374854
3	Product_9a	28	61	50	74	...	25	2022-02-17 6:26	Thursday	10.440606	12.133030
4	Product_b9	127	0	0	11	...	0	2022-03-13 15:38	Sunday	20.767619	50.015714

[코드 2] 데이터 불러오기 및 상단 5개 데이터 확인

[단계 ②] 데이터 탐색

②-1. 탐색할 데이터 선택

- 전체 117개 제품 중에서 선택된 제품 데이터를 활용해 데이터 탐색을 진행한다. 임의로 선정된 제품인 'Product_86'의 데이터를 제품 식별자가 지정된 열인 'Product_Number' 변수를 기준으로 선택한 뒤, Product_86 데이터프레임에 저장한다. 이후 pandas 패키지의 'reset_index()' 메서드를 사용하여 데이터프레임의 인덱스를 초기화한다.

```
Product_86 = df[(df["Product_Number"]=="Product_86")]
Product_86.reset_index(drop=True, inplace=True)
Product_86.head()
```

	Product_Number	T일 예정 수주량	T+1일 예정 수주량	T+2일 예정 수주량	T+3일 예정 수주량	...	T+4일 예상 수주량	DateTime	DoW	Temperature	Humidity
0	Product_86	609	411	0	0	...	436	2022-02-12 13:33	Saturday	12.164211	30.457895
1	Product_86	632	0	0	0	...	344	2022-02-06 6:33	Sunday	10.743180	15.460502
2	Product_86	632	0	0	0	...	344	2022-02-08 6:02	Tuesday	13.430141	12.980845
3	Product_86	678	627	585	0	...	726	2022-03-06 15:01	Sunday	14.541818	13.350000
4	Product_86	555	470	0	0	...	488	2022-02-23 5:58	Wednesday	11.647015	16.433731

[코드 3] 탐색하고자 하는 제품 데이터를 선택, 저장 및 출력

②-2. 기초 통계 확인

- 제품 86 데이터의 기초 통계를 확인한다. pandas 패키지의 'describe()' 메서드를 사용하면 데이터의 행 개수, 평균, 표준편차 등의 통계치(statistics)를 확인할 수 있다.

```
Product_86.describe()
```

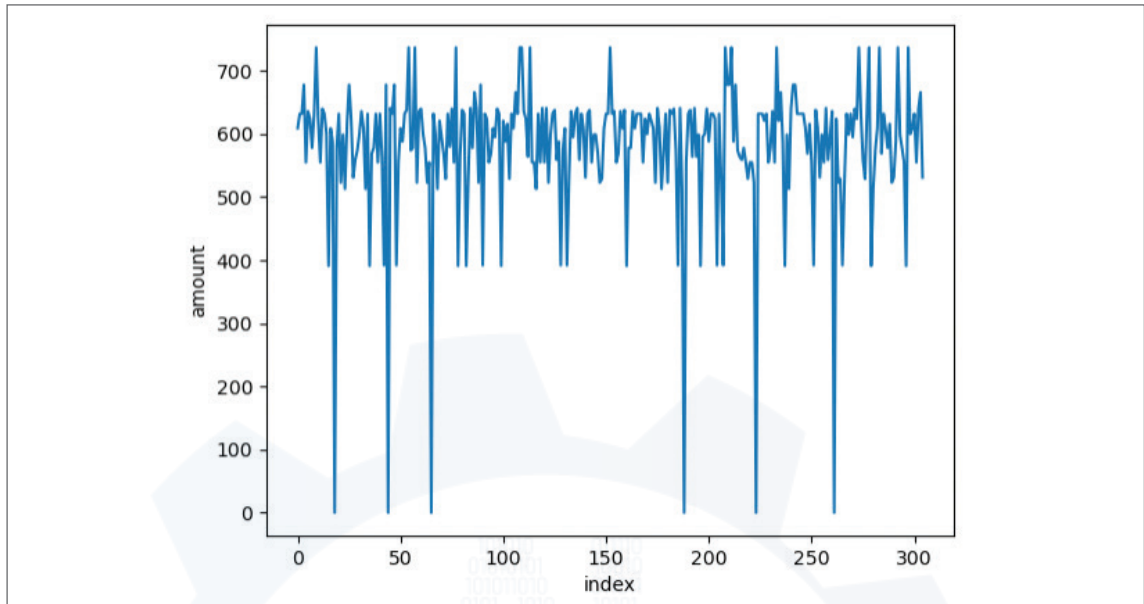
	T일 예정 수주량	T+1일 예정 수주량	T+2일 예정 수주량	T+3일 예정 수주량	T+4일 예정 수주량	...	T+2일 예상 수주량	T+3일 예상 수주량	T+4일 예상 수주량
count	305.000000	305.000000	305.000000	305.000000	305.000000	...	305.000000	305.000000	305.000000
mean	580.511475	410.721311	286.590164	277.560656	319.436066	...	355.534426	381.232787	381.481967
std	110.235218	208.825658	252.710841	249.917902	242.421847	...	208.855795	252.025091	225.271650
min	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000
25%	555.000000	411.000000	0.000000	0.000000	0.000000	...	237.000000	207.000000	288.000000
50%	600.000000	477.000000	415.000000	375.000000	423.000000	...	443.000000	480.000000	392.000000
75%	632.000000	558.000000	493.000000	514.000000	524.000000	...	523.000000	548.000000	488.000000
max	737.000000	627.000000	622.000000	603.000000	610.000000	...	725.000000	926.000000	969.000000

[코드 4] 데이터의 통계치 확인

②-3. 데이터 시각화

- matplotlib 패키지를 사용하여 제품 86 데이터의 수주량을 시각화한다. 시각화하려는 변수를 선택하고 'plot()' 함수를 활용하여 그래프를 그릴 수 있다. 예시로는 당일 수주 예정량인 'T일 예정 수주량' 변수를 선택하였다.

```
plt.plot(Product_86[['T일 예정 수주량']])
plt.xlabel('index')
plt.ylabel('amount')
```



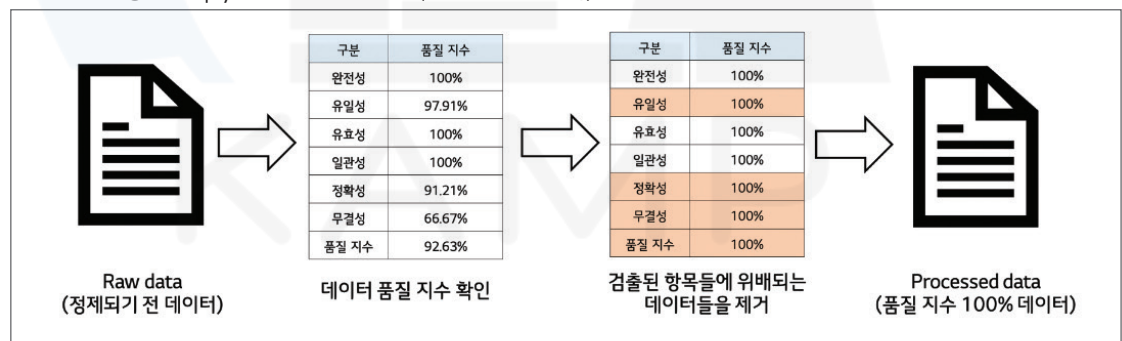
[코드 5] 데이터 시각화

[단계 ③] 데이터 정제(전처리)

③-1. 데이터 전처리

• 데이터 전처리 방법

- 개발 언어 : Python (파이썬)
- 개발 환경 : Jupyter Notebook (주피터 노트북)



[그림 18] 데이터 전처리 과정 도식화



③-2. 데이터 정제

- 데이터 내부에 'NaN(Not-a-Number)'으로 표기된 누락 데이터가 있는지 확인한다.
pandas 패키지의 'isna()' 메서드를 통해 데이터 내에 누락값의 경우 True, 누락되지 않은 값의 경우 False가 출력된다.

```
df.isna()
```

	Product_Number	T일 예정 수주량	T+1일 예정 수주량	T+2일 예정 수주량	T+3일 예정 수주량	...	T+4일 예상 수주량	DateTime	DoW	Temperature	Humidity
0	False	False	False	False	False	...	False	False	False	False	False
1	False	False	False	False	False	...	False	False	False	False	False
2	False	False	False	False	False	...	False	False	False	False	False
3	False	False	False	False	False	...	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...
34613	False	False	False	False	False	...	False	False	False	False	False
34614	False	False	False	False	False	...	False	False	False	False	False
34615	False	False	False	False	False	...	False	False	False	False	False
34616	False	False	False	False	False	...	False	False	False	False	False

[코드 6] 데이터 누락값 여부 확인

- 'isna().sum()' 메서드를 사용하여 변수별 누락 데이터의 개수를 확인할 수 있다. 논리 연산에서 논리합은 값들 중 하나라도 True 값을 가질 경우 True의 결과가 나오므로, 누락값이 존재하는 열의 'isna().sum()' 결과는 1이 반환된다.

```
df.isna().sum()
```

```
Product_Number    0
T일 예정 수주량    0
T+1일 예정 수주량    0
T+2일 예정 수주량    0
T+3일 예정 수주량    0
T+4일 예정 수주량    0
작년 T일 예정 수주량    0
작년 T+1일 예정 수주량    0
작년 T+2일 예정 수주량    0
작년 T+3일 예정 수주량    0
작년 T+4일 예정 수주량    0
T일 예상 수주량    0
T+1일 예상 수주량    0
T+2일 예상 수주량    0
T+3일 예상 수주량    0
T+4일 예상 수주량    0
DateTime          0
DoW               0
Temperature        0
Humidity           0
dtype: int64
```

[코드 7] 누락 데이터 개수 확인



③-3. 데이터 열 추가 및 제거

- pandas 패키지의 속성(attribute)인 'columns'를 통해 데이터에 존재하는 변수의 종류를 확인한다. 'DateTime' 열에는 각 데이터 행이 수집된 날짜와 시간 정보가 들어 있다. 날짜와 시간 변수를 각각 'Date' 와 'Time' 열로 따로 저장한다.

```
df.columns
```

```
Index(['Product_Number', 'T일 예정 수주량', 'T+1일 예정 수주량', 'T+2일 예정 수주량',
      'T+3일 예정 수주량', 'T+4일 예정 수주량', '작년 T일 예정 수주량',
      '작년 T+1일 예정 수주량', '작년 T+2일 예정 수주량', '작년 T+3일 예정 수주량',
      '작년 T+4일 예정 수주량', 'T일 예상 수주량', 'T+1일 예상 수주량',
      'T+2일 예상 수주량', 'T+3일 예상 수주량', 'T+4일 예상 수주량', 'DateTime', 'DoW',
      'Temperature', 'Humidity'],
      dtype='object')
```

[코드 8] 데이터의 변수 확인

```
df["Date"] = pd.to_datetime(df["DateTime"]).dt.date
df["Time"] = pd.to_datetime(df["DateTime"]).dt.time
```

[코드 9] 필요한 데이터 열 추가

- 'Date'와 'Time' 변수로 분리되어 추가되었기 때문에, 'drop()' 메서드를 이용하여 기존의 'DateTime' 변수를 제거한다.

```
select_col_df = df.drop(['DateTime'], axis=1)
select_col_df.columns
```

```
Index(['Product_Number', 'T일 예정 수주량', 'T+1일 예정 수주량', 'T+2일 예정 수주량',
      'T+3일 예정 수주량', 'T+4일 예정 수주량', '작년 T일 예정 수주량',
      '작년 T+1일 예정 수주량', '작년 T+2일 예정 수주량', '작년 T+3일 예정 수주량',
      '작년 T+4일 예정 수주량', 'T일 예상 수주량', 'T+1일 예상 수주량',
      'T+2일 예상 수주량', 'T+3일 예상 수주량', 'T+4일 예상 수주량', 'DoW',
      'Temperature', 'Humidity', 'Date', 'Time'],
      dtype='object')
```

[코드 10] 필요 없는 변수 제거 및 변수 확인

- 'DateTime' 변수를 제거하는 방법 대신에, 'loc[]' 메서드를 사용하여 해당 변수를 제외하고 필요한 변수만 지정하는 방법을 사용할 수 있다. 아래의 예시 코드는 위 코드의 결과와 동일하다.

```
select_col_df = df.loc[:, ['Product_Number', 'T일 예정 수주량', 'T+1일 예정 수주량', 'T+2일 예정 수주량', 'T+3일 예정 수주량', 'T+4일 예정 수주량', '작년 T일 예정 수주량', '작년 T+1일 예정 수주량', '작년 T+2일 예정 수주량', '작년 T+3일 예정 수주량', '작년 T+4일 예정 수주량', 'T일 예상 수주량', 'T+1일 예상 수주량', 'T+2일 예상 수주량', 'T+3일 예상 수주량', 'T+4일 예상 수주량', 'DoW', 'Temperature', 'Humidity', 'Date', 'Time']]
select_col_df.columns
```

```
Index(['Product_Number', 'T일 예정 수주량', 'T+1일 예정 수주량', 'T+2일 예정 수주량', 'T+3일 예정 수주량', 'T+4일 예정 수주량', '작년 T일 예정 수주량', '작년 T+1일 예정 수주량', '작년 T+2일 예정 수주량', '작년 T+3일 예정 수주량', '작년 T+4일 예정 수주량', 'T일 예상 수주량', 'T+1일 예상 수주량', 'T+2일 예상 수주량', 'T+3일 예상 수주량', 'T+4일 예상 수주량', 'DoW', 'Temperature', 'Humidity', 'Date', 'Time'],
      dtype='object')
```

[코드 11] 필요한 변수만 추출하여 데이터 저장 및 변수 확인

③-4. 불필요한 행 제거 (하루에 여러 번 수집된 데이터)

- 동일 제품에서 하루에 여러 번 데이터가 수집된 경우가 있다. (예: 2022년 1월 27일에 'Product_86'에 대한 데이터 로그가 4차례 존재한다.) 이 경우, 가장 마지막 시간의 로그가 해당일의 전체 정보를 담고 있기 때문에, 같은 날 저장된 같은 제품의 로그 중 가장 늦은 시간에 기록된 로그만을 남긴다. 'Product_Number', 'Date', 'Time' 순으로 데이터프레임을 오름차순으로 정렬 뒤, 'drop_duplicates()' 메서드를 사용하여 동일한 'Product_Number'와 'Date'를 가진 행 중 하나의 행만을 남긴다. 이 때 가장 마지막 행만 남기기 위해 메서드 내 변수인 keep을 last로 지정하여 사용한다. 불필요한 열을 제거한 이후, 'reset_index()' 메서드를 통해 인덱스를 초기화한다.

```
select_col_df.sort_values(by=['Product_Number', 'Date', 'Time'], inplace=True)
drop_df = select_col_df.drop_duplicates(['Product_Number', 'Date'], keep='last')
drop_df.reset_index(drop=True, inplace=True)
drop_df.head()
```

	Product_Number	T 일 예 정 수 주 량	T+1 일 예 정 수 주 량	T+2 일 예 정 수 주 량	T+3 일 예 정 수 주 량	T+4 일 예 정 수 주 량	작 년 T 일 예 정 수 주 량	...	T+3 일 예 상 수 주 량	T+4 일 예 상 수 주 량	DoW	Temperature	Humidity	Date	Time
0	Product_84	0	0	0	0	0	0	...	0	0	Wednesday	12.836897	35.813966	2022-01-26	18:30:00
1	Product_84	0	0	0	0	0	0	...	0	0	Thursday	10.601230	30.046817	2022-01-27	17:30:00
2	Product_84	0	0	0	0	0	1	...	0	0	Friday	12.714545	21.401818	2022-01-28	16:33:00
3	Product_84	0	0	0	0	1	6	...	0	0	Saturday	13.313757	12.471768	2022-01-29	17:06:00
4	Product_84	0	0	0	0	0	0	...	0	0	Sunday	9.106250	17.952750	2022-01-30	06:32:00

5 rows × 21 columns

[코드 12] 불필요한 행 제거



③-5. 불필요한 행 제거 (95일 연속 수집되지 않은 데이터)

- '사출성형.csv' 데이터는 117개의 제품에 대하여 예정 수주량, 작년 수주량, 예상 수주량이 95일 동안 연속적으로 수집되었다. 하지만 일부 데이터의 경우 데이터가 수집되지 않은 일자가 존재했는데, 이를 제거하여 동일기간동안 연속적으로 모든 데이터가 수집된 제품의 데이터만 남겨둔다.
- pandas 패키지의 'unique()' 메서드를 통해 'Product_Number' 변수의 값 리스트, 즉 모든 제품 식별자를 저장한다. 이후 각 제품 데이터의 개수가 95개가 되지 않는다면 해당 제품을 drop_list에 저장한다. 전체 데이터프레임에서 drop_list에 해당하는 제품들의 데이터 행을 제거하고, 'reset_index()'을 사용해 인덱스를 초기화한다.

```
product_list = drop_df["Product_Number"].unique()
drop_list = []
for product in product_list:
    if drop_df["Product_Number"].value_counts()[product] != 95:
        drop_list.append(product)
drop_list
```

```
['Product_84',
'Product_85',
'Product_9b',
'Product_b1',
'Product_e1',
'Product_e4',
'Product_e9',
'Product_f1',
'Product_f2',
'Product_f3',
'Product_f4',
'Product_f5',
'Product_f6',
'Product_f7',
'Product_f8']
```

[코드 13] 95일간 연속적으로 모이지 않은 Product 찾아 리스트에 저장



```
drop_idx = drop_df[drop_df["Product_Number"].isin(drop_list)].index

drop_df.drop(drop_idx, inplace=True)
drop_df.reset_index(drop=True, inplace=True)

drop_df["Product_Number"].value_counts()
```

```
Product_86  95
Product_c8  95
Product_d2  95
Product_d1  95
Product_d0  95
..
Product_a6  95
Product_a5  95
Product_a4  95
Product_a3  95
Product_f0  95
Name: Product_Number, Length: 102, dtype: int64
```

[코드 14] 95일간 연속적으로 보이지 않은 데이터 제거

- 처음 저장해둔 제품 리스트의 크기와, 데이터 제거 후 제품 리스트의 크기를 비교한다. 95일 연속으로 수집되지 않은 제품 리스트인 drop_list의 15개 제품이 제거되어 총 117개 중 102개의 제품이 남은 것을 볼 수 있다.

```
prev_product_num = len(product_list)
curr_product_num = len(drop_df["Product_Number"].unique())
print(prev_product_num, curr_product_num)
```

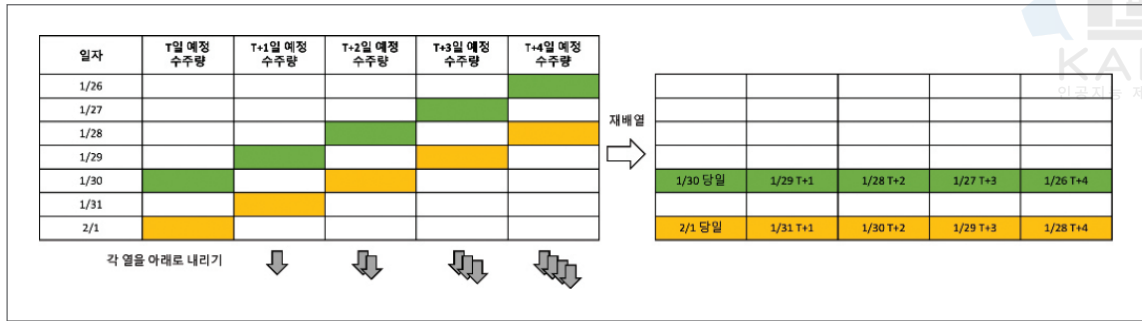
```
117 102
```

[코드 15] 데이터 제거 전후 제품 수 비교

[단계 ④] 주요 변수 선택

④-1. 변수별 상관관계 파악을 위한 데이터 재배열

- 전체 변수 중 유의미한 특성을 가진 주요 변수를 선정하기 위해서는 변수간 상관관계를 파악해야 한다. 당일 수주량에 해당되는 'T일 예정 수주량' 열(변수)을 기준으로 다른 열과의 상관관계를 계산하면 당일 수주량과 연관성이 있는 열을 파악할 수 있다. 이러한 변수간 상관관계 파악을 위해서는 주어진 데이터를 재배열할 필요가 있다. 예를 들어, 당일 수주 수량과 1, 2일 전의 계획 수주 수량 간의 상관관계를 확인하는 것이 목적인 경우 동일한 일자에 대한 계획으로 데이터를 맞춰주는 재배열을 시행해야 한다. [그림 19]처럼 1월 30일에 대한 수주 수량은 1) 1월 30일의 당일 계획 수주량(T일 예정 수주량), 2) 1월 29일의 하루 뒤 계획된 수주량(T+1일 예정 수주량), 3) 1월 28일의 이틀 뒤 계획된 수주량(T+2일 예정 수주량) 등과 연관된다고 볼 수 있다.



[그림 19] 변수별 상관관계 파악을 위한 재배열

- 본 실습에서는 당일 예정 수주량(T일 예정 수주량)과 4일간 예상했던 수주량(T+1~T+4일 예정 수주량), 동일 일자의 작년 수주량(작년 T~T+4일 예정 수주량), 그리고 기업에서 5일간 예상한 수주량(T~T+4일 예상 수주량)에 대한 상관관계를 확인하고자 하였다. 각 열을 원하는 만큼 내리기 위해 pandas의 'shift()' 메서드를 사용했다.
- 'shift()'는 int 자료형의 인자를 하나 필요로 하는 메서드로, 입력해준 수치 만큼 데이터 열을 세로로 밀어준다. 인자가 양수일 때는 열을 내리고, 음수일 때는 열을 올린다. axis 옵션을 통해 열을 가로로도 밀 수 있지만, 전처리 과정에서는 기본값인 세로 방향으로 메서드를 사용한다.
- 전체 데이터 열 중 'shift()' 메서드가 적용되어야 할 부분은 'Product_Number', 'DoW', 'Temperature' 등을 제외한 날짜 관련 열들이다. 해당 열들은 총 15개 존재하는데, 5일(T~T+4일)동안의 예정 수주량, 작년 수주량, 예상 수주량으로 구성된다. 현재 데이터는 날짜를 기준으로 오름차순 정렬되어있기 때문에, 각 열의 날짜를 T일에 맞춰주기 위해 T+1일 관련 열을 한 칸 내리고, T+2일 열을 두 칸 내려야 한다. 즉 T+N일 열을 N칸 내려주면 데이터 열들의 날짜를 맞춰줄 수 있다. 위의 정보들을 이용하여, for문의 index i를 통해 원하는 데이터 열을 고르고, 각 데이터 열이 얼마만큼 아래로 밀려야 하는지 계산한다.
- 우선 'columns' 메서드로 추출한 변수의 리스트 columns_TS에서 'shift()' 메서드를 적용할 변수를 찾는다. for문이 날짜 관련 변수 개수인 15만큼 반복되고, for 문의 index를 i라고 했을 때, i의 범위는 0부터 14까지이다. 날짜 관련 변수는 'columns' 메서드로 추출된 columns_TS 리스트 상에서 1부터 15까지 위치하기 때문에, 리스트의 인덱스는 i+1로 구할 수 있다.
- 이후 'shift()' 메서드의 인자, 즉 각 열마다 내려야하는 칸의 수를 계산한다. 날짜 관련 데이터의 일자가 T, T+1, T+2, T+3, T+4의 순서를 반복하는 형태이기 때문에, 다음 표와 같은 관계가 성립한다. 메서드의 인자는 i를 5로 나눈 나머지로 계산할 수 있다. 파이썬에서 나머지 값은 연산기호 '%'를 사용해 구할 수 있다.

for문 index(i)	0	1	2	3	4	5	6	7	...	13	14
shift 인자(N)	0	1	2	3	4	0	1	2	...	3	4

[표 2] for문 내부 인덱스와 필요한 shift 횟수에 대한 관계표

- 각 열을 아래로 내려 데이터 날짜를 맞춘 뒤, 모든 제품의 첫 4개 행은 'shift()' 연산을 통해 이전 제품 데이터가 다음 제품 칸으로 넘어간 데이터이다. 따라서 같은 제품명을 가진 행들 중 제일 위의 행을 네번 지워준다. 이후 인덱스 초기화 후 맞춰진 날짜에 맞게 데이터프레임 변수의 이름을 수정한 뒤, 'to_csv()' 메서드로 전처리 완료된 파일을 저장한다.

for문 index	columns_TS내 index	해당 열 관련 날짜	내려야 하는 칸 수
i(0~14)	i+1(1~15)	T+(i%5)일 관련	i%5

[표 3] for문 인덱스로 변수 이름을 찾고, shift 인자를 계산하는 방법에 대한 정리표

```
time_series_df = drop_df.copy()
time_series_columns = time_series_df.columns
for i in range(15):
    column_name = time_series_columns[i+1]
    time_series_df[column_name] = time_series_df[column_name].shift(i%5)
for i in range(4):
    time_series_df = time_series_df[time_series_df["Product_Number"].duplicated()]
```

[코드 16] 같은 날짜 데이터를 같은 행으로 재배열



```
time_series_df.reset_index(drop=True, inplace=True)
```

```
time_series_new_columns = ['Product_Number', 'T일 예정 수주량', 'T-1일 예정 수주량', 'T-2일 예정 수주량', 'T-3일  
예정 수주량', 'T-4일 예정 수주량',  
'작년 T일 예정 수주량', '작년 T-1일 예정 수주량', '작년 T-2일 예정 수주량', '작년 T-3일 예정 수주량', '작년 T-4일 예정  
수주량',  
'T일 예상 수주량', 'T-1일 예상 수주량', 'T-2일 예상 수주량', 'T-3일 예상 수주량', 'T-4일 예상 수주량', 'DoW',  
'Temperature', 'Humidity', 'Date', 'Time']
```

```
time_series_df.columns = time_series_new_columns
```

```
time_series_df.head()
```

Product_Number	T일 예정 수주량	T-1일 예정 수주량	T-2일 예정 수주량	T-3일 예정 수주량	T-4일 예정 수주량	작년 T일 예정 수주량		
0	Product_86	632	558.0	526.0	555.0	532.0	0	
1	Product_86	632	0.0	0.0	0.0	0.0	0	
2	Product_86	632	0.0	0.0	0.0	0.0	0	
3	Product_86	632	0.0	0.0	0.0	449.0	0	
4	Product_86	632	0.0	0.0	0.0	0.0	0	
작년 T-1일 예정 수주량 작년 T-2일 예정 수주량 작년 T-3일 예정 수주량 작년 T-4일 예정 수주량 T일 예상 수주량 T-1일 예상 수주량 T-2일 예상 수주량								
511.0		548.0		560.0		533.0	586	176.0
0.0		504.0		492.0		459.0	586	528.0
0.0		0.0		341.0		0.0	586	528.0
0.0		0.0		0.0		0.0	586	528.0
0.0		0.0		0.0		0.0	586	528.0
예상 수주량 T-2일 예상 수주량 T-3일 예상 수주량 T-4일 예상 수주량 DoW Temperature Humidity Date Time								
176.0		307.0		904.0		877.0	Sunday	9.106250 17.952750 2022-01-30 06:32:00
528.0		272.0		926.0		863.0	Monday	9.184500 14.939750 2022-01-31 07:13:00
528.0		551.0		641.0		969.0	Tuesday	8.190541 19.415135 2022-02-01 06:30:00
528.0		551.0		480.0		571.0	Wednesday	9.003851 24.231491 2022-02-02 06:29:00
528.0		551.0		480.0		344.0	Thursday	9.169762 18.761905 2022-02-03 06:31:00

[코드 17] 재배열된 데이터에 맞게 변수 이름 수정

④-2. 변수별 상관관계 파악 및 현장의 도메인 지식(domain knowledge)을 기반으로 한 유의미한 변수 선택

- 예측 대상인 T일 예정 수주량과 나머지 수치형 변수간의 표준 상관계수를 'corr()' 메서드를 이용해 자동으로 계산한다. 'head(1)'을 통해 첫번째 행만 출력할 수 있다.
- T-1~T-4일 예정 수주량, T~T-4일의 작년 수주량, 예상 수주량은 T일 예정 수주량과 대체로 높은 상관관계를 보였다. 따라서 'T일 예정 수주량' 예측에 나머지 14개의 수주량 관련 데이터를 모두 사용한다.
- 'Temperature'와 'Humidity'는 상관관계가 낮게 나왔고, 당일 수주량 예측에 크게 중요하지 않다는 전문가의 판단에 따라 예측 시 활용될 변수에서 제외한다. 수치형 데이터가 아닌 'DoW', 'Date', 'Time' 변수도 마찬가지로 제외되었다. 이처럼 산업 분야



에서 인공지능을 활용한 예측을 진행할 때, 수치적인 상관관계 파악뿐만 아니라 전문가의 현장 도메인 지식에 따라 변수를 선정하는 것은 매우 중요하다.

```
time_series_df.corr().head(1)
```

	T일 예정 수주량	T-1일 예정 수주량	T-2일 예정 수주량	T-3일 예정 수주량	T-4일 예정 수주량	작년 T
T일 예정 수주량	1.0	0.706154	0.629467	0.577544	0.623149	
작년 T일 예정 수주량		작년 T-1일 예정 수주량	작년 T-2일 예정 수주량	작년 T-3일 예정 수주량	작년 T-4일 예정 수주량	
	0.553627	0.592432	0.788053	0.665411	0.475061	
T일 예상 수주량	T-1일 예상 수주량	T-2일 예상 수주량	T-3일 예상 수주량	T-4일 예상 수주량	Temperature	Humidity
	0.723922	0.704081	0.687514	0.712446	0.671327	-0.009475 0.003519

[코드 18] 데이터 변수 간 상관관계 확인

④-3. 주요 변수 선택 및 분석을 위한 데이터 선정

- 'T일 예정 수주량'과 유의미한 관계가 있는 변수로 'T-1~T-4일 예정 수주량', '작년 T~T-4일 예정 수주량', 'T~T-4일 예상 수주량'을 최종 선택하였고, 이를 기반으로 당일 수주량 예측 모델을 설계한다. 필요에 의해서, 더 미리 예측을 진행해야 하는 경우는 T일, T-1일 데이터를 제외시킬 수 있다.

```
time_series_df = time_series_df.loc[:, ['Product_Number', 'T일 예정 수주량', 'T-1일 예정 수주량', 'T-2일 예정 수주량', 'T-3일 예정 수주량', 'T-4일 예정 수주량', '작년 T일 예정 수주량', '작년 T-1일 예정 수주량', '작년 T-2일 예정 수주량', '작년 T-3일 예정 수주량', '작년 T-4일 예정 수주량', 'T일 예상 수주량', 'T-1일 예상 수주량', 'T-2일 예상 수주량', 'T-3일 예상 수주량', 'T-4일 예상 수주량']].reset_index(drop=True)
```

```
time_series_df.head()
```

	T일 예정 수주량	T-1일 예정 수주량	T-2일 예정 수주량	T-3일 예정 수주량	T-4일 예정 수주량	작년 T일 예정 수주량	작년 T-1일 예정 수주량	작년 T-2일 예정 수주량
0	632	558.0	526.0	555.0	532.0	0	511.0	548.0
1	632	0.0	0.0	0.0	0.0	0	0.0	504.0
2	632	0.0	0.0	0.0	0.0	0	0.0	0.0
3	632	0.0	0.0	0.0	449.0	0	0.0	0.0
4	632	0.0	0.0	0.0	0.0	0	0.0	0.0
	작년 T-2일 예정 수주량	작년 T-3일 예정 수주량	작년 T-4일 예정 수주량	T일 예상 수주량	T-1일 예상 수주량	T-2일 예상 수주량	T-3일 예상 수주량	T-4일 예상 수주량
	548.0	560.0	533.0	586	176.0	307.0	904.0	877.0
	504.0	492.0	459.0	586	528.0	272.0	926.0	863.0
	0.0	341.0	0.0	586	528.0	551.0	641.0	969.0
	0.0	0.0	0.0	586	528.0	551.0	480.0	571.0
	0.0	0.0	0.0	586	528.0	551.0	480.0	344.0

[코드 19] 주요 변수 선택 및 저장

[단계 ⑤] 모델의 입력 데이터 형태 설정

⑤-1. 데이터의 독립변수 및 종속변수 정의

- [단계 ④-1]의 데이터 재배열을 통해, 각 행에는 해당 날짜에 관련이 있는 변수들(예정 및 예측 등)로 구성이 되어있으므로 일자 및 제품의 종류는 각 행 별로 독립적이라고 가정할 수 있다. 예를 들어, 'DateTime'이 2022-10-05인 데이터의 경우, 'T-1일 예

정 수주량', 'T-2일 예정 수주량', 'T-3일 예정 수주량', 및 'T-4일 예정 수주량'은 각각 2022년 10월 4일, 3일, 2일, 1일에 예정된 10월 5일의 수주량에 해당된다. 가정에 따라, 각 행별로 독립이므로 제품의 종류인 'Product Number' 변수는 제거한다

```
time_series_df.drop(['Product Number'],axis=1, inplace=True)
time_series_df
```

	T일 예정 수주량	T-1일 예정 수주량	T-2일 예정 수주량	T-3일 예정 수주량	T-4일 예정 수주량	작년 T일 예정 수주량	작년 T-1일 예정 수주량	작년 T-2일 예정 수주량	작년 T-3일 예정 수주량	작년 T-4일 예정 수주량	T일 예상 수주량	T-1일 예상 수주량	T-2일 예상 수주량	T-3일 예상 수주량	T-4일 예상 수주량
0	632	558.0	526.0	555.0	532.0	0	511.0	548.0	560.0	533.0	586	176.0	307.0	904.0	877.0
1	632	0.0	0.0	0.0	0.0	0	0.0	504.0	492.0	459.0	586	528.0	272.0	926.0	863.0
2	632	0.0	0.0	0.0	0.0	0	0.0	0.0	341.0	0.0	586	528.0	551.0	641.0	969.0
3	632	0.0	0.0	0.0	449.0	0	0.0	0.0	0.0	0.0	586	528.0	551.0	480.0	571.0
4	632	0.0	0.0	0.0	0.0	0	0.0	0.0	0.0	0.0	586	528.0	551.0	480.0	344.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9277	7	3.0	0.0	72.0	1.0	0	0.0	0.0	0.0	0.0	0	0.0	0.0	0.0	0.0
9278	7	3.0	1.0	72.0	1.0	0	0.0	0.0	0.0	0.0	0	0.0	0.0	0.0	0.0
9279	7	3.0	1.0	0.0	1.0	0	0.0	0.0	0.0	0.0	0	0.0	0.0	0.0	0.0
9280	6	3.0	1.0	0.0	0.0	0	0.0	0.0	0.0	0.0	0	0.0	0.0	0.0	0.0
9281	6	0.0	0.0	0.0	0.0	0	0.0	0.0	0.0	0.0	0	0.0	0.0	0.0	0.0

9282 rows x 15 columns

[코드 20] 변수 'Product Number' 제거 후 time_sereis_df 출력 결과

- 나머지 변수도 모두 10월 5일을 예측 혹은 예정하는 관련 정보이다. 그러므로, 'T일 예정 수주량'을 종속변수로, 그 외의 나머지 변수들을 모두 독립변수로 활용한다. 앞서 전처리 완료한 데이터 time_series_df를 'copy()' 메서드를 활용하여 독립변수를 의미하는 새로운 데이터프레임 X에 복사한다. 그리고 'pop()' 메서드를 통해 데이터프레임 X에 있는 'T일 예정 수주량' 변수만 출력하고, 종속변수를 의미하는 데이터프레임 y로 지정한다. 'pop()' 메서드는 입력된 아이템(변수명)에 해당하는 데이터를 반환하고 해당 데이터를 기존 데이터프레임에서 삭제한다. 그러므로, 데이터프레임 X에서 종속변수인 'T일 예정 수주량'이 제거되어, 남은 변수들은 모두 독립변수를 의미하게 된다.



```
X = time_series_df.copy() # 독립변수
y = X.pop('T일 예정 수주량') # 종속변수
X
```

	T-1일 예정 수주량	T-2일 예정 수주량	T-3일 예정 수주량	T-4일 예정 수주량	작년 T일 예정 수주량	작년 T-1일 예정 수주량	작년 T-2일 예정 수주량	작년 T-3일 예정 수주량	작년 T-4일 예정 수주량	T일 예상 수주량	T-1일 예상 수주량	T-2일 예상 수주량	T-3일 예상 수주량	T-4일 예상 수주량
0	558.0	526.0	555.0	532.0	0	511.0	548.0	560.0	533.0	586	176.0	307.0	904.0	877.0
1	0.0	0.0	0.0	0.0	0	0.0	504.0	492.0	459.0	586	528.0	272.0	926.0	863.0
2	0.0	0.0	0.0	0.0	0	0.0	0.0	341.0	0.0	586	528.0	551.0	641.0	969.0
3	0.0	0.0	0.0	449.0	0	0.0	0.0	0.0	0.0	586	528.0	551.0	480.0	571.0
4	0.0	0.0	0.0	0.0	0	0.0	0.0	0.0	0.0	586	528.0	551.0	480.0	344.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9277	3.0	0.0	72.0	1.0	0	0.0	0.0	0.0	0.0	0	0.0	0.0	0.0	0.0
9278	3.0	1.0	72.0	1.0	0	0.0	0.0	0.0	0.0	0	0.0	0.0	0.0	0.0
9279	3.0	1.0	0.0	1.0	0	0.0	0.0	0.0	0.0	0	0.0	0.0	0.0	0.0
9280	3.0	1.0	0.0	0.0	0	0.0	0.0	0.0	0.0	0	0.0	0.0	0.0	0.0
9281	0.0	0.0	0.0	0.0	0	0.0	0.0	0.0	0.0	0	0.0	0.0	0.0	0.0

9282 rows x 14 columns

[코드 21] 독립 변수 출력

```
print(y)
```

```
0    632
1    632
2    632
3    632
4    632
...
9277    7
9278    7
9279    7
9280    6
9281    6
Name: T일 예정 수주량, Length: 9282, dtype: int64
```

[코드 22] 종속 변수 출력

[단계 ⑥] 학습, 검증, 평가 데이터셋 분리 및 스케일링

⑥-1. 학습, 검증, 평가용 데이터셋 분리

- 좋은 모델은 학습 시에 접한 적 없는 새로운 데이터에 대해 일반화 가능한 성능을 보이는 모델이라고 할 수 있다. 따라서 평가 데이터를 사용하여 AI 분석모델의 일반화 성능을 확인하는 것이 보편적이다. 데이터셋을 용도에 맞게 총 3가지의 학습(train), 검증(validation), 평가(test) 데이터로 나눌 필요가 있다. 학습 데이터셋은 AI 분석모델 학습 시에 사용하고, 검증 데이터셋은 학습 데이터셋으로 훈련된 모델의 성능 측정 시에 사용된다. 모델의 성능이 검증 데이터셋을 통해 일차적으로 검증이 되면, 마지막으로 평가 데이터셋을 사용하여 모델의 성능을 최종적으로 판단한다.

- 모델의 성능 판단을 위해 데이터셋을 검증용과 평가용으로 분리하는 이유는 모델이 학습 데이터에 과적합(overfitting)되지 않았는지 확인하기 위한 목적이다. 과적합된 모델은 학습 데이터에 대해서는 잘 작동하지만, 실제 산업 현장에서 사용될 본 적 없는 데이터(unseen data)에 대해서는 낮은 성능을 보인다. 학습 중간에 검증 데이터를 사용하여 모델의 성능을 검증하고, 이를 바탕으로 모델의 초매개변수를 조정함으로써 학습된 모델이 평가 데이터셋에도 잘 작동하는 우수한 일반화 성능(generalization power)을 확보할 수 있다.
- 본 분석에서는 전체 데이터셋의 70%, 15%, 15%를 각각 학습, 검증, 평가 데이터로 활용한다. 특히 전처리 이후의 시계열 데이터가 시간순으로 존재하므로, 학습-검증-평가 데이터를 시간순으로 분할한다. scikit-learn에서 제공하는 train_test_split 라이브러리를 활용하여 데이터셋을 분리한다. 먼저 학습-평가 데이터를 70%, 30%로 먼저 나누고(test_size인자를 0.3으로 할당), 생성된 평가 데이터를 다시 검증-평가 데이터로 50%, 50% 비율로 분리(test_size인자를 0.5로 할당)함으로써, 학습-검증-평가 데이터의 비율이 70%, 15%, 15%가 되도록 한다.



[그림 20] 학습, 검증, 평가용 데이터셋 분리

```
# 데이터셋 분리, train:validation:test = 70:15:15
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
X_valid, X_test, y_valid, y_test = train_test_split(X_test, y_test, test_size=0.5, random_state=42)

# 분리 이후 데이터 형태
print("X 학습: {}, X 검증: {}, X 평가: {}".format(X_train.shape, X_valid.shape, X_test.shape))
print("y 학습: {}, y 검증: {}, y 평가: {}".format(y_train.shape, y_valid.shape, y_test.shape))
```

```
X 학습: (6497, 14), X 검증: (1392, 14), X 평가: (1393, 14)
y 학습: (6497,), y 검증: (1392,), y 평가: (1393,)
```

[코드 23] 데이터셋 분리 및 형태 확인

⑥-2. 데이터 스케일링(Data Scaling)

- AI 분석모델의 입력 데이터로 활용하기 위해서는 데이터 변수의 크기를 적절히 조정할 필요가 있다. 범위가 큰 값을 가지는 특성에 대해 큰 오차를 맞추어 가중치를 최적화할 수 있다. 값의 범위가 다른 변수들이 있는 경우, 모델 학습 시 변수별로 공평하게 기여하지 못하거나 편향이 발생할 수 있다. 일반적으로 이를 방지하기 위해 변수별로 표준화(standardization) 혹은 정규화(normalization)를 적용한다. 표준화는 데이

터가 평균이 0, 분산이 1인 정규분포를 따르도록 변환하는 방법이고, 정규화는 특성의 스케일을 [0, 1] 범위에 맞추는 것을 의미하며, 대표적으로 최소-최대 스케일 변환(min-max scaling)을 사용하여 정규화를 적용한다.

- 하지만, 만약 데이터 집합에 다른 데이터와 동떨어진 아웃라이어(Outlier)가 존재하는 경우 표준화 적용 시 데이터가 0이 아닌 다른 곳으로 모일 수 있다. 본 실습에서는 아웃라이어의 영향을 최소화하기 위하여 이상치에 대해 강건한 통계치를 활용하는 전처리 방법을 활용하고자 한다.
- scikit-learn(sklearn)에서 제공하는 RobustScaler 클래스는 중앙값이 0, IQR(interquartile range, 사분위수 범위)이 1이 되도록 변환하기 때문에, 아웃라이어가 섞여 있어도 대부분의 데이터는 0 주위에 남아있게 된다. 여기서 IQR은 1사분위수(first quartile, 최솟값과 중앙값을 2등분하는 값)와 3사분위수(thrid quartile, 중앙값과 최댓값을 2등분하는 값) 사이의 범위를 의미한다.

$$X_{\text{scale}} = \frac{x_i - x_{\text{median}}}{Q_3 - Q_1}$$

[그림 21] RobustScaler 공식

- 스케일링 시 주의할 점은 학습용 데이터의 통계량을 활용해 학습용, 검증용, 평가용 데이터셋을 스케일링 해야 한다. RobustScaler를 활용하여 학습 데이터를 기준으로 중앙값과 IQR을 계산한 후 학습, 검증, 평가 데이터셋 스케일링을 실시한다. 학습용 데이터를 입력으로 'fit()' 메서드를 실행하면 중앙값과 IQR을 계산하여 객체내에 저장한다. 다시 학습용 데이터를 입력으로 하여 'transform()' 메서드를 실행하면 저장했던 중앙값을 빼서 새로운 중앙값이 0이 되도록 만들고, 저장한 IQR로 나누어 데이터를 변환 및 출력한다. 두 과정을 합쳐서 'fit_transform()' 메서드를 실행할 수도 있다. 검증용 데이터와 평가 데이터셋 변환 시에는 데이터 정보 유출을 막기 위해 'transform()' 메서드만 실행해야 한다. 학습용 데이터에서 계산된 중앙값과 IQR를 사용하여 검증용 및 테스트 데이터를 변환한다.

```
# 전처리시 필요한 패키지 불러오기
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
X_train_sc = scaler.fit_transform(X_train)
X_valid_sc = scaler.transform(X_valid)

X_test_sc = scaler.transform(X_test)
```

[코드 24] 데이터 스케일링



⑥-3. 데이터 입력 차원 변환

- 본 실습에서 사용하는 1차원 합성곱 신경망은 입력의 형태를 (samples, timesteps, features)의 3차원을 취한다. 현재까지 전처리 완료된 X_train_sc, X_valid_sc, X_test_sc의 경우 모두 (samples, timesteps)의 2차원 형태이므로 3차원의 형태로 늘리는 것이 필요하다. numpy 패키지에서 제공하는 'newaxis' 메서드를 활용하여, numpy 행렬의 차원을 확장시킨다. 즉, 현재 (samples, timesteps)의 2차원 행렬을 (samples, timesteps, 1)의 3차원 행렬로 차원의 크기를 늘린다.

```
X_train_sc = X_train_sc[..., newaxis]
X_valid_sc = X_valid_sc[..., newaxis]
X_test_sc = X_test_sc[..., newaxis]

print(X_train_sc.shape, X_valid_sc.shape, X_test_sc.shape)
```

```
(6497, 14, 1) (1392, 14, 1) (1393, 14, 1)
```

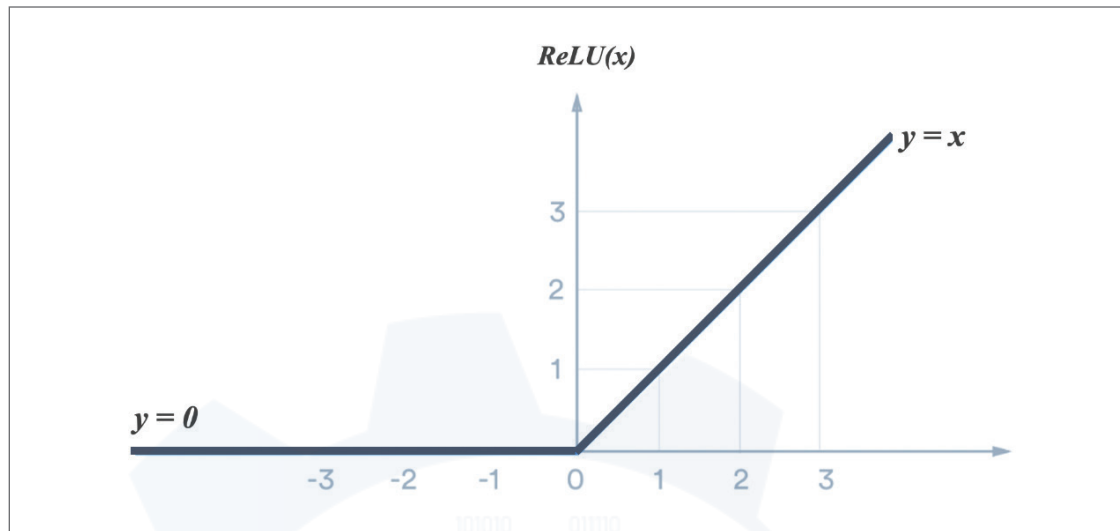
[코드 25] 데이터셋의 행렬 차원 확장

[단계 ⑦] 모델 구축 및 학습

⑦-1. 분석모델 구축

- TensorFlow 라이브러리 내 Keras 패키지의 Sequential 모델을 사용하여 순서대로 연결된 층을 일렬로 쌓아서 구성한다. 훈련에 필요한 계산과 모델을 표현하는 구조를 구성하는 과정에서 사용자가 직접 초매개변수를 설정한다. 사용할 분석모델의 경우 입력층, 은닉층 및 출력층의 크기와 학습 속도 등을 직접 설정할 수 있다.
- 처음 4개의 신경망 층으로 Conv1D(1차원 합성곱 신경망) 은닉층을 사용하고 내부 매개변수로 (1) 필터 수(filters)와 커널 크기(kernel_size), (2) 패딩(padding), (3) 활성화 함수(activation) 및 (4) 모델의 입력 형태(input_shape)를 설정한다.
 - (1) 필터는 곧 커널을 의미한다. 필터 수는 데이터를 순회하는 필터의 개수를 의미하며, 그 수는 출력 특징 맵의 차원의 수가 된다. 즉, 하나의 필터마다 한 차원의 특징 맵을 출력하게 된다. 커널의 크기는 데이터를 순회하는 필터의 크기를 의미한다.
 - (2) 패딩은 경계 처리 방법을 정의한다. 본 실습에서는 'valid'와 'same'을 적용하였다. 'valid'의 경우 유효한 영역만 출력하여, 출력 데이터의 사이즈는 입력 데이터의 사이즈보다 작다. 'same'의 경우 출력과 입력 데이터의 사이즈가 동일하도록 경계를 처리한다.
 - (3) 활성화 함수란 입력된 데이터의 가중 합을 출력 신호로 변환하는 함수이다. Conv1D의 인자로 제공하는 활성화 함수는 총 4개로, 'linear', 'sigmoid', 'softmax', 'relu'이다. 'linear'는 입력 뉴런과 가중치에 계산된 곱값이 그대로 출력으로 나온다. 'sigmoid'는 주로 이진 분류 문제에서, 'softmax'는 다중 클래스분류 문제의 출력층에 주로 쓰인다. 본 실습에서 사용한 활성화 함수인 'relu'는 Rectified Linear Unit(ReLU)의 줄임 표현으로, [그림 22]와 같이 0보다 작은 입력은 0으로 만들고, 0 이상의 입력은 그대로 출력함으로써 빠른 학습을 가능하게

한다. ReLU는 시그모이드(sigmoid)나 하이퍼볼릭탄젠트(tanh)와 같은 다른 활성화 함수보다 간단한 연산을 사용하므로 빠른 학습이 가능하며, 기울기 소실 등의 문제를 해결하는 장점이 있어 매우 빈번하게 사용되는 활성화 함수이다.



[그림 22] ReLU 활성화 함수

- (4) input_shape 은 샘플 수를 제외한 입력 형태를 정의한다. 모델에서 첫 레이어일 때만 정의하며, 본 실습에서는 (N, T, D)의 3차원 텐서 데이터 형태에서 샘플 수 N을 제외한 (T, D)에 해당하는 (14, 1) 크기로 설정한다.
- 이후, 풀링을 위해 MaxPooling1D을 활용한다. 합성곱 층의 출력 결과에서 주요 값, 즉 최대의 값을 선택해 출력의 크기를 감소시킨다. pool_size 인자를 통해 다운샘플링(downsampling)의 대상이 되는 영역의 크기를 설정한다.
- 추출된 주요 특징을 완전 연결 층에 전달하기 위해 Flatten을 활용하여 1차원의 형태로 바꾼다.
- 마지막으로 뉴런을 1개를 가진 완전 연결 층인 Dense 층을 추가하여 출력값이 1개(1일 후의 수주량)가 될 수 있도록 설정한다.

```
# 필요한 패키지 불러오기
import tensorflow as tf
tf.random.set_seed(42)
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Conv1D, MaxPooling1D, Flatten
```

[코드 26] 분석에 필요한 필요 패키지 불러오기



```
model = Sequential()
model.add(Conv1D(filters=64, kernel_size=3, padding='same', input_shape=(14,1), activation='relu')) # (samples,
timesteps, features)
model.add(Conv1D(filters=32, kernel_size=3, padding='same', activation='relu'))
model.add(Conv1D(filters=32, kernel_size=3, padding='valid', activation='relu'))
model.add(Conv1D(filters=32, kernel_size=3, padding='valid', activation='relu'))
model.add(MaxPooling1D(pool_size=2))
model.add(Flatten())
model.add(Dense(16, activation='relu'))
model.add(Dense(1))
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
conv1d (Conv1D)	(None, 14, 64)	256
conv1d_1 (Conv1D)	(None, 14, 32)	6176
conv1d_2 (Conv1D)	(None, 12, 32)	3104
conv1d_3 (Conv1D)	(None, 10, 32)	3104
max_pooling1d (MaxPooling1D)	(None, 5, 32)	0
flatten (Flatten)	(None, 160)	0
dense (Dense)	(None, 16)	2576
dense_1 (Dense)	(None, 1)	17
=====		
Total params: 15,233		
Trainable params: 15,233		
Non-trainable params: 0		

[코드 27] 분석모델 구축

⑦-2. 모델 컴파일(compile)

- 모델을 훈련시키기 전에 훈련 과정에 필요한 매개변수를 정의해야 한다. 훈련 매개변수는 'compile()' 메서드에 전달하며, 메서드를 실행할 때 필요한 매개변수는 다음과 같다.

(1) 손실 함수(loss) : 실제 값과 예측값의 차이를 계산하는 함수이며, 값이 작을수록 오차가 적음을 의미한다. 본 실습에서는 연속형 변수인 수주 수량을 예측하는 문제이므로 이에 적합한 목적식인 평균 절대 오차(mean absolute error, MAE)를 사용한다.

$$\text{MAE} = \frac{\sum_{i=1}^N |y_{\text{실제}_i} - y_{\text{예측}_i}|}{N}$$

[그림 23] 평균 절대 오차(MAE)

- (2) 옵티마이저(optimizer) : 앞서 정의한 손실값을 최소화하는 신경망의 모수를 찾을 수 있도록 하게 하는 최적화 알고리즘이다. 그 예시로 경사 하강법이 있다. 본 실습에서는 다양한 인공신경망 모델 구조에서 좋은 성능을 보이는 'Adaptive Moment Estimation (ADAM)' 을 옵티마이저로 사용한다.

```
optim = tf.keras.optimizers.Adam(learning_rate=0.001)
model.compile(optimizer=optim, loss='mae')
```

[코드 28] 옵티마이저, 손실함수 정의

⑦-3. 모델 학습

- 딥러닝 혹은 인공신경망 기반 모델 학습 시에는 CPU 혹은 GPU를 사용할 수 있다. 일반적으로 사진, 동영상과 같이 용량이 큰 데이터셋 사용 시 병렬처리를 통한 학습 속도 향상을 위해 GPU를 사용한다. 하지만 이런 경우에는 GPU를 추가로 구매하고 설치해야 하므로, 작은 데이터셋을 사용하여 분석할 경우에는 CPU만을 사용하여 AI 분석모델을 학습하기도 한다. 본 분석에서 사용되는 학습 데이터셋은 용량이 크지 않으므로, 본 분석에서는 학습 과정의 편의를 위해 CPU를 사용한다.
- 최종적으로 모델 훈련을 위해 'fit()' 메서드를 사용하며, 필요한 매개변수는 다음과 같다.
 - (1) 입력 및 출력 데이터 : 훈련 데이터의 X와 y가 각각 입력과 출력 데이터이며, 모델을 학습시키는 데 활용된다.
 - (2) 에폭(epochs) : 모델이 전체 데이터를 순회하며 학습한 횟수를 의미하며, 에폭이 클수록 학습 정도는 증가하지만 횟수가 너무 클 경우 과적합 현상이 발생하므로 적절한 훈련 횟수를 설정하는 것이 중요하다.
 - (3) 배치 사이즈(batch_size) : 한 번의 신경망 모수 업데이트 시 사용하는 데이터의 개수를 의미한다. 배치의 크기는 인공신경망 학습 및 성능에 영향을 미친다. 전체 훈련 데이터를 한 번에 학습시키면 학습 시간이 오래 걸리거나 일반화 성능이 떨어지는 것이 관측되기도 하며, 너무 적은 양으로 쪼개어 학습시키면 각 실행 결과의 변동이 생겨 전체 결과값이 불안정해질 수 있으므로 적절한 배치의 크기를 찾아야 한다.
 - (4) 검증용 데이터(validation_data) : 앞서 전체 데이터에서 분리해둔 검증 데이터를 사용한다. 검증 데이터를 이용하면 모델의 과적합 여부와 같은 학습 정도를 파악할 수 있다. 구체적인 설명은 [단계 ⑦-4]에서 설명한다.

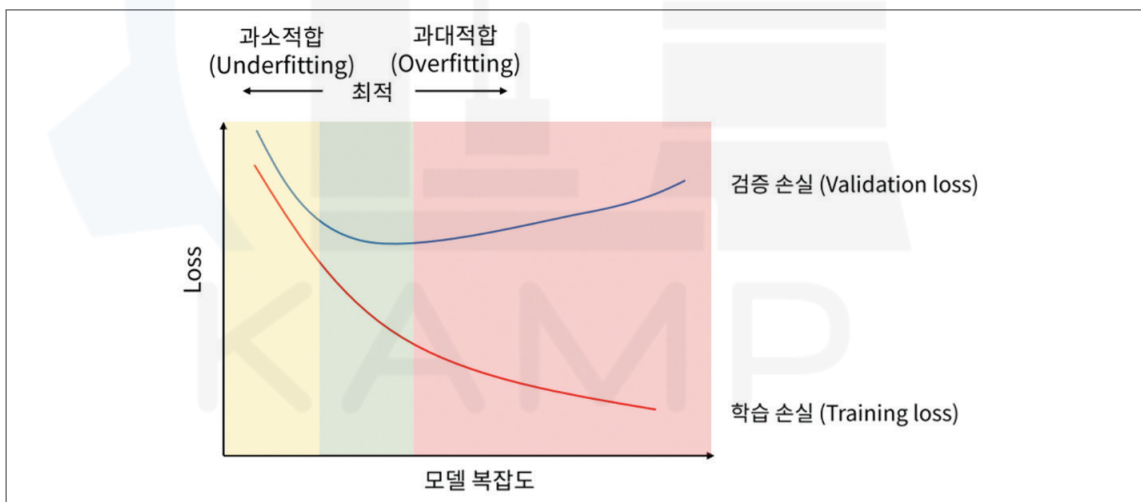
```
history = model.fit(X_train_sc, y_train, epochs=50, batch_size=64,
                    validation_data=(X_valid_sc, y_valid))
```

```
Epoch 1/50
102/102 [=====] - 1s 6ms/step - loss: 65.7318 - val_loss: 45.3580
Epoch 2/50
102/102 [=====] - 0s 5ms/step - loss: 42.5638 - val_loss: 40.6895
...
...
...
Epoch 49/50
102/102 [=====] - 1s 5ms/step - loss: 25.4371 - val_loss: 27.5913
Epoch 50/50
102/102 [=====] - 1s 5ms/step - loss: 25.6827 - val_loss: 27.9193
```

[코드 29] 모델 학습 및 저장

⑦-4. 학습 결과 확인

- 학습 손실 값과 검증용 데이터 손실 값을 비교함으로써 모델의 손실이 줄어들면서 잘 학습되었는지, 과적합이 발생되었는지 등을 확인할 수 있다. 모델의 과적합 여부를 판단하는 방법은 [그림 24]처럼 학습 데이터에 대한 손실은 지속적으로 떨어지지만, 검증 데이터에 대한 손실은 상승하는 것을 확인하는 것이다. 이는 학습 데이터에 지나치게 학습되어 오차가 감소하지만, 실제 평가 데이터와 유사할 것으로 가정한 검증 데이터에 대해서는 예측 오차가 증가하는 것을 의미한다.



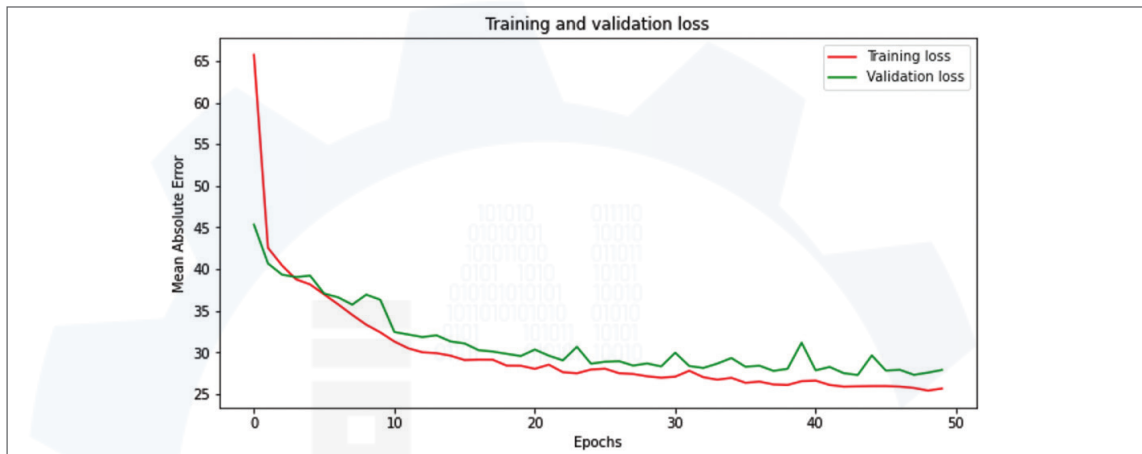
[그림 24] 과적합 판단 기준

- matplotlib 패키지를 활용하여 에폭별 학습 진행 상황을 확인할 수 있다. 'plot()' 메서드를 통해 학습 데이터에 대한 손실값(loss)과 검증 데이터에 대한 손실값(val_loss)을 그래프로 나타낸다. 'title()' 과 'legend()' 함수로 제목과 범례를 만들고 난 후, 'show()' 함수로 해당 그래프를 시각화한다. 범례는 'plot()' 메서드의 인자인 label로 입력된 값으로 생성되므로, 학습 손실값은 'Training loss', 검증 손실값은

'Validation loss'로 나타나는 것을 확인할 수 있다.

```
loss = history.history['loss']
val_loss = history.history['val_loss']
epoch = history.epoch

plt.figure(figsize=(10,5))
plt.plot(epoch, loss, 'r', label='Training loss')
plt.plot(epoch, val_loss, 'g', label='Validation loss')
plt.title('Training and validation loss')
plt.xlabel('Epochs')
plt.ylabel('Mean Absolute Error')
plt.legend()
plt.show()
```



[코드 30] 학습 결과 시각화

[단계 ⑧] 모델 성능 확인

⑧-1. 평가용 데이터셋에 대한 예측 성능 확인

- Keras 패키지의 Sequential 인스턴스의 'evaluate()' 메서드를 사용하여 평가 데이터셋에 대한 성능을 확인한다.

```
model.evaluate(X_test_sc, y_test) # MAE
44/44 [=====] - 0s 1ms/step - loss: 26.5341
26.53412628173828
```

[코드 31] 평가 데이터셋에 대한 성능 확인

⑧-2. 예측값 확인

- 마찬가지로 Sequential 인스턴스의 'predict()' 메서드를 사용하여 모델의 예측 결과인 예측 수주 수량을 확인할 수 있다.

```
y_pred = model.predict(X_test_sc)
y_pred
```

```
array([[521.2062 ],
       [ 5.2946157],
       [140.3364 ],
       ...,
       [ 48.49761 ],
       [ 54.75738 ],
       [ 27.739471 ]], dtype=float32)
```

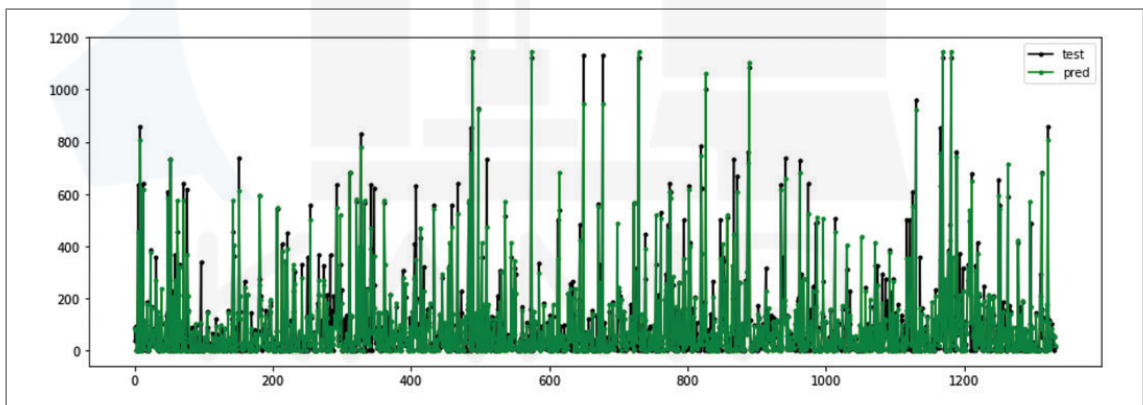
[코드 32] 예측 및 예측 결과 확인

[단계 ⑨] 분석결과에 대한 논의 및 해석

⑨-1. 분석 결과 해석 가이드

- [단계 ⑧]에서 도출한 실제 수주 수량과 모델의 예측 결과에 대한 직관적 이해를 위해 두 데이터를 시각화 한다. 먼저 전체 평가셋에 대한 실제(y_test) 및 예측 결과(y_pred)를 비교한다. 'plot()' 메서드에서 x축 값 인자는 'range()' 메서드에 데이터의 개수(len)를 입력하고, y축 값 인자에는 비교하고자 하는 데이터를 할당한다.

```
plt.figure(figsize=(15,5))
plt.plot(range(len(y_test)), y_test, 'k.-', label="test")
plt.plot(range(len(y_pred)), y_pred, 'g.-', label="pred")
plt.legend()
plt.show()
```

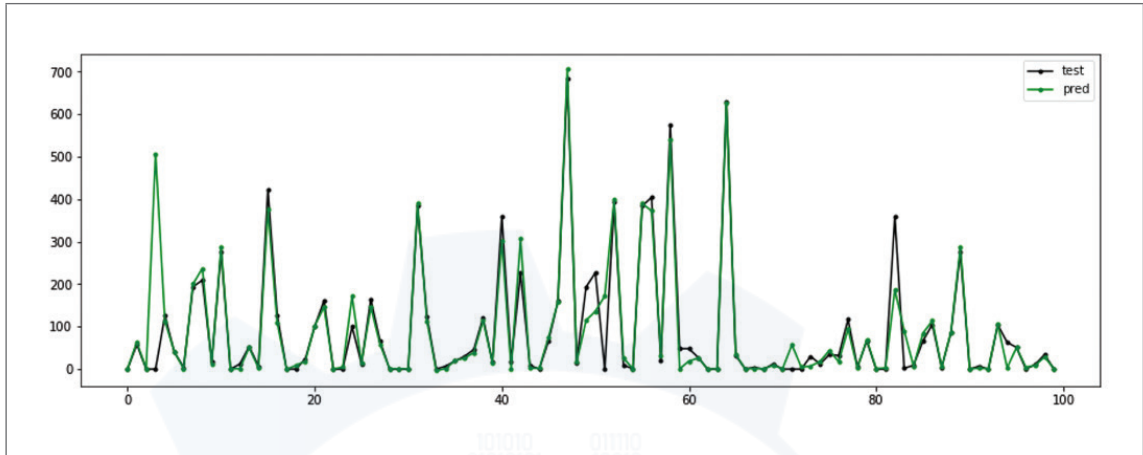


[코드 33] 최종 성능 확인

- 위의 시각자료의 경우 데이터의 수가 많아 예측한 결과가 실제 수주 수량과 얼마나 유사한지에 대한 명확한 비교를 하는 것에 어려움이 있다. 그러므로, 아래와 같이 'plot()' 메서드에서 x축과 y축 값 인자를 통해 100개씩 나누어 예측 결과와 실제 결과를 비교한다. x축 값 인자는 'range()' 메서드를 활용하여 0부터 99까지 총 100개가 나열될 수 있도록 하고, y축 값 인자에는 모델의 예측 수주 수량과 실제 수주 수량의 초기 100개만 설정한다. 모델의 예측 결과인 예측 수주 수량(초록색 실선)이 실제 수

주 수량(검은색 실선)과 유사한 패턴으로 출력하고 있다는 것을 확인할 수 있다.

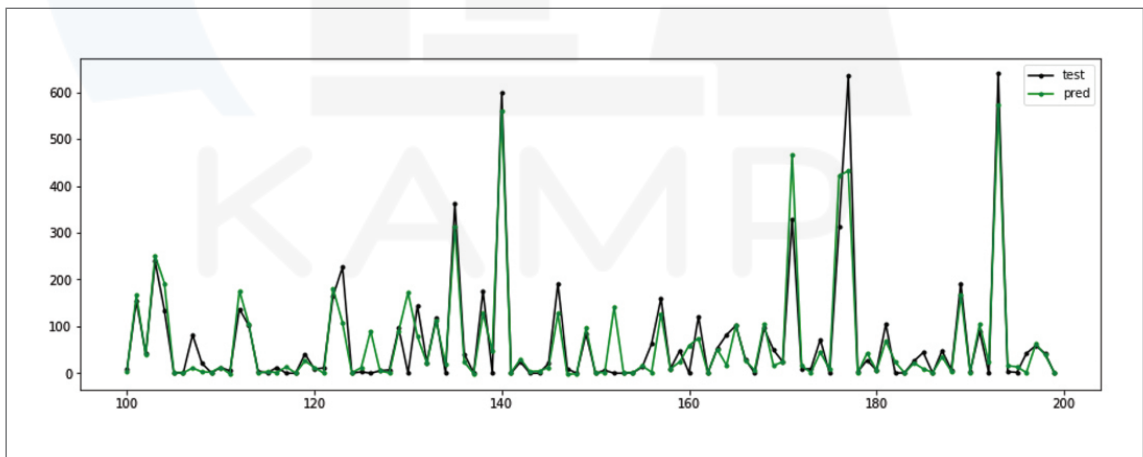
```
plt.figure(figsize=(15,5))
plt.plot(range(100), y_test[:100], 'k.-',label="test")
plt.plot(range(100), y_pred[:100], 'g.-',label="pred")
plt.legend()
plt.show()
```



[코드 34] 최종 성능 확인

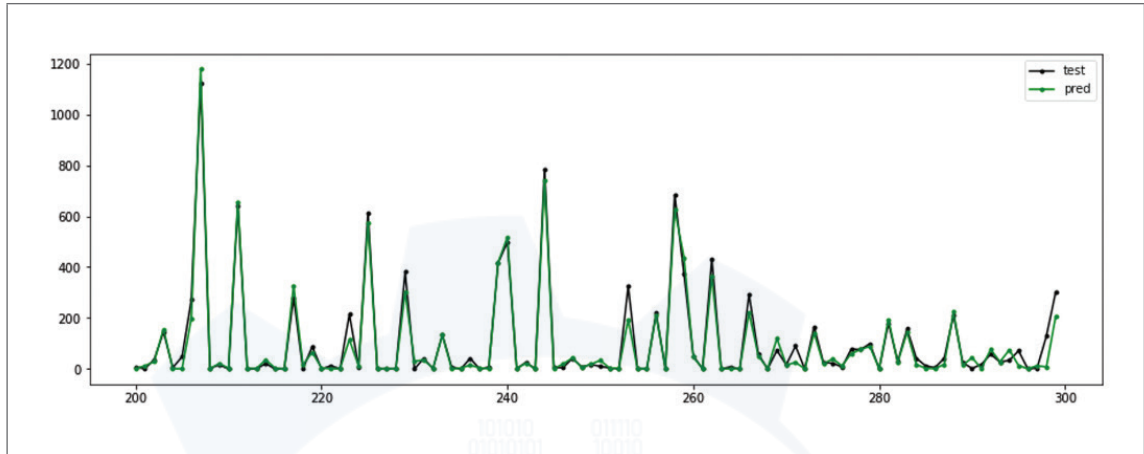
- 마찬가지로 그 다음 100개의 데이터인 200~300번째, 300~400번째 예측 결과와 실제 데이터를 시각화 한다.

```
plt.figure(figsize=(15,5))
plt.plot(range(200,300), y_test[200:300], 'k.-',label="test")
plt.plot(range(200,300), y_pred[200:300], 'g.-',label="pred")
plt.legend()
plt.show()
```



[코드 35] 최종 성능 확인

```
plt.figure(figsize=(15,5))
plt.plot(range(200,300), y_test[200:300], 'k.-',label="test")
plt.plot(range(200,300), y_pred[200:300], 'g.-',label="pred")
plt.legend()
plt.show()
```



[코드 36] 최종 성능 확인

⑨-2. 분석 결과 활용 가이드

- 본 실습에서는 1차원 합성곱 모델을 통해 1~4일전 계획 수주 수량 및 기업의 예정 수량 등을 기반으로 당일 수주 수량을 예측하였다. 위의 시각화 자료를 통해, 예측한 결과가 실제 수주량과 매우 유사하다는 것을 확인할 수 있다.
- 최종 예측 성능은 [코드 31]에 따르면 MAE 기준 약 28이므로, 예측 수량보다 28개 이상의 재고를 보유하고 있는 것이 적절하다고 할 수 있다.



3. 유사 타 현장의 「사출성형 공급망최적화 AI 데이터셋」 분석 적용

3.1 본 분석이 적용 가능한 제조현장 소개

- 본 실습에서는 최적의 재고 관리를 목적으로 공급망 최적화를 위한 AI 기반의 수주량 예측 모델을 구축하였다. 공급망 최적화 기술은 수주량 예측뿐만 아니라 다양한 목적으로 적용될 수 있는데, 공급망을 구성하는 변동적인 요소들의 예측에도 활용 가능하다. 변수 간의 상관관계를 파악하고, 독립 변수를 활용하여 종속 변수의 값을 예측하는 분석을 통해 정확도가 높은 물류량 예측, 소비자 수요 예측 등이 가능할 것으로 예상된다. 특히, AI 분석모델을 활용하여 공급망 내의 다양한 예측을 실시할 때, 관리자의 직접적인 관여 없이도 재고 관리 및 물류 이동이 최적화될 수 있으며, 고객사 등의 니즈의 변화와 공급망 움직임에 대한 계획 수립 및 실행 전략이 유연해질 수 있다. 더 나아가, 급격하게 변하는 시장 수요 변화에 대한 대응성 역시 기대할 수 있을 것이다.
- 기존에는 사출성형 공정의 공급망 계획 수립 시 기업 간 통상적인 수주량을 평균적으로 산출하여 재고관리를 하는 방식으로 진행되어 왔다. 수주량을 예측하여 정확한 수주량을 계획하는 것은 불가능한 일로 여겨지고 있었으나, 본 분석을 통해 구축한 AI 분석모델의 예측 결과를 바탕으로 재고 적정성을 확보할 수 있을 것으로 기대된다.
- 또한, 본 실습은 사출성형 공정에만 한정된 알고리즘이 아닌, 수주-계획-발주의 형태를 따르는 대다수의 공급망을 관리하기 위해 적용할 수 있다. 수집된 데이터의 특성을 파악하고 본 실습을 적절히 적용한다면 효율적인 재고 관리와 물류 최적화 기술 확보를 위한 공급망 경쟁력 강화에 도움이 될 것이라 예상된다.

3.2 본 분석을 원용하여 타 제조현장 적용시, 주요 고려사항

- 예측하고자 하는 값과 연관 있는 변수들을 사용하거나, 필요시 양질의 데이터를 새로 수집하는 과정이 요구된다.
- 당일 수주 수량 및 계획 수주 수량이 누락 없이 정확하게 기입되고 있는지 확인하는 것이 필요하다. 정확하지 않은 데이터를 분석하고 AI 모델을 학습시키는 것은 무의미하다.
- 과거와 현재의 데이터 간에 큰 변화가 있는 경우, 최근 시점의 변화에 대한 가중치를 두어 예측할 필요가 있다. 같은 맥락에서, 불필요한 과거의 데이터를 제거하여 양질의 데이터만으로 예측 정확도를 높이는 등의 학습 유효성을 고려하는 것이 중요하다. 예를 들어, 코로나19와 같이 예측하지 못한 돌발 사건으로 인한 큰 변화가 있을 때 기존의 분석모델을 활용한 예측 결과는 무의미하게 된다. 새로운 예측 결과를 도출하기 위해서는 새로운 데이터를 수집하거나 정제하고, 변수의 종류와 가중치를 모두 새롭게 구성하는 과정이 필요하다.



- 수집하는 데이터의 형태에 대한 논의 및 관리가 필수적이다. 데이터를 수집하는 현장 관리자와 데이터 분석 및 모델 구축을 담당하는 분석 전문가의 충분한 소통을 통해, 분석에 필요한 데이터의 형태를 논의하여 가공 가능한 형태로 데이터를 수집하고 저장함으로써 AI 분석모델 적용의 토대를 구축하는 것이 중요하다.



3 부록_분석환경 구축을 위한 설치 가이드

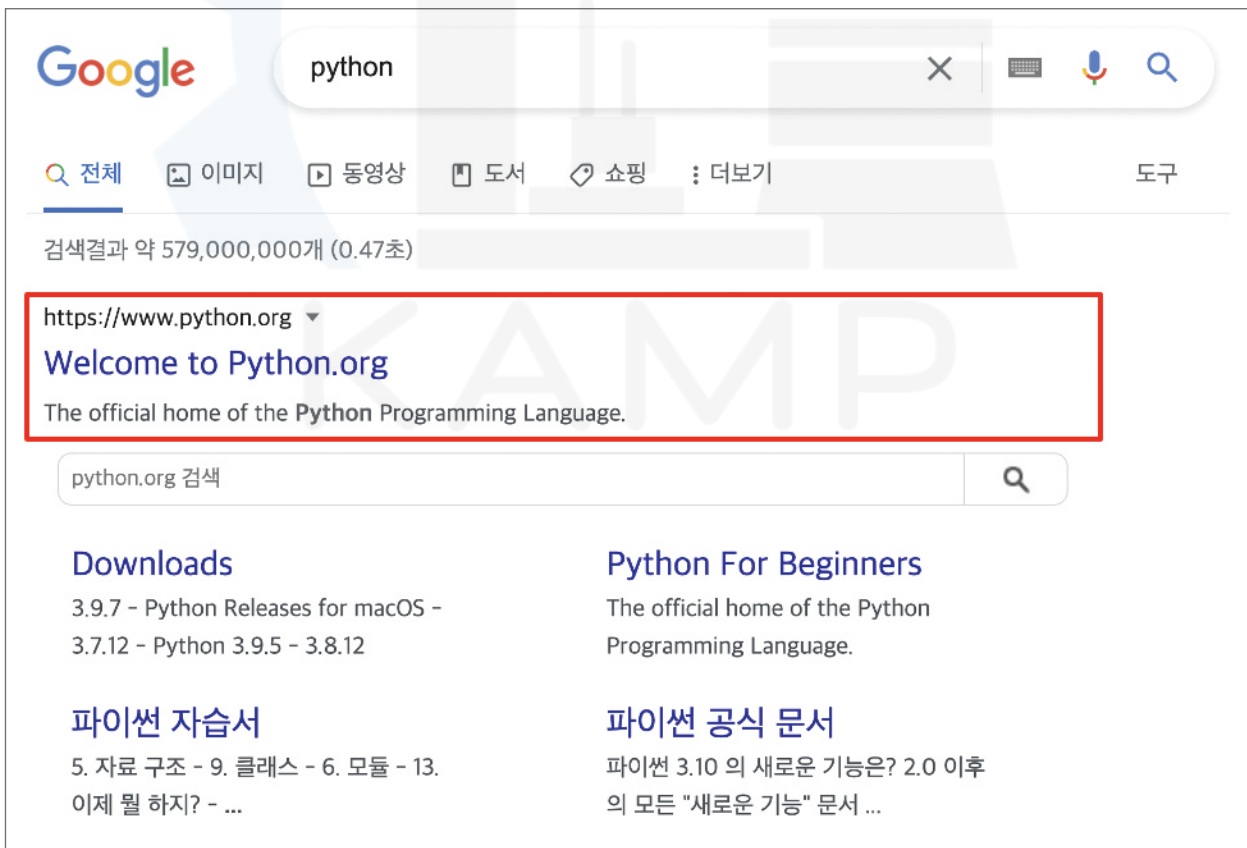
1. 파이썬(Python) 설치

파이썬은 컴퓨터 언어로서 최근 데이터 분석 및 AI 분석모델 개발 등에 널리 사용되는 도구이다. 다운로드 및 설치가 간편하고 활용도가 높은 파이썬을 로컬 컴퓨터에 설치하고 적용하는 방법을 안내한다.

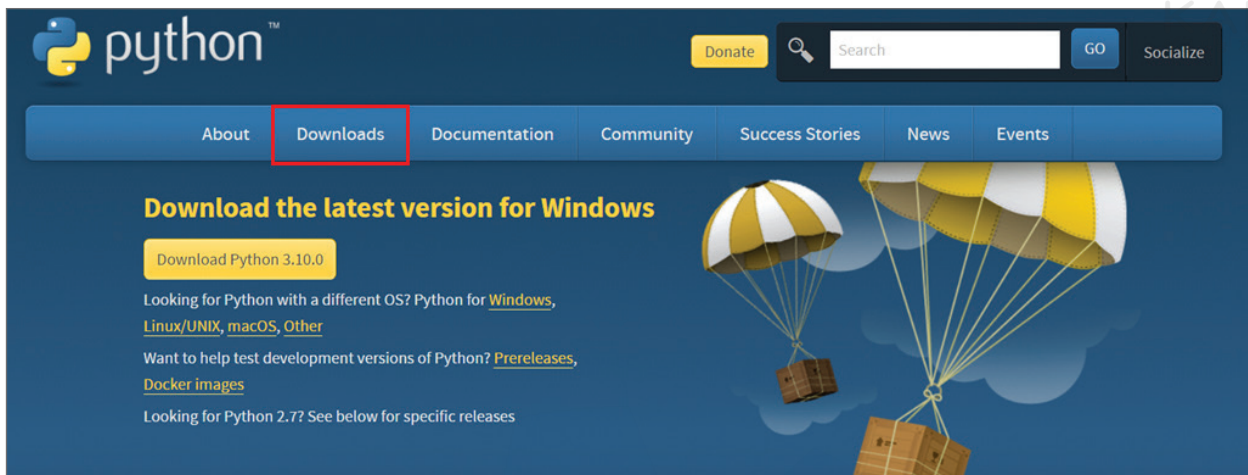
① google.com 등의 검색 엔진에 'python'을 검색한다.



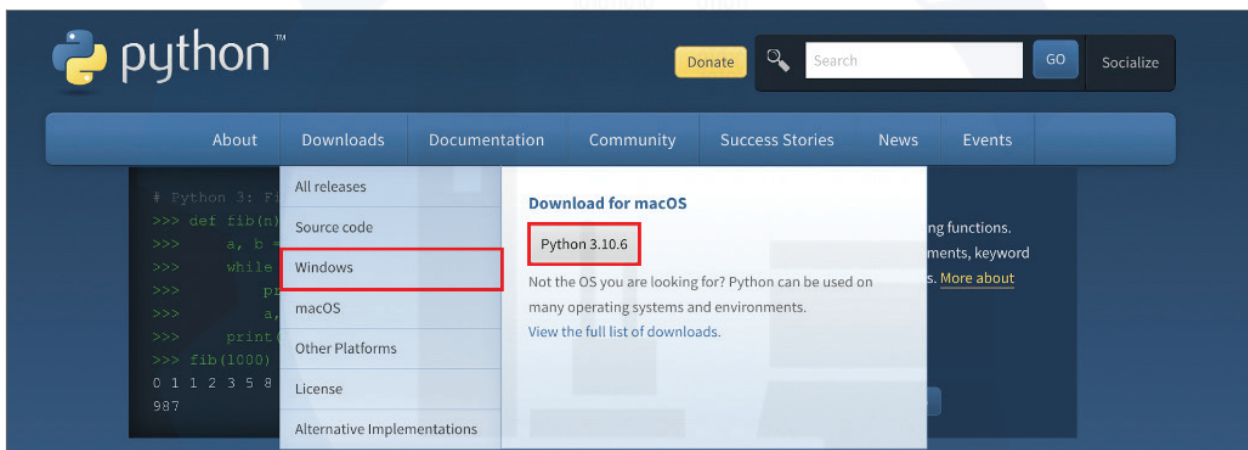
② 제일 처음에 보이는 'Welcome to Python.org'를 클릭한다.



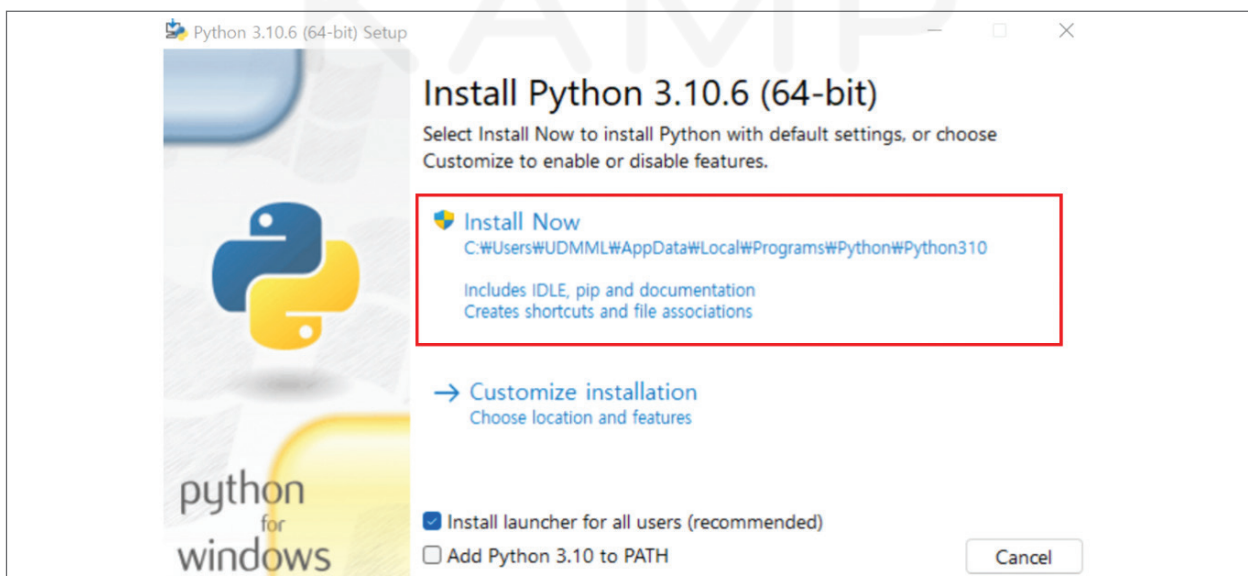
③ 클릭후 보이는 페이지의 정면에서 상단 좌측 2번째 'Downloads'를 클릭한다.



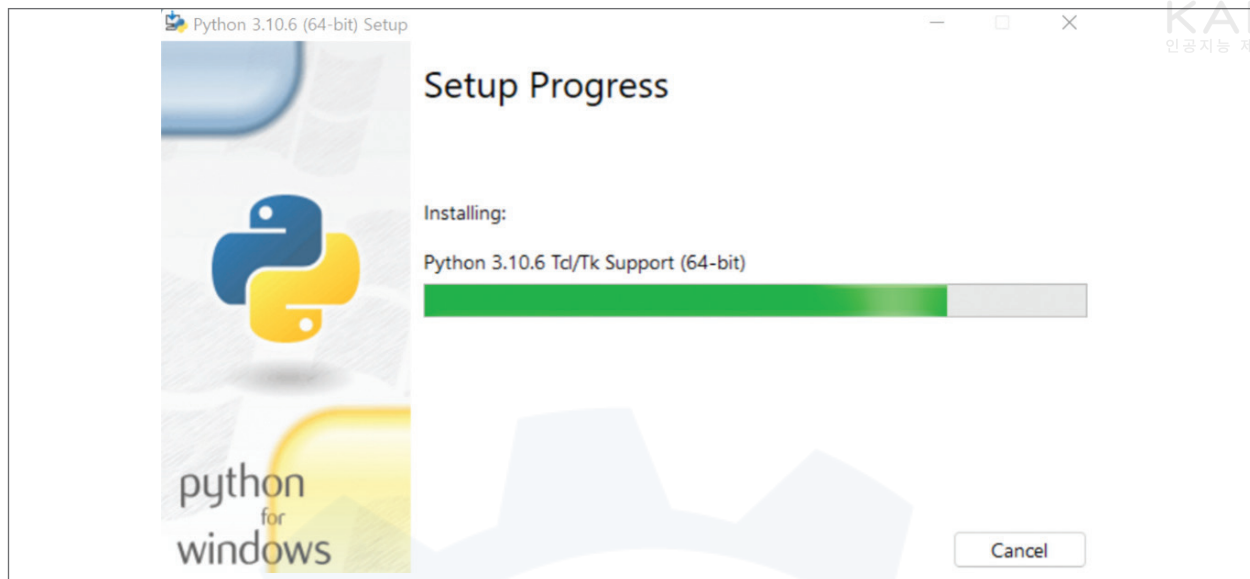
④ 컴퓨터의 운영체제에 따라 상단에서 세번째(macOS의 경우 네번째) 탭을 선택한 후, Python 3.10.6을 다운로드한다. (Python 3.10.6 버전은 업데이트 될 수 있다)



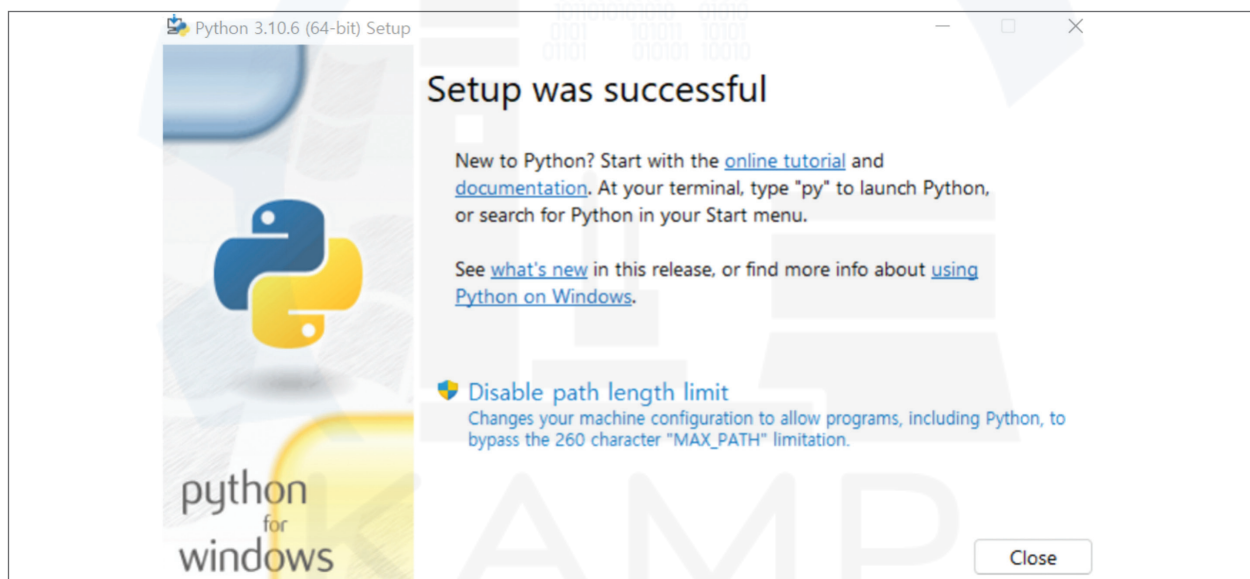
⑤ 아래와 같은 설치창이 뜨면, 'Install Now'를 클릭한다.



⑥ 아래와 같은 설치 진행창이 완료가 될 때까지 유지한다.



⑦ 완료가 되면 아래와 같은 창이 뜨는 것을 확인 후 종료한다. [설치완료]

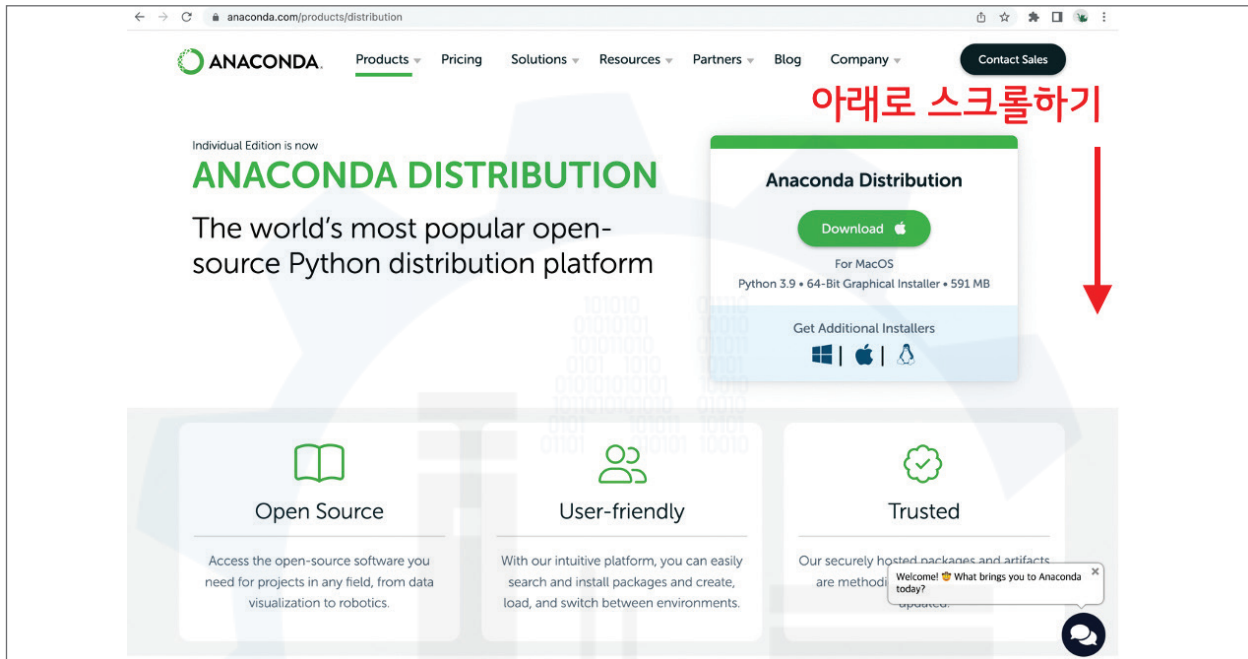




2. 아나콘다(Anaconda) 설치

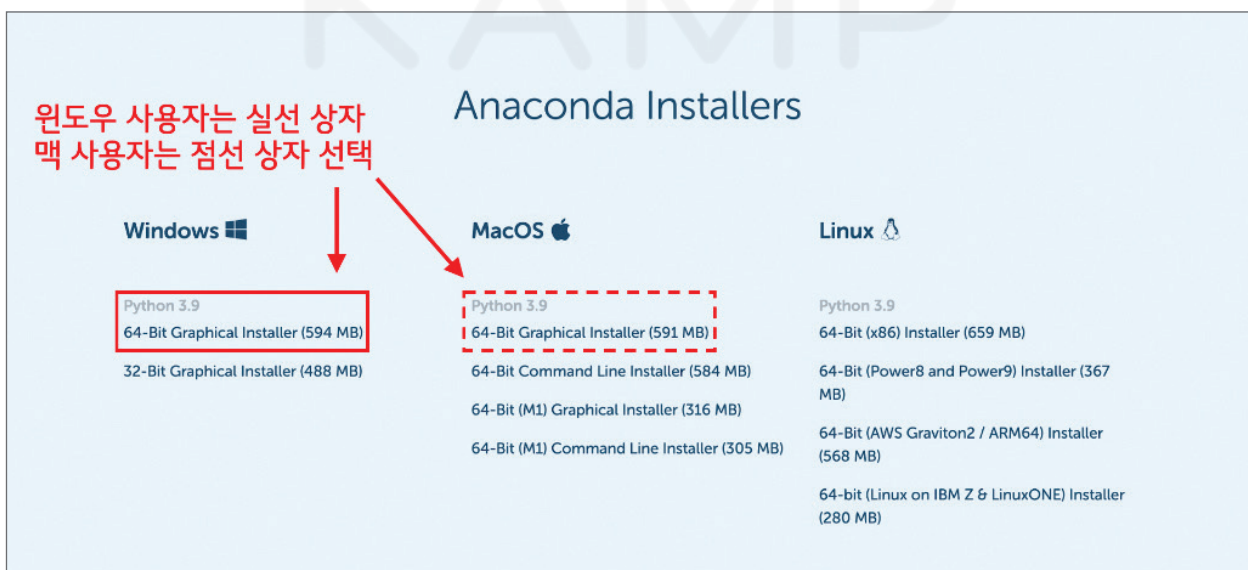
아나콘다란 파이썬과 같은 분석 도구를 사용할 때 필요한 고급 기능 및 분석을 보조하는 도구이다. 아나콘다를 설치함으로써 다양한 기능들을 바로 사용할 수 있고, 결과물을 쉽게 확인할 수 있다. 아나콘다를 설치하고 분석을 실시할 수 있는 환경을 구축하는 방법에 대해 안내한다.

① <https://www.anaconda.com/distribution/> 로 접속한다.



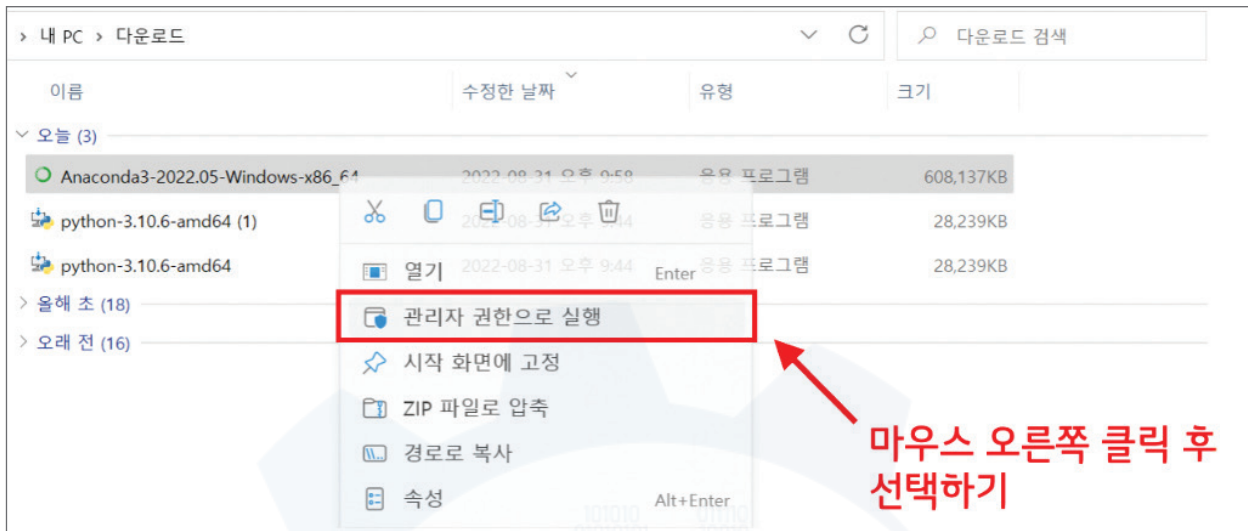
② 다음과 같은 화면이 나올 때 까지 스크롤하여 아래로 내린 후, 로컬 컴퓨터 사용 환경에 맞는 파일을 다운받는다.

- ▶ Windows: 64-Bit Graphical Installer
- ▶ MacOS: 64-Bit Graphical Installer

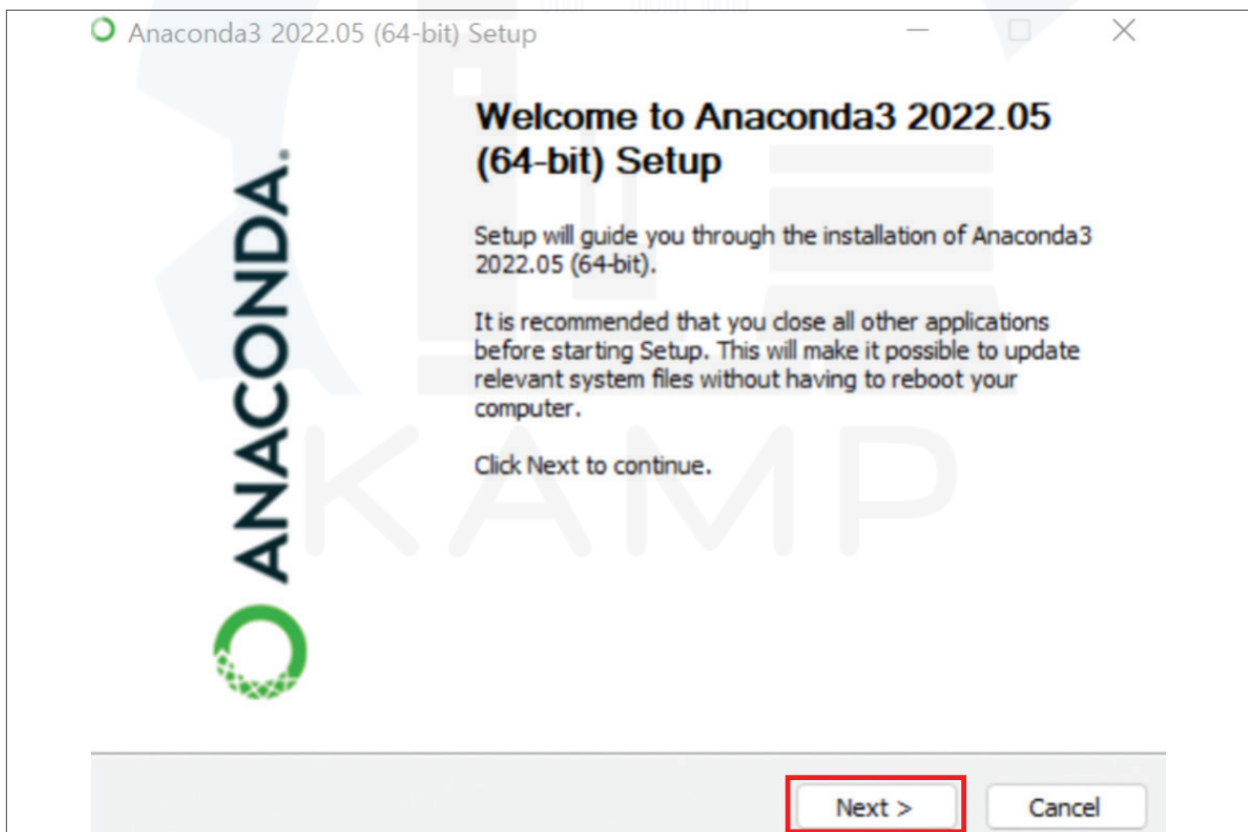


③ 다운로드 받은 파일로 이동하여, 설치된 아나콘다 파일 위에서 마우스 오른쪽쪽을 클릭한 후, 방패모양의 ‘관리자 권한으로 실행’을 클릭한다.

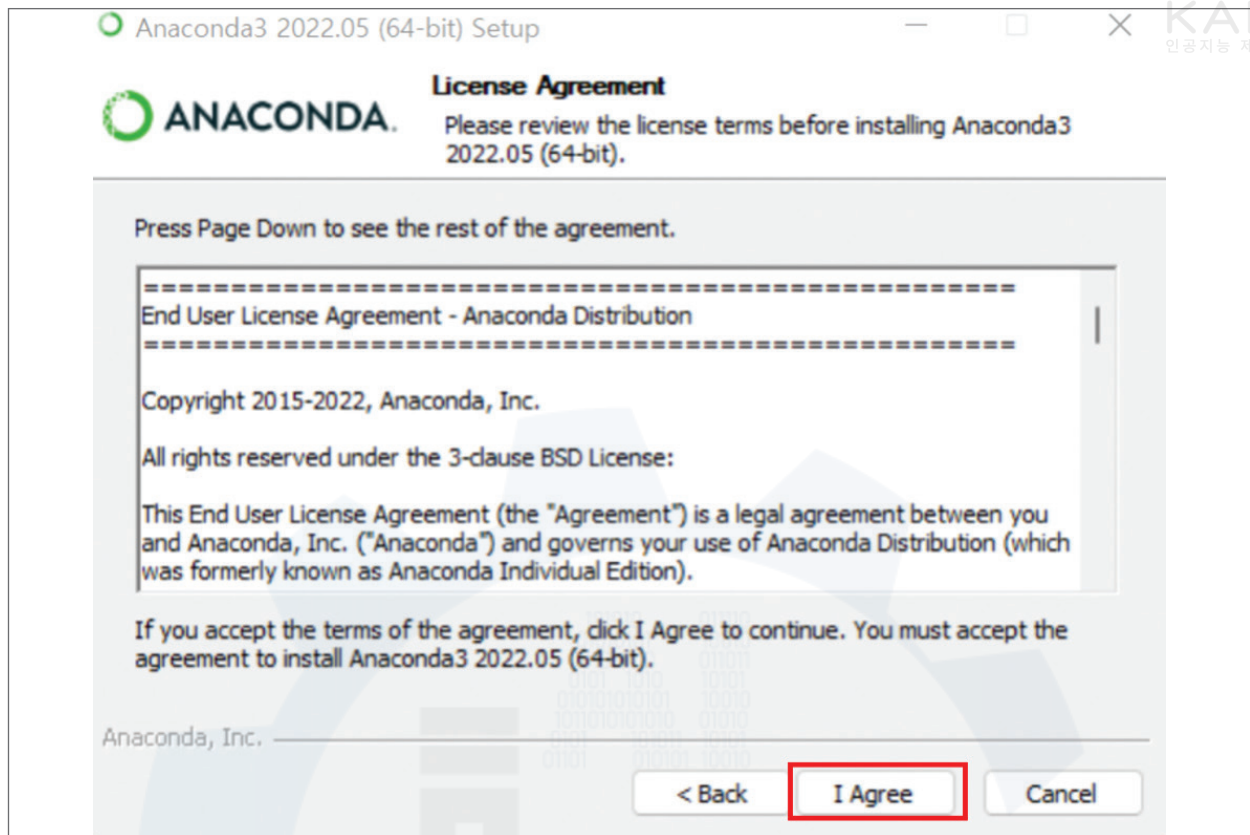
▶ (예) ‘다운로드’ 폴더로 아나콘다를 다운 받은 경우



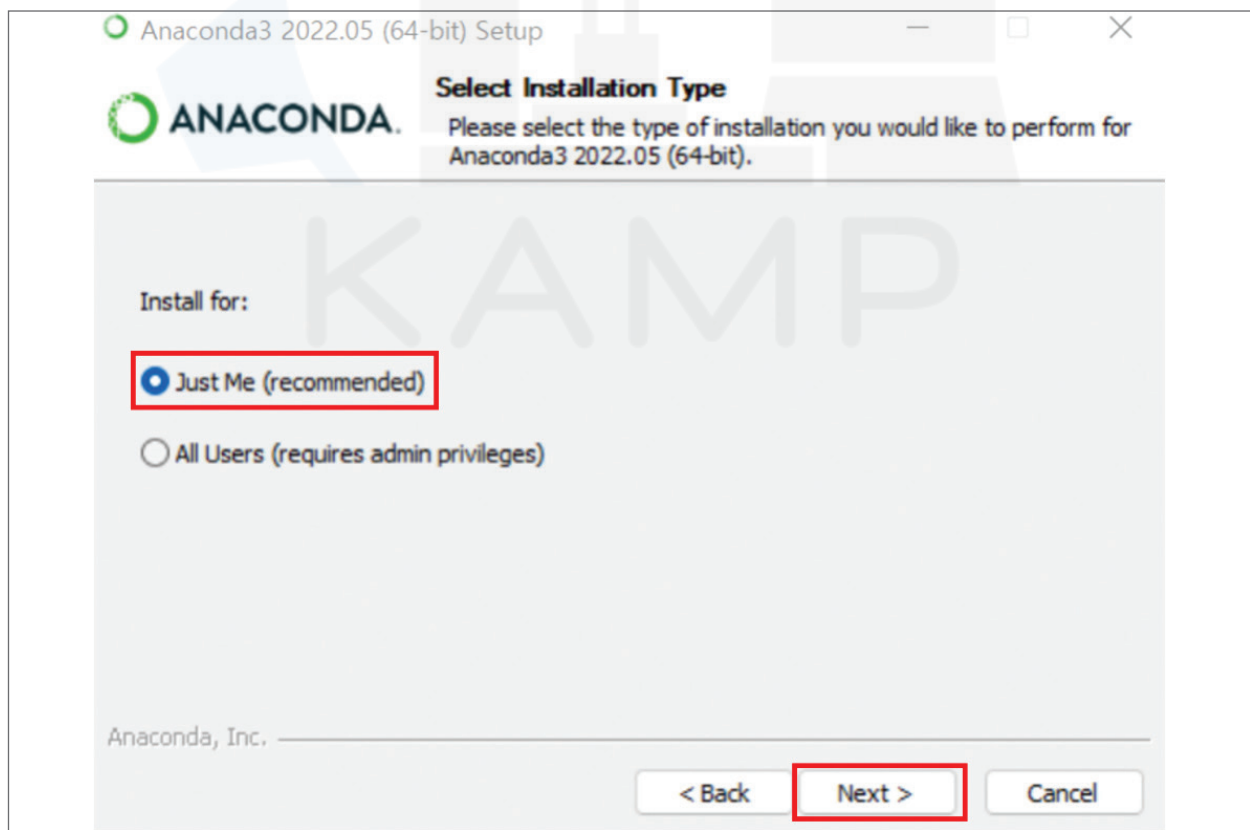
④ 파일을 실행 한 후, ‘Next’ 버튼을 클릭한다.



⑤ 다음 창이 나타나면 'I Agree'를 선택한다.

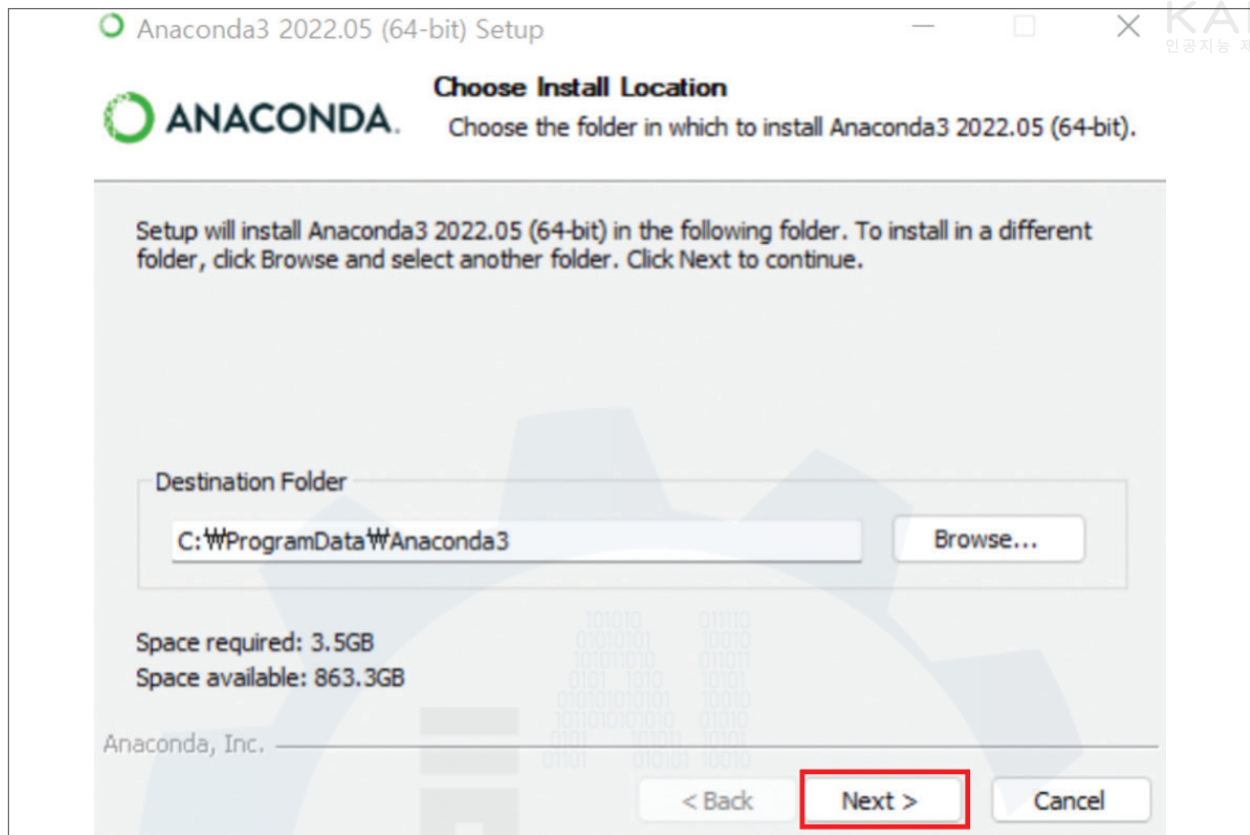


⑥ 셋팅 창이 뜨면 화면의 'Just Me'를 선택 후 아래의 'Next'를 클릭한다.

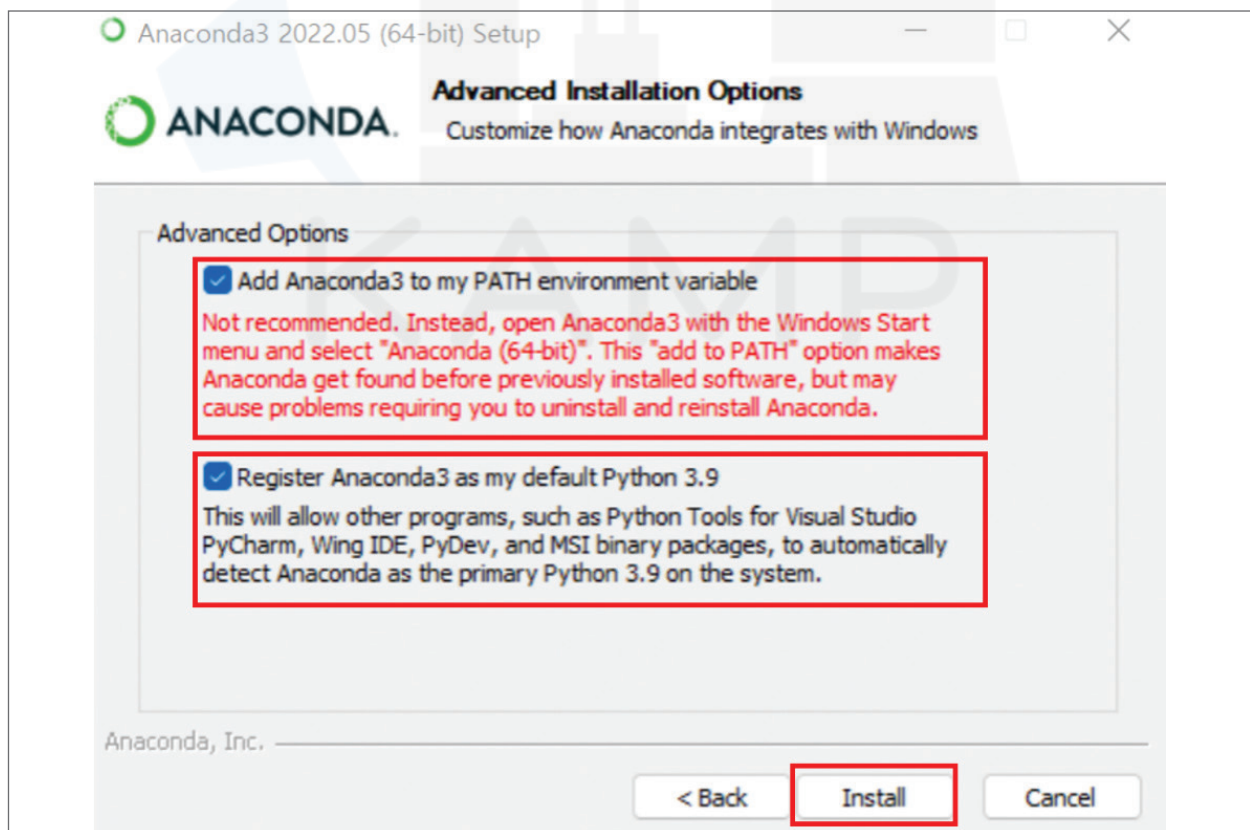




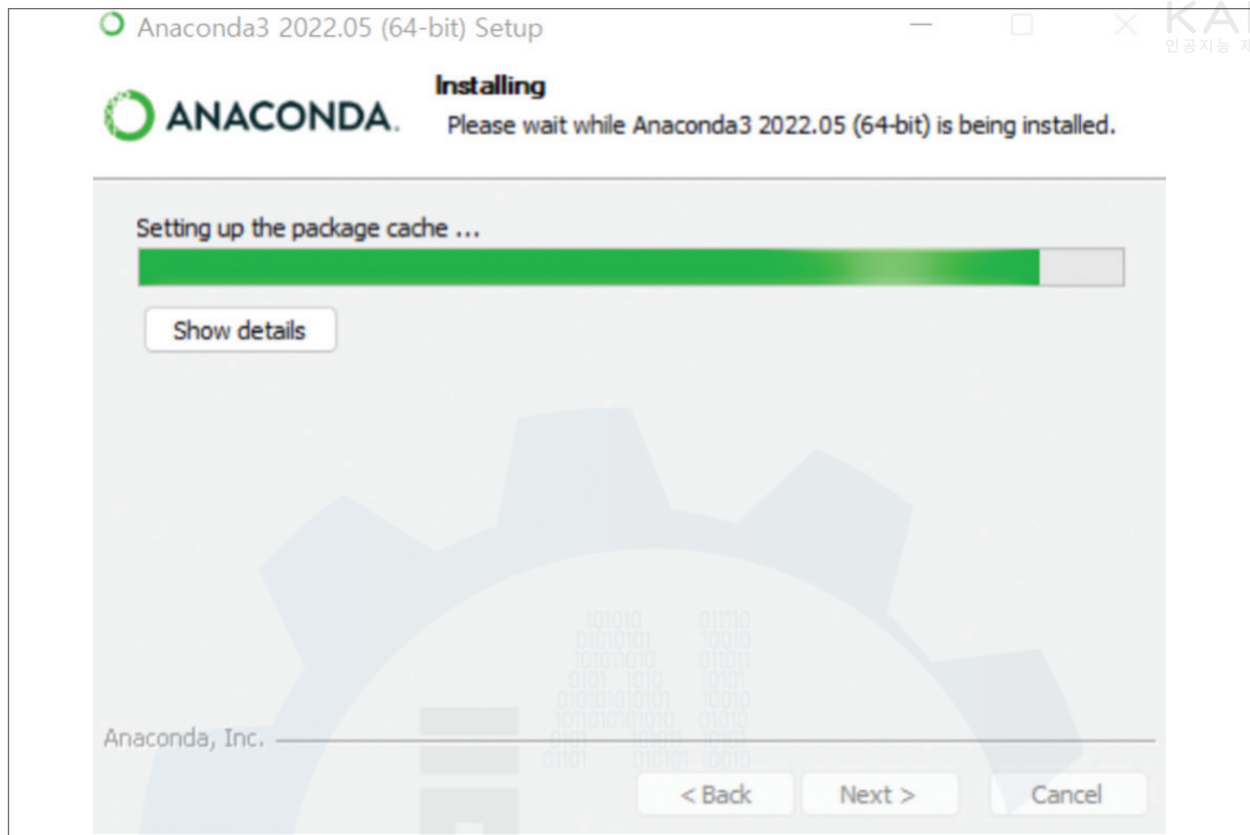
⑦ 다운로드 받을 경로를 물어보는 창에 설치할 경로를 지정하고, 아래의 'Next'를 클릭한다.



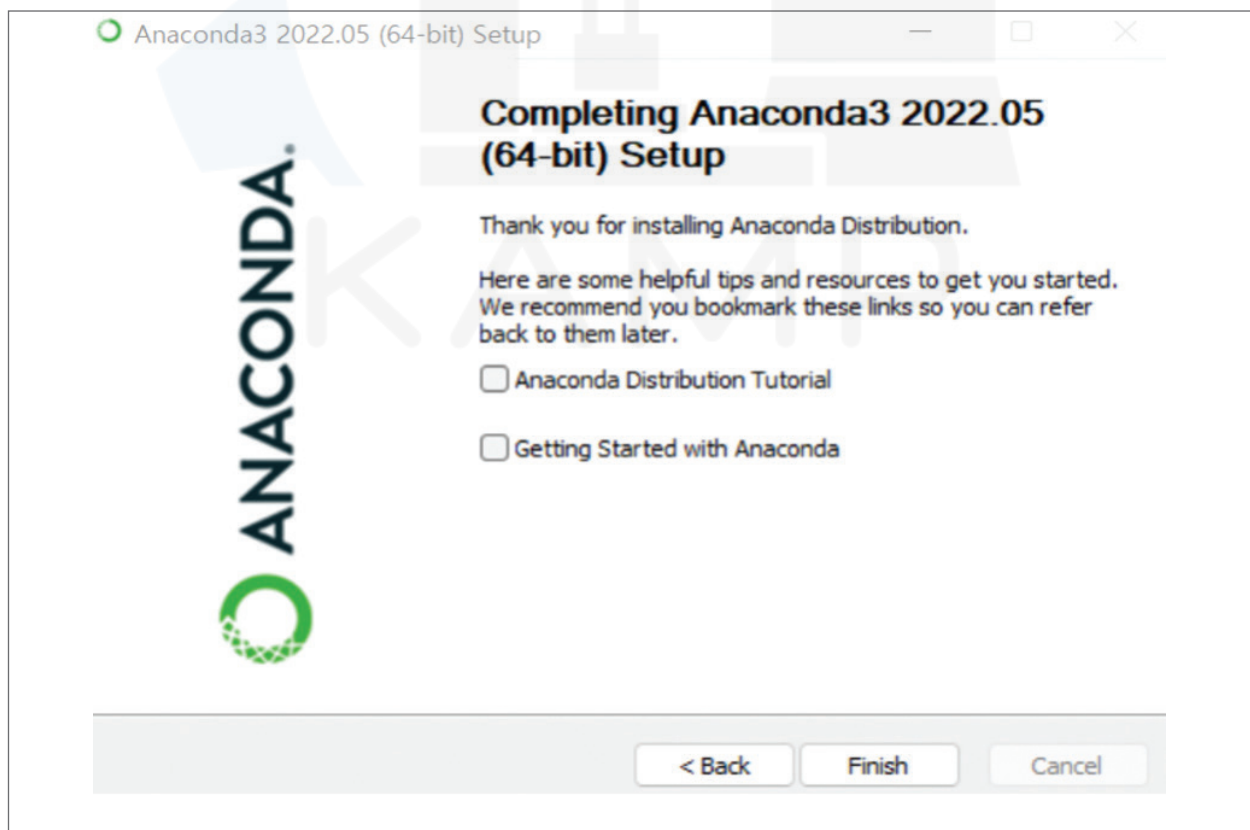
⑧ 고급 옵션 선택창이 뜨면, 아래와 같이 모두 선택 후, 'Install'을 클릭한다.



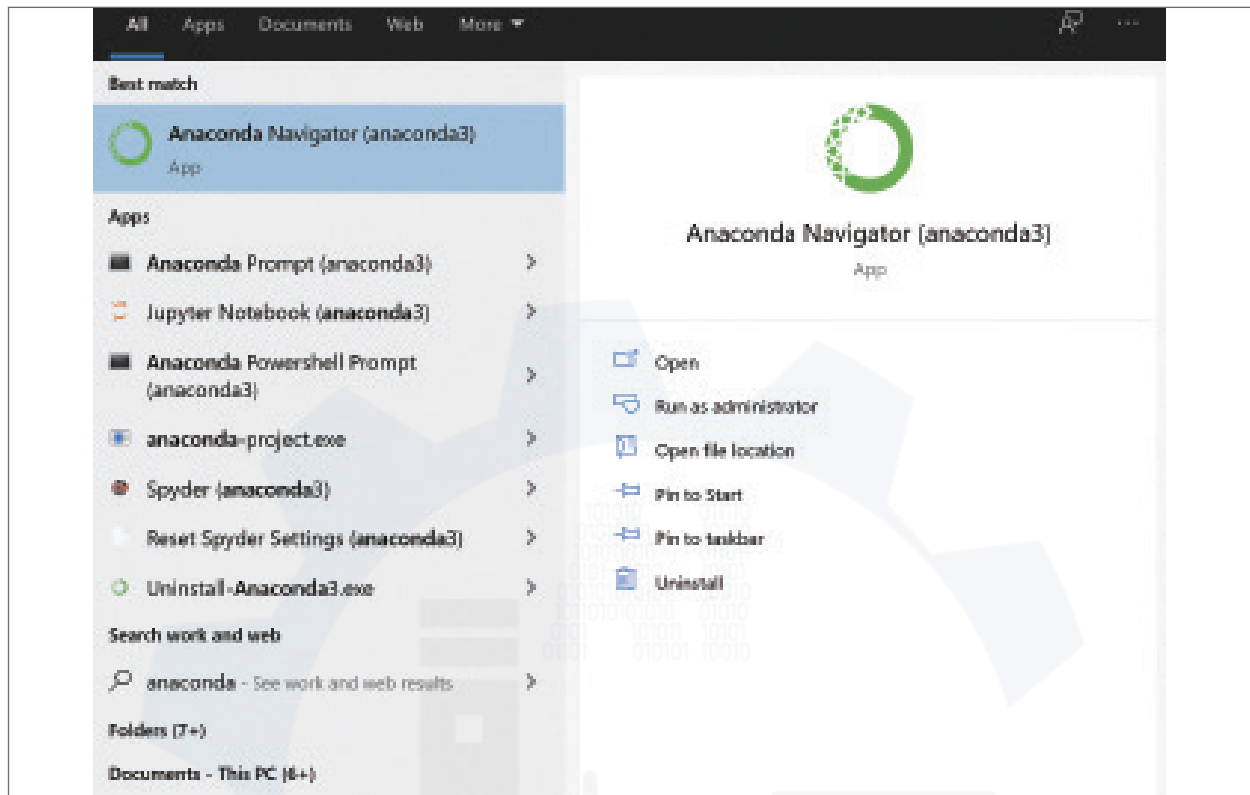
⑨ 다음과 같은 설치창이 뜨면 완료가 될 때까지 대기 (5분이상 소요)



⑩ 마지막 화면에서, 모두 체크 해제한 후, 'Finish'를 눌러 설치를 완료한다.



- ⑪ 화면상의 '홈(🏠)'키를 눌러서 화면과 같이 anaconda prompt가 잘 설치 되었는지를 확인한다. Anaconda Navigator, Anaconda Prompt, Jupyter Notebook 등의 다른 응용 프로그램들도 함께 확인이 된다면 설치가 완료된 것이다.

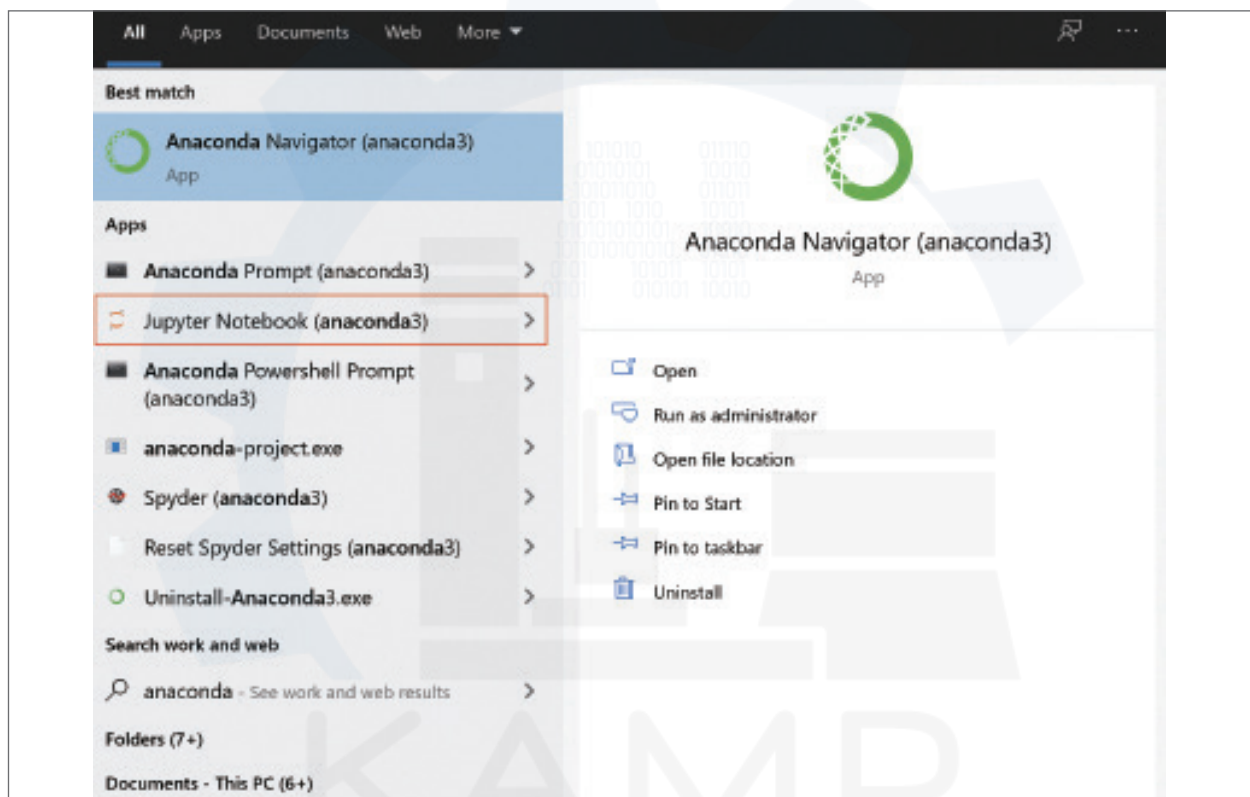




3. 주피터 노트북 (Jupyter Notebook) 실행

주피터 노트북은 실제로 사용자가 코딩(분석 문장 작성)을 할 수 있는 도구이다. 쉽게 비교하자면, 문서 도구로 마이크로소프트 사의 ‘Word’ 프로그램이나, 한컴소프트 사의 ‘한글’ 프로그램 등과 같은 도구라고 생각할 수 있다. 데이터 분석에 다양한 입력, 실행 도구가 있지만, 본 가이드북에서는 주피터 노트북을 활용하는 방법을 안내하기로 한다. Anaconda를 설치한 이후 주피터 노트북(Jupyter Notebook) 설치 방법을 확인하면 된다.

- ① 화면상의 ‘홈(Windows)’키를 눌러서 화면과 같이 ‘anaconda’ 검색 및 폴더 리스트 중 ‘Jupyter Notebook (anaconda3)’을 실행한다.





② Jupyter Notebook을 클릭시 아래처럼 2개의 윈도우가 실행된다.

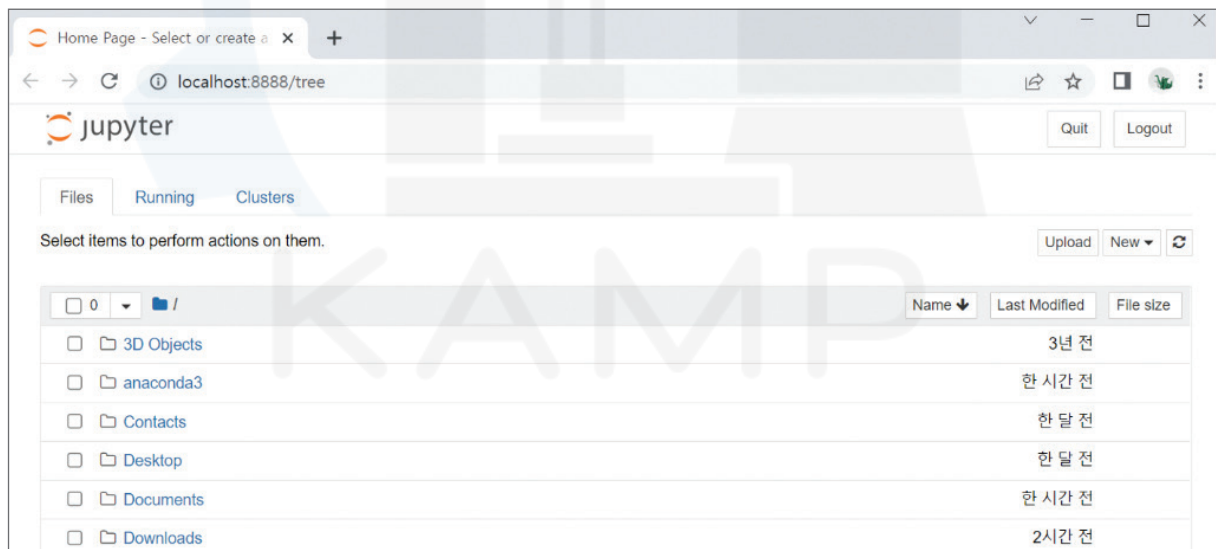
(1) 검은색 배경의 화면은 주피터 노트북이 실행되는 환경에 대한 상태를 나타내는 상태 표시 창이다. 주피터 노트북을 사용하는 동안 종료하면 안된다.

```

Jupyter Notebook (anaconda3)
[W 2022-08-31 23:28:02.700 LabApp] 'notebook_dir' has moved from NotebookApp to ServerApp. This config will be passed to
ServerApp. Be sure to update your config before our next release.
[W 2022-08-31 23:28:02.700 LabApp] 'notebook_dir' has moved from NotebookApp to ServerApp. This config will be passed to
ServerApp. Be sure to update your config before our next release.
[I 2022-08-31 23:28:02.710 LabApp] JupyterLab extension loaded from C:\Users\UDMML\anaconda3\lib\site-packages\jupyterlab
[I 2022-08-31 23:28:02.710 LabApp] JupyterLab application directory is C:\Users\UDMML\anaconda3\share\jupyterlab
[I 2022-08-31 23:28:02.710 NotebookApp] Serving notebooks from local directory: C:\Users\UDMML
[I 2022-08-31 23:28:02.710 NotebookApp] Jupyter Notebook 6.4.8 is running at:
[I 2022-08-31 23:28:02.710 NotebookApp] http://localhost:8888/?token=48c81fb7aa6c69796126efbd6ed99252501720b32361cae0
[I 2022-08-31 23:28:02.710 NotebookApp] or http://127.0.0.1:8888/?token=48c81fb7aa6c69796126efbd6ed99252501720b32361cae0
[I 2022-08-31 23:28:02.710 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 23:28:02.740 NotebookApp]

To access the notebook, open this file in a browser:
file:///C:/Users/UDMML/AppData/Roaming/jupyter/runtime/nbserver-22196-open.html
Or copy and paste one of these URLs:
http://localhost:8888/?token=48c81fb7aa6c69796126efbd6ed99252501720b32361cae0
or http://127.0.0.1:8888/?token=48c81fb7aa6c69796126efbd6ed99252501720b32361cae0
  
```

(2) 사용하는 인터넷 프로그램(크롬, 인터넷 익스플로러)에 주피터 노트북이 열린다. (다른 확장 프로그램 사용하고 싶다면 기본 브라우저를 변경해 주어야 한다.)

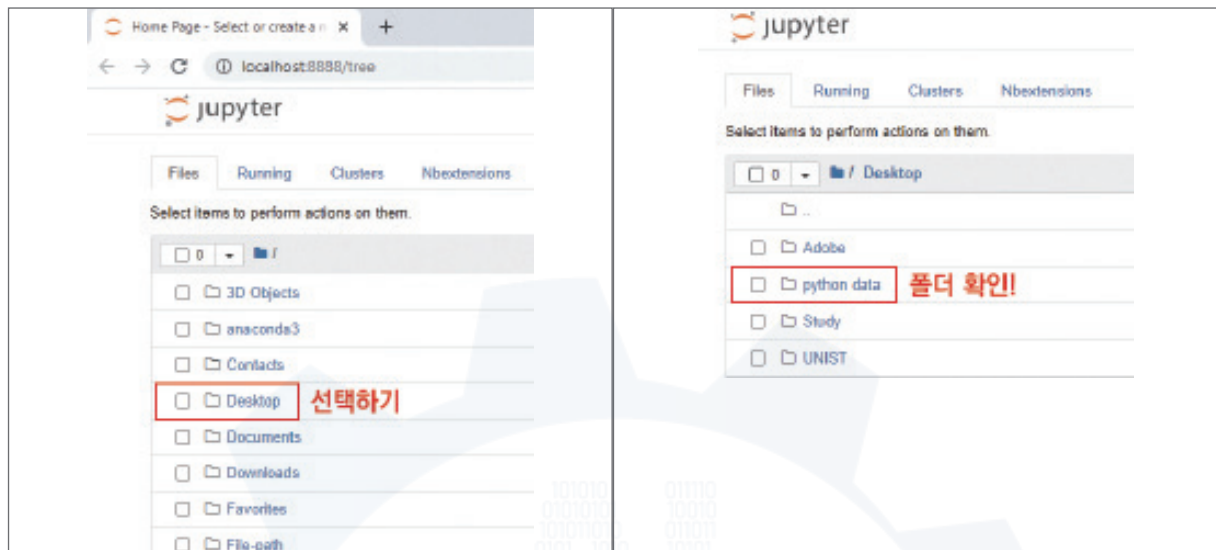




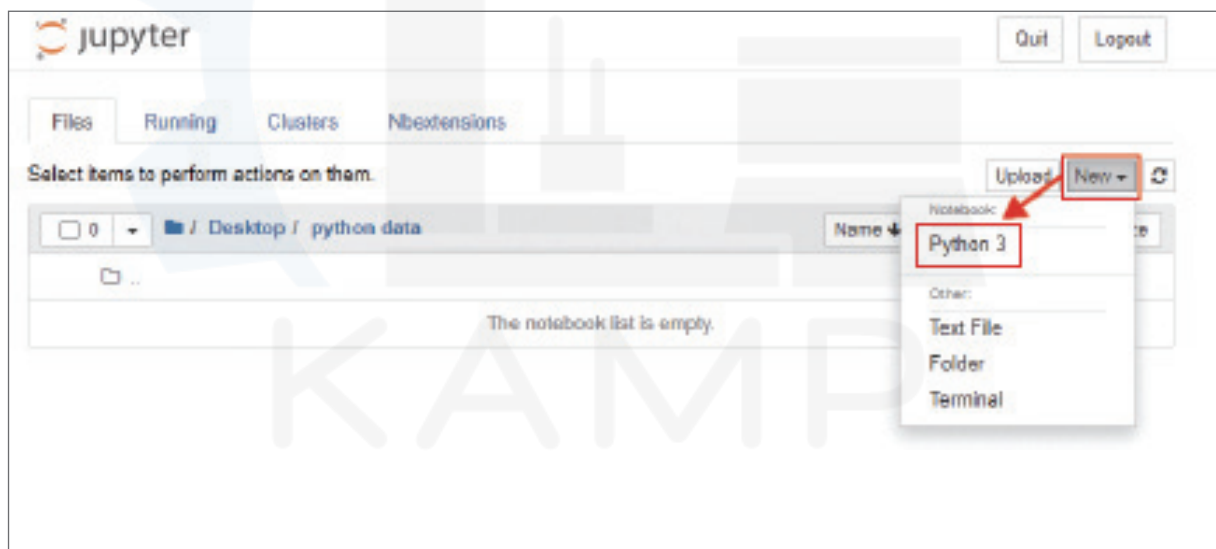
③ 데이터를 저장하고, 불러오고, 분석할 경로의 폴더를 하나 생성한다.

▶ (예) '바탕화면'에 'python data' 폴더를 생성 후 (미리 생성), 파이썬 코드 실행하고 저장한다.

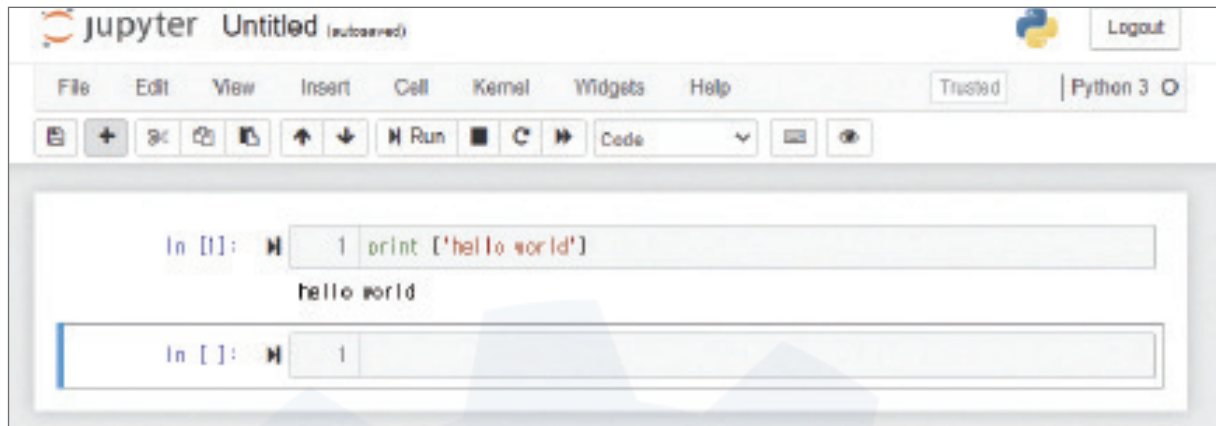
(1) 주피터 노트북에서 'python data' 폴더를 확인한다.



(2) python data에서 파이썬 파일을 생성한다. 오른쪽 상단의 'New'를 누른 후, 'Python 3'을 선택한다.



- (3) 'hello world' 출력을 확인해본다. 보이는 In [] 우측 회색 창에 print('hello world') 입력 후, 상단의  Run 버튼 혹은 shift + Enter 키를 눌러서 실행한다. 코드 파일의 확장자는 '.ipynb'로 저장된다.



4 부록_데이터 품질 전처리 [실습 코드]

앞선 '2.분석 실습'에서 데이터 품질 전처리에 필요한, Gartner에서 제시한 6종의 데이터 품질 평가 지표에 대하여 논의하였다.

(완전성, 유일성, 유효성, 일관성, 정확성, 무결성)

▶ 데이터 품질 확인을 하기 위해 분석에 활용할 '사출성형.csv' 를 df라는 이름으로 불러온다.

```
import pandas as pd
import warnings
warnings.filterwarnings(action='ignore')

df = pd.read_csv('data/사출성형.csv')
```

[코드 1] 데이터 불러오기

▶ 완전성 품질 지수 = $(1 - (\text{결측치} / \text{전체 데이터 수})) \times 100$

① Null 값이 30% 이상인 데이터들은 데이터의 완전성이 떨어지기 때문에 컬럼별 Null 값의 비율을 확인하여 삭제한다. 본 데이터는 30%를 넘는 컬럼이 존재하지 않으므로 컬럼을 제거하지 않는다.

```
perc=30
(df.isna().sum()/len(df))*100
```

Product_Number	False
T일 예정 수주량	False
T+1일 예정 수주량	False
T+2일 예정 수주량	False
T+3일 예정 수주량	False
T+4일 예정 수주량	False
작년 T일 예정 수주량	False
작년 T+1일 예정 수주량	False
작년 T+2일 예정 수주량	False
작년 T+3일 예정 수주량	False
작년 T+4일 예정 수주량	False
T일 예상 수주량	False
T+1일 예상 수주량	False
T+2일 예상 수주량	False
T+3일 예상 수주량	False
T+4일 예상 수주량	False
DateTime	False
DoW	False
Temperature	False
Humidity	False

```
perc=30
(df.isna().sum()/len(df))*100>perc
```

Product_Number	0.0
T일 예정 수주량	0.0
T+1일 예정 수주량	0.0
T+2일 예정 수주량	0.0
T+3일 예정 수주량	0.0
T+4일 예정 수주량	0.0
작년 T일 예정 수주량	0.0
작년 T+1일 예정 수주량	0.0
작년 T+2일 예정 수주량	0.0
작년 T+3일 예정 수주량	0.0
작년 T+4일 예정 수주량	0.0
T일 예상 수주량	0.0
T+1일 예상 수주량	0.0
T+2일 예상 수주량	0.0
T+3일 예상 수주량	0.0
T+4일 예상 수주량	0.0
DateTime	0.0
DoW	0.0
Temperature	0.0
Humidity	0.0

[코드 2] 분석 도구를 활용한 완전성 품질 지수 확인

- ② 데이터의 결측치를 확인하기 위해 'isna()' 메서드를 사용한 뒤 'sum()' 메서드를 이용하여 총 결측치 개수를 구한다.

```
df.isna()
```

Product_Number	T일 예정 수주량	T+1일 예정 수주량	T+2일 예정 수주량	T+3일 예정 수주량	T+4일 예정 수주량	작년 T일 예정 수주량	작년 T+1일 예정 수주량	작년 T+2일 예정 수주량	작년 T+3일 예정 수주량	작년 T+4일 예정 수주량	T일 예상 수주량	T+1일 예상 수주량	T+2일 예상 수주량	T+3일 예상 수주량	T+4일 예상 수주량	DateTime	DoW	Temperature	Humidity
0	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
34612	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
34613	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
34614	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
34615	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
34616	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False

[코드 3] 결측치 확인

```
df.isna().sum()
```

```
Product_Number      0
T일 예정 수주량      0
T+1일 예정 수주량    0
T+2일 예정 수주량    0
T+3일 예정 수주량    0
T+4일 예정 수주량    0
작년 T일 예정 수주량  0
작년 T+1일 예정 수주량 0
작년 T+2일 예정 수주량 0
작년 T+3일 예정 수주량 0
작년 T+4일 예정 수주량 0
T일 예상 수주량      0
T+1일 예상 수주량    0
T+2일 예상 수주량    0
T+3일 예상 수주량    0
T+4일 예상 수주량    0
DateTime            0
DoW                 0
Temperature          0
Humidity             0
dtype: int64
```

[코드 4] column별 결측치 개수 확인

```
cmpt_len=df.isna().sum().sum()
print(cmpt_len)
```

```
0
```

[코드 5] 결측치 개수 확인



③ 구한 결측치의 개수를 이용하여 완전성 품질 지수를 구한다.

```
print("결측치 = %d개 \n완전성 지수 : %.2f%%"%(cmpt_len,(1-cmpt_len/len(df))*100))
#결과는 아래에서 확인 가능하다.
```

```
결측치 = 0개
완전성 지수 : 100.00%
```

[코드 6] 분석 도구를 활용한 완전성 품질지수 구하기

▶ 유일성 품질 지수 = ((유일한 데이터 수)/전체 데이터 수) × 100

: 본 데이터는 'Product_Number'별 'DateTime' 열이 유일한 값을 가지는 기본키이다.

- ① 'groupby()' 메서드를 이용하여 'Product_Number'별 'DateTime' 열이 가지는 유일한 값들을 확인하고, 유일한 데이터 개수를 구한다.

```
df.groupby(['Product_Number','DateTime']).size()
```

```
Product_Number DateTime
Product_84      2022-01-26 18:30      1
              2022-01-27 17:30      1
              2022-01-27 6:05       1
              2022-01-27 6:25       1
              2022-01-27 6:34       1
              ..
Product_f8      2022-05-10 07:06:00    1
              2022-05-10 12:34:00     1
              2022-05-10 15:03:00     1
              2022-05-10 15:38:00     1
              2022-05-11 07:04:00     1
Length: 33894, dtype: int64
```

[코드 7] 분석 도구를 활용한 유일한 값들 확인하기

```
len(df.groupby(['Product_Number','DateTime']).size())
```

```
33894
```

[코드 8] 분석 도구를 활용한 유일한 값들의 개수 확인하기

② 전처리 이전의 유일성 지수 값

: 구한 유일한 데이터 개수를 이용하여 유일성 품질지수를 구한다.

```
uniq_len=len(df.groupby(['Product_Number','DateTime']).size())
print("유일성 지수 : %.2f%%"%((uniq_len/len(df))*100)) #결과는 아래에서 확인 가능하다.
```

```
유일성 지수 : 97.91%
```

[코드 9] 분석 도구를 활용한 유일성 품질지수 구하기



③ 전처리 이후의 유일성 지수 값

: 본 데이터에는 모든 수주 주문 로그가 기록되어 있다. 즉, 최종 주문 로그뿐만 아니라 모든 수주 주문 로그 데이터들이 수집되어 있다. 전처리과정에서 불필요한 행들을 제거하여 최종 일일 수주 주문 데이터만 저장하여 분석에 사용한다. 불필요한 행을 제거한 데이터를 valid_df에 저장하고 전처리 이후의 유일성 지수는 아래와 같다.

```
# [단계 ③] 데이터 정제(전처리) 과정을 거친 데이터 valid_df로 유일성 품질지수를 구한다
df['Date'] = pd.to_datetime(df['DateTime']).dt.date
valid_df = df.drop_duplicates(['Product_Number', 'Date'], keep='last')
valid_df.reset_index(drop=True, inplace=True)
uniq_len=len(valid_df.groupby(['Product_Number','DateTime']).size())
print("유일성 지수 : %.2f%%" % ((uniq_len/len(valid_df))*100)) #결과는 아래에서 확인 가능하다.
```

유일성 지수 : 100.00%

[코드 10] 분석 도구를 활용한 전처리 후의 유일성 품질지수 구하기

▶ 유효성 품질 지수 = (유효성 만족 데이터 수/전체 데이터 수) × 100

① 데이터가 유효 범위 내에 있는가?

: 데이터셋에 정의된 수집 범위를 이용하여 조건(유효 범위)을 모두 충족하는 데이터들을 valid_df에 저장한다. 본 데이터는 전체 부품 117개의 수주 주문 로그가 저장되어 있는 데이터다. 'nunique()' 메서드를 사용하여 고유한 'Product_Number'의 개수가 117개 인지 확인을 한다.

```
print(df['Product_Number'].nunique())
```

117

[코드 11] 유효 범위 내에 있는 데이터 개수 확인하기

: 모든 데이터가 유효 범위 내에 있는 것을 알 수 있다.

```
valid_df=df
```

[코드 12] 분석 도구를 활용하여 유효 범위내에 있는 데이터를 valid_df에 저장하기

② 데이터가 형식에 맞는가?

: 'DateTime' 열의 타입을 datetime으로 변경할 때 errors='coerce' 옵션을 추가한다. errors='coerce' 옵션은 올바르지 않은 날짜 형식을 Null값으로 채워주는 옵션이다.

```
valid_df['DateTime']=pd.to_datetime(valid_df['DateTime'],format='%Y-%m-%d %H:%M',errors='coerce')
```

[코드 13] 분석 도구를 활용하여 데이터 형식 맞춰주기

: 'DateTime' 이 Null인 개수를 'isnull()' 메서드를 사용한 뒤 'sum()' 메서드를 통해 확인한다. 잘못된 데이터의 개수는 0개로 모든 데이터의 형식이 올바르다는 것을 알 수 있다.

```
valid_df['DateTime'].isnull().sum()
```

```
0
```

[코드 14] 분석 도구를 활용한 데이터 형식 확인하기

: 'Product_Number' 열의 데이터 형태가 'Product_XX' 형태여야 한다. 해당 열의 데이터에 유효하지 않은 형식이 들어있는지 'Product_Number' 열 데이터의 앞에서 8자리까지 문자열을 추출하고 'unique()' 메서드를 이용하여 확인한다. [코드 15]의 결과를 보면 모든 값이 'Product_'로 시작되므로 유효성을 위배하는 데이터가 없는 것을 알 수 있다.

```
valid_df['Product_Number'].str[:8].unique()
```

```
array(['Product_', dtype=object)
```

[코드 15] 분석 도구를 활용한 데이터 유효성 검사

③ 수집된 날짜 안에 들어가 있는가?

: 2022년 01월 26일부터 2022년 05월 11일에 수집된 데이터이기 때문에, 데이터가 해당 기간을 벗어나지 않았는지 확인한다.

```
import datetime
d0=pd.Timestamp(datetime.date(2022,1,26)) #데이터 수집 시작 날짜
d1=pd.Timestamp(datetime.date(2022,5,12)) #데이터 수집 종료 날짜+1
con1=valid_df['DateTime']>=d0
con2=valid_df['DateTime']<=d1
valid_df=valid_df[con1&con2]
```

[코드 16] 분석 도구를 활용하여 데이터 수집 기간 확인하기

④ 따라서 3가지 경우를 모두 만족하는 데이터는 valid_df에 저장하고, 최종적으로 완성된 데이터를 이용하여 유효성 품질 지수를 구한다.

```
valid_len=len(valid_df)
print("유효성 지수 : %.2f%%"%(valid_len/len(df)*100)) #결과는 아래에서 확인 가능하다.
```

```
유효성 지수 : 100%
```

[코드 17] 분석 도구를 활용하여 유효성 품질 지수 최종 확인하기



▶ 일관성 품질 지수 = (일관성 만족 데이터 수/ 전체 데이터 수) × 100

: 데이터가 구조, 값, 형태를 일관성 있게 지키고 있는지 확인한다. 'DateTime' 열의 데이터가 일관된 날짜 형태의 데이터를 가졌는지, 'Product_Number' 열의 데이터는 'Product_XX'의 형태를 가졌는지는 유효성을 확인하면서 확인하였다. 'dtypes()' 메서드를 이용하여 이외의 수주 수량 데이터 값들의 형태가 int64인지 확인한다.

df.dtypes

```
Product_Number    object
T일 예정 수주량    int64
T+1일 예정 수주량  int64
T+2일 예정 수주량  int64
T+3일 예정 수주량  int64
T+4일 예정 수주량  int64
작년 T일 예정 수주량  int64
작년 T+1일 예정 수주량 int64
작년 T+2일 예정 수주량 int64
작년 T+3일 예정 수주량 int64
작년 T+4일 예정 수주량 int64
T일 예상 수주량    int64
T+1일 예상 수주량  int64
T+2일 예상 수주량  int64
T+3일 예상 수주량  int64
T+4일 예상 수주량  int64
DateTime          datetime64[ns]
DoW               object
Temperature        float64
Humidity           float64
dtype: object
```

[코드 18] 분석 도구를 활용하여 데이터 일관성 확인하기

▷ 본 데이터는 일관성이 있다고 할 수 있다.

▶ 정확성 품질 지수 = (1-(정확성 위배 데이터 수/전체 데이터 수)) × 100

① Product 별 데이터가 수집된 기간은 95일로 동일해야한다. 'DateTime' 열의 데이터에서 'date'를 이용해 날짜를 'Date' 열에 저장한다. 'drop_duplicates' 메서드를 사용하여 'Product_Number'와 'Date' 기준으로 중복값이 있으면 마지막 값을 제외하고 제거한다.



```
df['Date'] = pd.to_datetime(df['DateTime']).dt.date
valid_df = df.drop_duplicates(['Product_Number', 'Date'], keep='last')
valid_df
```

Product_Number	T일 예정 수주 량	T+1 일 예 정 수 주량	T+2 일 예 정 수 주량	T+3 일 예 정 수 주량	T+4 일 예 정 수 주량	작년 T 일 예 정 수 주량	작년 T+1일 예 정 수 주량	작년 T+2일 예 정 수 주량	작년 T+3일 예 정 수 주량	...	T일 예 상 수주 량	T+1 일 예 상 수 주량	T+2 일 예 상 수 주량	T+3 일 예 상 수 주량	T+4 일 예 상 수 주량	DateTime	DoW	Temperature	Humidity	Date
9	Product_ad	0	0	46	16	35	52	42	50	0 ...	32	43	135	196	0	2022-02-16 07:03:00	Wednesday	12.673571	13.080714	2022-02-16
40	Product_c4	0	0	34	50	51	47	44	46	0 ...	43	7	3	2	0	2022-02-16 07:03:00	Wednesday	12.673571	13.080714	2022-02-16
44	Product_a8	27	0	0	29	12	0	0	0	0 ...	0	15	9	0	0	2022-03-08 07:08:00	Tuesday	15.331724	28.030345	2022-03-08
53	Product_d7	0	0	0	21	19	19	15	19	20 ...	19	12	4	0	0	2022-02-01 06:30:00	Tuesday	8.190541	19.415135	2022-02-01
69	Product_ac	0	0	0	0	0	1	6	4	0 ...	0	0	1	1	4	2022-03-09 06:33:00	Wednesday	16.680000	21.752000	2022-03-09
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...

[코드 19] 분석 도구를 활용하여 중복값을 제거한 데이터를 valid_df에 저장하기

- ② 총 117개의 제품에 대하여 예정 수주량, 작년 수주량, 예상 수주량이 95일동안 연속적으로 수집되었다. 그러므로 'Product_Number' 별 'Date' 가 중복없이 존재하며 각 'Product_Number' 별 길이가 95가 맞는지 확인한다. 'groupby' 메서드를 사용하여 'Product_Number' 별 행의 길이를 구하고 95인 경우를 추출한다.

```
valid_df.groupby('Product_Number').filter(lambda g:g['Product_Number'].size==95)
```

Product_Number	T일 예정 수주 량	T+1 일 예 정 수 주량	T+2 일 예 정 수 주량	T+3 일 예 정 수 주량	T+4 일 예 정 수 주량	작년 T 일 예 정 수 주량	작년 T+1일 예 정 수 주량	작년 T+2일 예 정 수 주량	작년 T+3일 예 정 수 주량	...	T일 예 상 수주 량	T+1 일 예 상 수 주량	T+2 일 예 상 수 주량	T+3 일 예 상 수 주량	T+4 일 예 상 수 주량	DateTime	DoW	Temperature	Humidity	Date
9	Product_ad	0	0	46	16	35	52	42	50	0 ...	32	43	135	196	0	2022-02-16 07:03:00	Wednesday	12.673571	13.080714	2022-02-16
40	Product_c4	0	0	34	50	51	47	44	46	0 ...	43	7	3	2	0	2022-02-16 07:03:00	Wednesday	12.673571	13.080714	2022-02-16
44	Product_a8	27	0	0	29	12	0	0	0	0 ...	0	15	9	0	0	2022-03-08 07:08:00	Tuesday	15.331724	28.030345	2022-03-08
53	Product_d7	0	0	0	21	19	19	15	19	20 ...	19	12	4	0	0	2022-02-01 06:30:00	Tuesday	8.190541	19.415135	2022-02-01
69	Product_ac	0	0	0	0	0	1	6	4	0 ...	0	0	1	1	4	2022-03-09 06:33:00	Wednesday	16.680000	21.752000	2022-03-09
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...

[코드 20] 분석 도구를 활용하여 'Product_Nuber' 별 95일동안 수집된 데이터 추출

```
len(valid_df.groupby('Product_Number').filter(lambda g:g['Product_Number'].size==95))
```

9690

[코드 21] 분석 도구를 활용하여 정확한 데이터 개수 구하기

- ③ 전처리 이전의 정확성 지수 값

: 중복 데이터를 제거한 데이터 내에서 정확성을 가지는 데이터는 총 9690개이며 그 결과, 정확성 지수는 91.21%인 것을 확인할 수 있다.

```
valid_len=len(valid_df.groupby('Product_Number').filter(lambda g:g['Product_Number'].size==95))
print("정확성 지수 : %.2f%%"%(valid_len/len(valid_df)*100)) #결과는 아래에서 확인 가능하다.
```

정확성 지수 : 91.21%

[코드 22] 분석 도구를 활용하여 데이터 정확성 확인하기

④ 전처리 이후의 정확성 지수 값

: 본 데이터에는 총 117개 제품의 수주 로그가 기록되어 있다. 본 실습의 단계 [③-5]에서 설명한 것처럼, 전처리 과정에서 95일간 데이터가 수집되지 않은 일부 데이터를 제거한다. 동일 기간 동안 연속적으로 데이터가 수집된 'Product_Number'만 추출하여 분석에 사용한다. 95일 이하의 기간 동안 수집된 'Product_Number'를 제거한 데이터의 정확성 지수는 아래와 같이 계산할 수 있다.

[단계 ③] 데이터 정제(전처리) 과정을 거친 데이터 valid_df로 유일성 품질지수를 구한다

```
df['Date'] = pd.to_datetime(df['DateTime']).dt.date
valid_df = df.drop_duplicates(['Product_Number', 'Date'], keep='last')
product_list = valid_df['Product_Number'].unique()
drop_list = []
for product in product_list:
    if valid_df['Product_Number'].value_counts()[product] != 95:
        drop_list.append(product)
valid_df.drop(valid_df[valid_df.Product_Number.isin(drop_list)].index, inplace=True)
valid_df.reset_index(drop=True, inplace=True)
valid_df['Product_Number'].value_counts()
valid_len=len(valid_df.groupby('Product_Number').filter(lambda g:g['Product_Number'].size==95))
print("정확성 지수 : %.2f%%"%(valid_len/len(valid_df)*100)) #결과는 아래에서 확인 가능하다.
```

정확성 지수 : 100.00%

[코드 23] 분석 도구를 활용하여 전처리 후, 데이터 정확성 확인하기

▶ 무결성 품질 지수 = $(1 - (\text{유일성, 유효성, 일관성 지수 중 100\%가 아닌 지수 개수} / 3)) \times 100$

: 본 데이터에서 유일성은 97.91%, 유효성과 일관성은 100%로, 세 가지 지수 중 유일성 지수가 100%를 만족하지 못하므로 무결성 품질 지수는 66.67%이다.

▶ 가중치지수 = 품질 지수 * 가중치

: 데이터의 특성에 따라 품질 지수의 중요도는 달라진다. 활용되는 데이터의 중요한 품질 지수가 무엇인지를 평가하고, 데이터의 특성에 따라 적절한 가중치를 부여한다.

▶ 데이터 품질 지수 = $\sum$ 가중치지수

: 최종적으로 구한 6가지 데이터 품질 지수에 적절한 가중치를 부여하여 품질 지수를 구한다. 품질 지수 가중치는 활용되는 데이터의 특성에 따라 부여할 수 있다. 중요한 품질지수가 무



엇인지를 평가하고, 데이터의 특성에 따라 가중치를 부여한다.

본 데이터는 수주량에 대한 로그 데이터라는 특성상 하루에 평균적으로 3~4회 주문이 기록된다. 즉, 유일성에 위반하는 데이터가 존재할 수 있다는 특성이 있기에 유일성의 가중치와 품질 지수 산출 과정에 유일성을 고려하는 무결성의 가중치를 10%로 부여하여 상대적으로 적게 가중치를 둔다. 이 외에 완전성, 유효성, 일관성, 정확성은 동일하게 중요하다는 가정 하에 가중치를 동일하게 각각 20%로 부여하였다.

: 제조 데이터 품질 지수 결과는 아래와 같다. 가중치를 부여하지 않고 6가지 품질 지수(완전성, 유일성, 유효성, 일관성, 정확성, 무결성)를 평균을 낸 품질 지수는 92.63%다. 가중치 지수는 각각의 품질 지수에 가중치를 적용한 값을 나타내며, 가중치를 적용한 품질 지수는 94.7%로, 가중치를 적용하지 않은 품질 지수 92.63%보다 2.07% 높은 값이 나왔다. 오류율 역시 7.37%에서 5.3%로 줄어들었다.

구분	품질 지수	가중치	가중치지수	오류율
완전성	100%	20%	20%	0%
유일성	97.91%	10%	9.79%	2.09%
유효성	100%	20%	20%	0%
일관성	100%	20%	20%	0%
정확성	91.21%	20%	18.24%	8.79%
무결성	66.67%	10%	6.67%	33.33%
품질 지수	92.63%	100%	94.7%	7.37%(5.3%)



「사출성형 공급망최적화 AI 데이터셋」 분석실습 가이드북